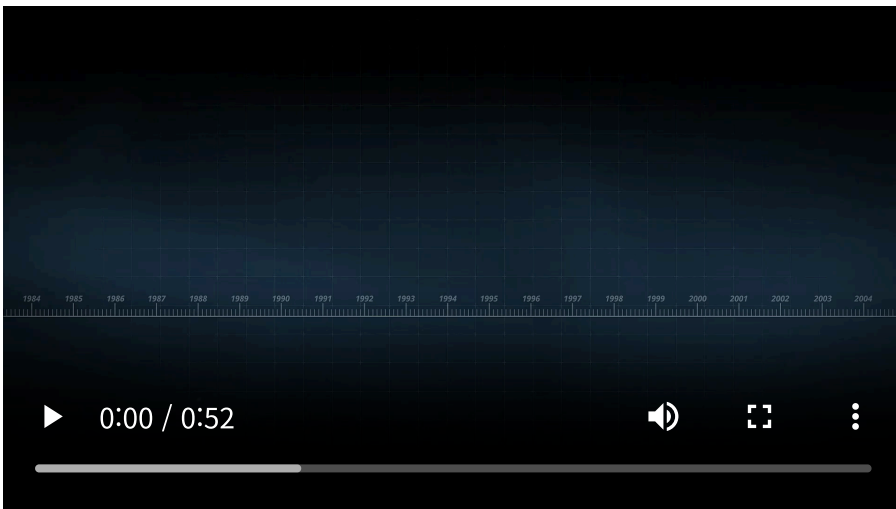


프로젝트 개요

올인원 커뮤니케이션 솔루션, 3D 메신저 기반 협업 플랫폼

제품영상



프로젝트 타입 : 3D 메타버스 기반 협업 플랫폼 개발 (SaaS)

제작사 : 알서포트 주식회사

서비스 링크 : <https://www.rfice.com/ko/>

제작인원 : 35명(2025.10월 기준)

- 유니티 클라이언트 개발자 9명
- 3D 디자인 5명
- 기반기술 2명
- 모바일 개발 3명
- 프론트엔드 2명
- 백엔드 3명
- 클라우드 6명
- PM 2명
- UX/UI 3명

참여 역할 : 유니티 클라이언트 개발

프로젝트 참여 기간 : 2022.10 - 2025.12

플랫폼 : WebGL, iOS, Android

에지 믹 드그 · Unity3D C# Git

메인 주소 : [Unity3D, C#, C++](#)

네트워킹 라이브러리 : [Fusion2-Shared](#) 

기여 요약

클라이언트 개발(최신순)

- 1 [MyRoomEditor](#) : 하우스징 시스템
- 2 **PrefabLocalIdHelper** : 프리팹 고유 ID 관리 유틸리티 (작성중)
- 3 **NetworkedScreen** : 실시간 파일공유 오브젝트 (작성중)
- 4 **SettingManagementSystem** : 시스템 설정 시스템 (작성중)
- 5 **PlayerVehicle** : 플레이어 탈것 시스템 (작성중)
- 6 **MobileController** : 모바일 플랫폼 컨트롤러 (작성중)
- 7 **AssistCamera** : 서브카메라 유틸리티(작성중)

하우스징 시스템(MyRoomEditor System) 소개

WebGL 런타임 3D 개인 공간 편집 및 공유 시스템(하우스징 시스템)

1. 기능 개요

- **MyRoomEditor**는 Rfice 어플리케이션의 하위 콘텐츠 중 하나로 Unity 기반의 3D 개인 룸 커스터마이징 시스템으로, 사용자가 가상 공간에 가구와 장식 요소들을 자유롭게 배치하고 편집할 수 있는 통합 편집 환경을 제공합니다.
- **개발 기간**: 2024.11 ~ 2025.03
- **시스템 개발 인원**: 7인 (유니티 클라이언트 2인, 3D 디자인 3인, 백엔드 1인, UX/UI 1인)

개발 배경 및 요구사항

- 사용자가 개인 공간을 3D 환경에서 직관적으로 커스터마이징하고 다른 사용자와 공유하는 소셜/크리에이티브 공간 제공
- 다양한 가구와 장식 요소들을 실시간으로 배치하고 편집하는 기능
- 가구의 위치(**Vector3**), 회전(**Quaternion**) 및 커스텀 데이터(**Material**, **URL**)을 JSON 데이터로 직렬화하여 서버와 통신 가능한 형태로 파싱
- 편집 화면에서 '꾸미기 종료' UI 조작 시 서버에 편집 데이터를 저장하고, 사용자의 룸에 유저가 진입 시 서버에서 데이터를 받아 오브젝트를 동적으로 생성/배치하는 방식으로 제작
- MVP 패턴과 의존성 주입을 활용한 확장 가능한 아키텍처 구축
- EA의 심즈4 건축 시스템을 참고하여 동작 플로우 제작
- 사용자로부터 이미지 URL 또는 파일을 받아 런타임에서 텍스처를 로드하고, 해당 텍스처를 액자 오브젝트의 Material에 적용하여 동적으로 커스터마이징

주요 기능

📄 : 직접구현 기능, 👤 : 클라이언트 팀원 제작 기능

기능	설명
👤 3D 공간 배치	UI조작 및 마우스 좌측 클릭을 통해 가구를 원하는 위치에 배치 및 편집 기능.
👤 3D 공간 카메라 조작	키보드 WASD를 통한 카메라 이동, 마우스 우측 클릭을 통해 카메라 회전, 마우스 휠을 통한 확대/축소 기능.

 오브젝트 선택	Raycast 를 통해 선택 가능한 오브젝트 필터링 및 마우스 좌클릭으로 선택.
 편집 UI	선택한 오브젝트의 데이터를 통해 사용 가능한 편집 기능을 UI로 표시.
 오브젝트 편집	오브젝트가 선택 될 시 상단 중앙에 편집UI를 표시하고 편집UI를 조작해 선택된 오브젝트 이동 및 회전, 삭제.
 오브젝트 콘텐츠 편집	특정 오브젝트 선택시 편집UI에 컬러 변경, 이미지 업로드 UI를 표시하고 UI를 통해 Material 및 Texture 변경.
 조작 기즈모	배치된 오브젝트를 이동 및 회전시 기능에 맞는 기즈모 표시 기능.
 배치 검증	오브젝트의 성격과 공간 제약을 고려한 배치 검증 기능.
 상태 표현	Material 색상 변화를 이용해 배치 검증 상태 및 선택된 오브젝트 표현 기능.
 실행 취소	편집중인 오브젝트를 ESC 키를 통해 편집 이전 상태로 복원 기능.
 데이터 저장	유저 커스터마이징 결과의 동적 로드 가능하게 서버에 저장하고 룸에 진입시 해당 데이터를 로드.
 동적 오브젝트 생성	룸에 진입시 유저의 커스터마이징 데이터를 통해 오브젝트 동적 생성 및 배치.
 아이템 데이터 파싱	csv로 작성된 아이템 데이터를 유니티 ScriptableObject 데이터로 파싱.
 카탈로그 UI	유저가 커스터마이징 가능한 아이템들을 카탈로그 UI를 통해 표시.

제작 기능 영상

1. 마이룸 편집화면 진입, 카메라 조작, 오브젝트 배치모드



2. 마이룸 오브젝트 편집모드 - 선택, 이동, 회전, 색변경





3. 마이룸 오브젝트 그룹화, 이미지 변경, 적용



2. 사용된 기술 요소

직접구현한 요소 및 협업 요소 기술, :협업요소

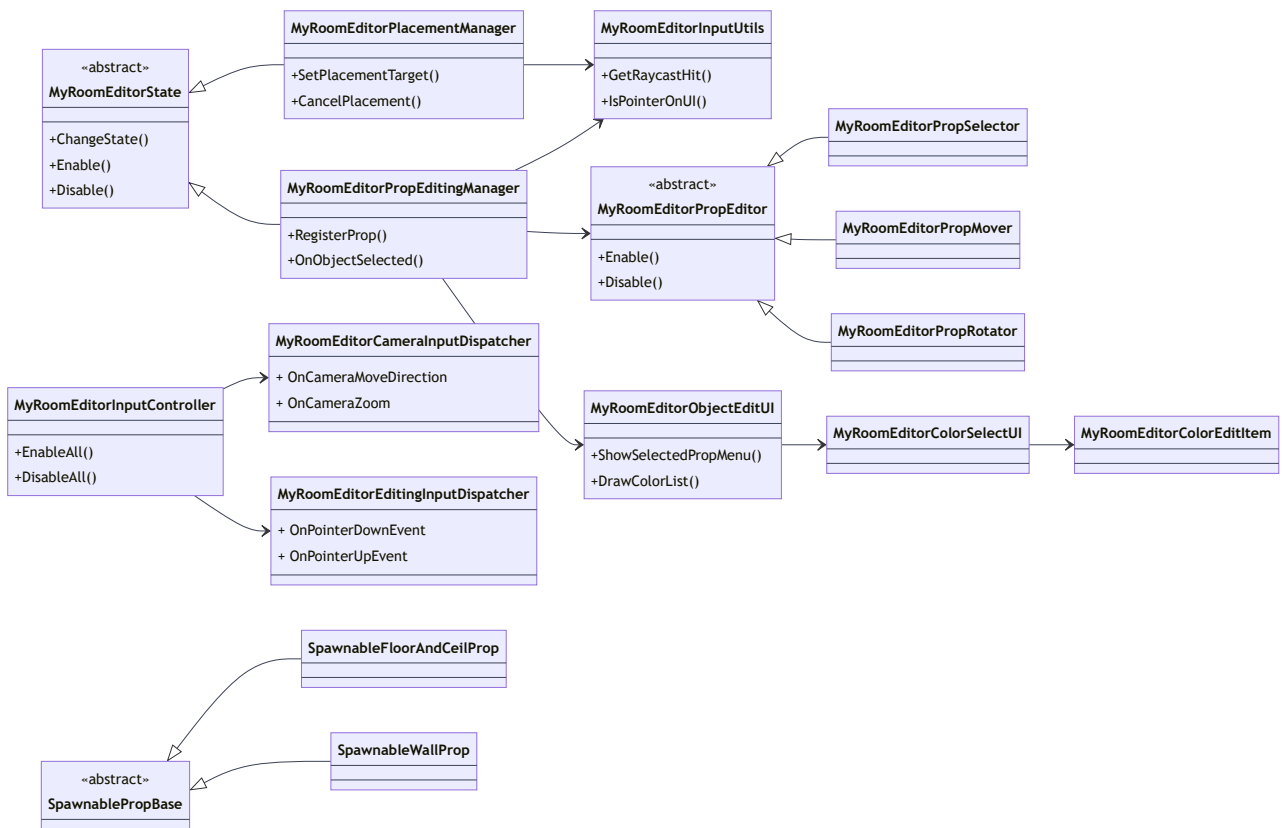
핵심 기술 요소 및 API 활용

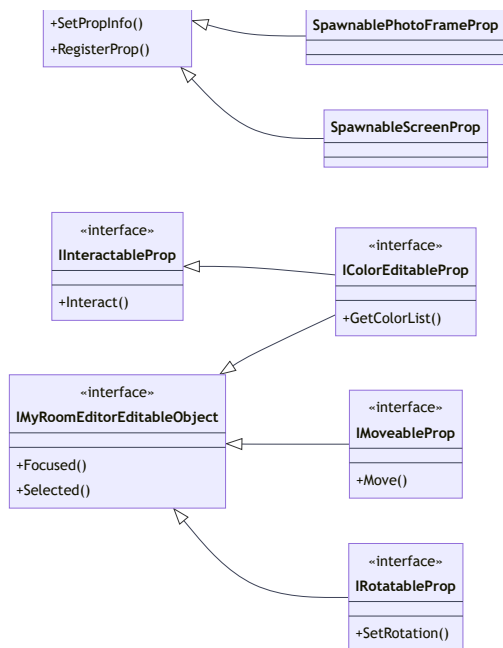
요소	설명
C#	전체 핵심 로직 및 유니티 컴포넌트 구현.
Input System	Action 기반 입력 통합 및 디바이스 독립적 처리.
Physics.Raycast	편집 씬에서 마우스 좌표와 오브젝트간 상호작용 및 선택 로직 구현.
UGUI / EventSystem	동적 UI 구성 및 오브젝트 편집 인터페이스 구현.

설계 활용 패턴

요소	설명
Clean Architecture	Presentation, Domain, Data 계층을 명확히 분리하여 비즈니스 로직과 서버 로직 (데이터 로직)을 완벽하게 분리하여 유지보수 및 확장성을 극대화 할 수 있게 아키텍처 구성.(협업 시스템 아키텍처)
MVP (Model-View-Presenter)	편집 UI(View)와 편집 로직(Presenter)의 책임을 분리하여 UI변경에 유연하게 대응.
전략 패턴 (Strategy Pattern)	오브젝트 배치/선택/편집 등 다양한 편집 모드별 처리 로직을 분리하여 새로운 편집 전략 추가에 유연하게 대응.
옵저버 패턴 (Observer Pattern)	오브젝트 상태 및 사용자 조작에 따른 변화를 관련 모듈에 실시간으로 전달하는 이벤트 시스템 구현.

3. 전체 시스템 구조도(간략)





4. 주요 클래스별 역할 및 관계

i 직접구현한 요소만 기술

상태관리 클래스

클래스

역할

MyRoomEditorState

«abstract»

- 💡 MyRoomEditor의 상태를 관리하는 abstract 클래스.
- 💡 **InputDispatcher**와 상태 핸들을 통해 입력 이벤트를 처리. 상태 변화 시 입력 활성화/비활성화 담당.
- 💡 다양한 편집 상태를 관리하고, 상태 변화 로직을 처리.

MyRoomEditorPlacementManager

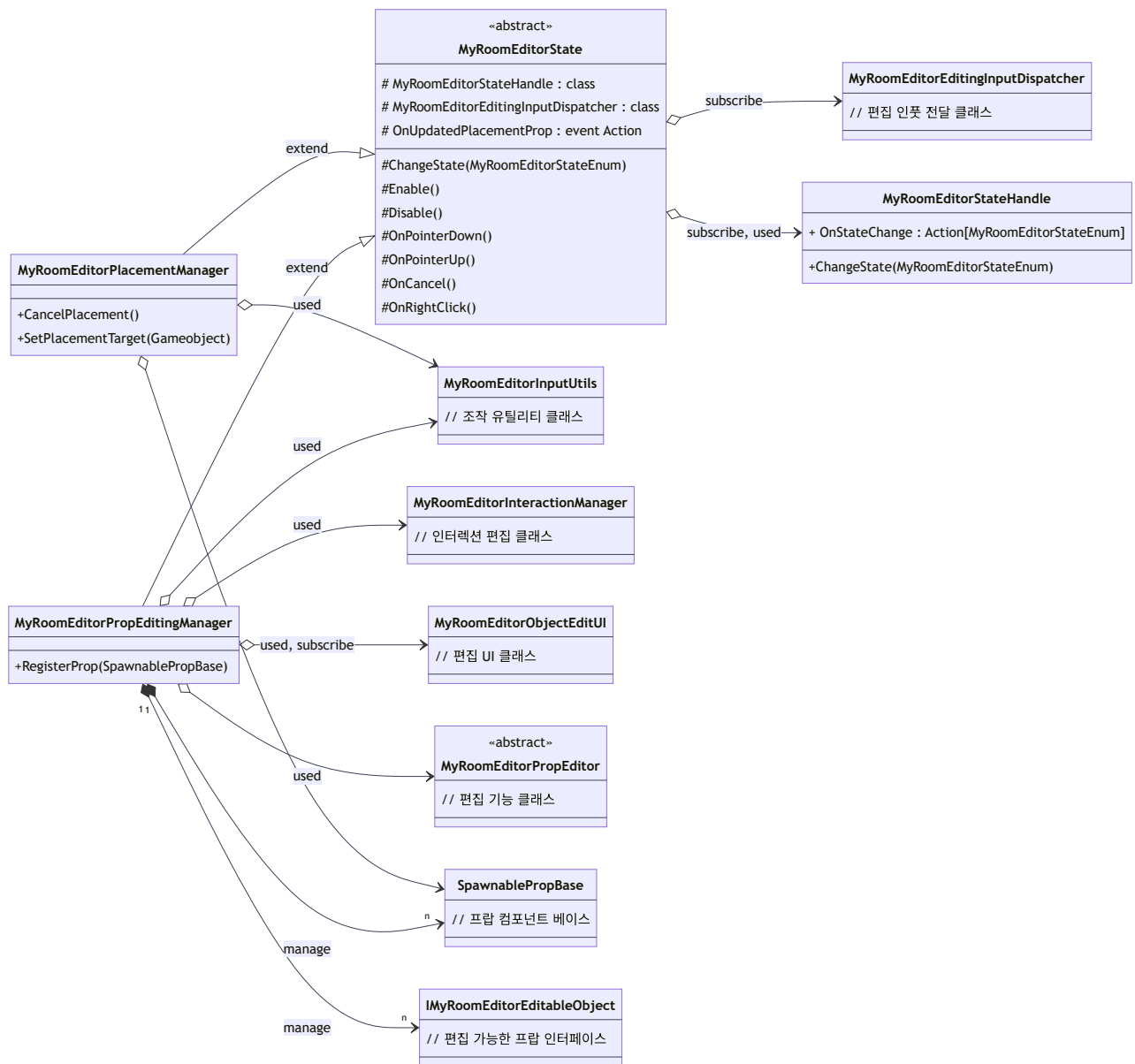
: MyRoomEditorState

- 💡 오브젝트 생성 및 배치 상태를 관리하는 implement 클래스.
- 💡 **Raycast**를 이용해 사용자의 입력에 따라 오브젝트 배치 위치를 검증.
- 💡 **InputDispatcher**를 통해 받는 이벤트와 배치 검증을 통해 오브젝트를 월드에 배치.
- 💡 배치상태에서 이동 및 회전 처리.

MyRoomEditorPropEditingManager

: MyRoomEditorState

- 💡 오브젝트 편집 상태를 관리하는 implement 클래스.
- 💡 오브젝트 선택, 이동, 회전, 색상 변경, 삭제 등의 기능을 수행.
- 💡 서브 클래스 (**MyRoomPropEditor**)와 UI이벤트를 통해 편집 동작을 처리.
- 💡 실행취소를 포함한 상태 스택 관리.



오브젝트 선택/편집기

클래스

역할

MyRoomPropEditor

«abstract»

💡 각각의 편집 기능에서 사용 할 공통 메서드와 멤버를 정의하는 abstract 클래스.

💡 **MyRoomEditorPropEditingManager** 를 통해 받은 입력 및 UI 동작에 대한 처리를 implement 클래스의 override로 호출.

MyRoomPropSelector

: MyRoomPropEditor

💡 오브젝트의 선택을 담당하는 implement 클래스.

💡 **Raycast** 를 이용해 편집 가능한 오브젝트를 선택하고, 결과 이벤트를 발생.

MyRoomPropMover

: MyRoomPropEditor

💡 오브젝트의 이동을 담당하는 implement 클래스.

💡 **Raycast** 를 통해 마우스 포인터 위치에 배치 영역 검증을 수행 하고 오브젝트 이동 이벤트를 발생.

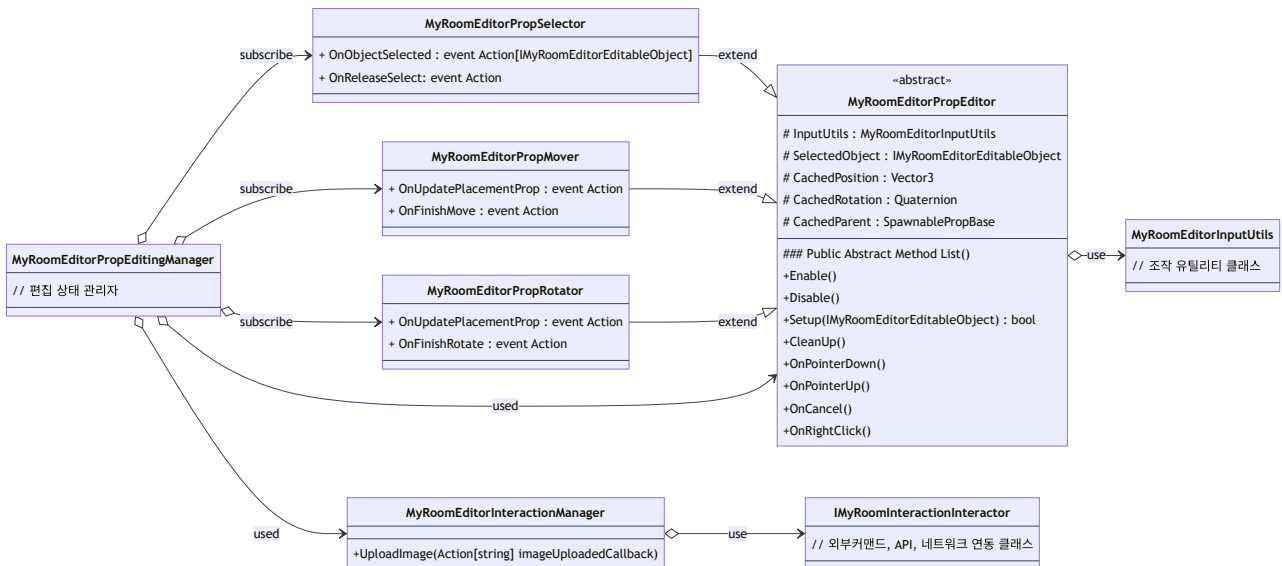
MyRoomPropRotator

: MyRoomPropEditor

- 💡 오브젝트의 회전을 담당하는 implement 클래스.
- 💡 **InputUtils**를 통해 초기 마우스 상태를 캐싱하고 마우스의 이동범위만큼 오브젝트를 좌/우로 회전.

MyRoomEditorInteractionManager

- 💡 오브젝트의 특수 인터랙션을 담당하는 클래스.
- 💡 **MyRoomEditorPropEditingManager**를 통해 특정 인터랙션 호출을 처리.



사용자 입력 및 입력 유틸리티

클래스

역할

MyRoomEditorInputController

- 💡 **InputAction**를 통해 발생하는 입력 이벤트를 각각의 **Dispatcher**로 전달.
- 💡 시스템 활성화 상태 기반 **InputAction** 토글

MyRoomEditorCameraInputDispatcher

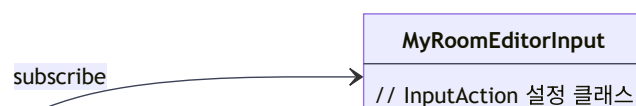
- 💡 카메라 조작 이벤트를 처리하는 클래스. WASD 이동, 마우스 회전, 줌 등 카메라 컨트롤 이벤트 관리.

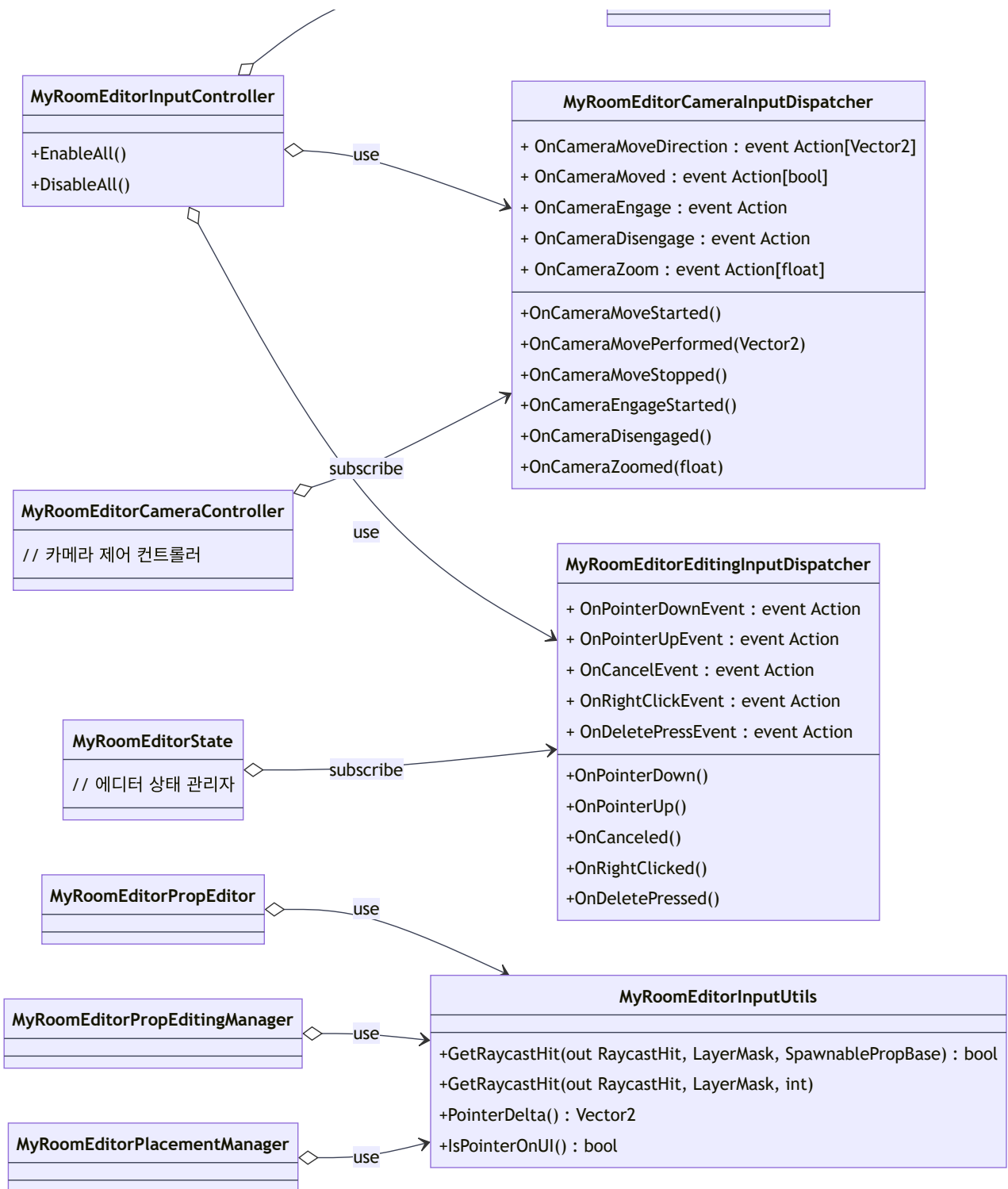
MyRoomEditorEditingInputDispatcher

- 💡 편집 관련 입력 이벤트를 처리하는 클래스. 포인터 이벤트, 삭제, 취소 등 편집 명령 처리.

MyRoomEditorInputUtils

- 💡 입력 유틸리티 클래스. UI 검증, **Raycast** 실행, 포인터 델타 계단 등 입력 관련 헬퍼 함수 제공.





오브젝트 동작 및 정의 Component

클래스

역할

SpawnablePropBase

«abstract»

- 💡 모든 배치 가능한 오브젝트의 abstract 클래스.
- 💡 공용 Property 데이터 설정, 상태 정보 관리, 부모-자식 오브젝트 관계 구성, 상태 표현 등 공통 로직 처리.

SpawnableFloorAndCeilProp

: SpawnablePropBase

: IColorEditableProp

- 💡 바닥과 천장에 배치되는 일반적인 오브젝트 클래스.

: IMoveableProp
: IRotatableProp

SpawnableWallProp

💡 벽에 배치되는 일반적인 오브젝트 클래스.

: SpawnablePropBase
: IColorEditableProp
: IMoveableProp

SpawnablePhotoFrameProp

💡 특수한 인터랙션을 포함한 오브젝트 클래스.

💡 텍스처 업로드 및 이미지 적용 기능 포함.

: SpawnablePropBase
: IPhotoFrameProp
: IMoveableProp

SpawnableScreenProp

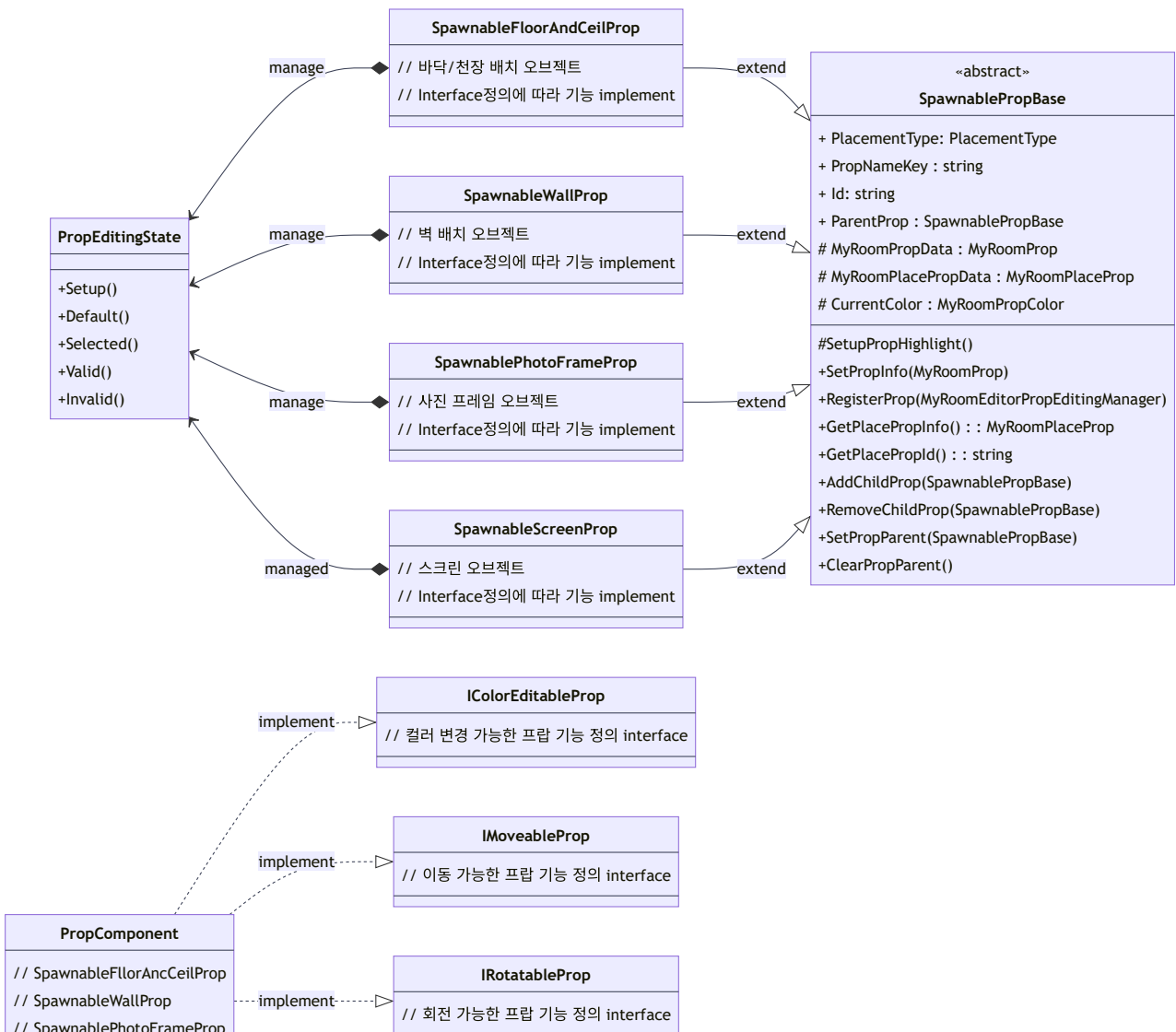
💡 특수한 인터랙션을 포함한 오브젝트 클래스.

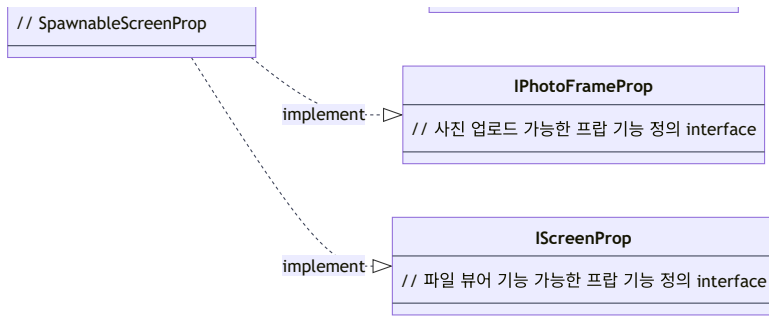
💡 개인공간에서 실시간 파일 공유 기능 모듈(**NetworkedScreen**)을 사용 할 수 있게 세팅.

: SpawnablePropBase
: IScreenProp
: IMoveableProp

PropEditingState

💡 오브젝트 편집 상태에 따라 오브젝트의 **Material** 효과를 표현하는 클래스.





오브젝트 동작 및 정의 Interface

클래스

역할

IMyRoomEditorEditableObject

💡 편집 가능한 오브젝트 정의 및 기능 정의

«interface»

IColorEditableProp

💡 색상 편집 가능한 오브젝트 정의. 컬러 리스트 제공 및 색상 적용 기능 정의.

«interface»

: IMyRoomEditorEditableObject

IMoveableProp

💡 이동 가능한 오브젝트 정의. 배치 영역 검증 및 위치 변경 기능 정의.

«interface»

: IMyRoomEditorEditableObject

IRotateableProp

💡 회전 가능한 오브젝트 정의. 자유 회전 및 스냅 회전 기능 정의.

«interface»

: IMyRoomEditorEditableObject

클래스

역할

InteractableProp

💡 특수한 인터랙션 가능한 오브젝트 정의 및 사용자 인터랙션 기능 정의.

«interface»

IPhotoFrameProp

💡 사진 프레임 정의 및 이미지 설정 기능 정의.

«interface»

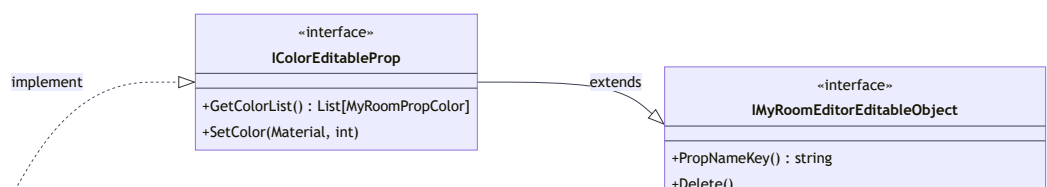
: IInteractableProp

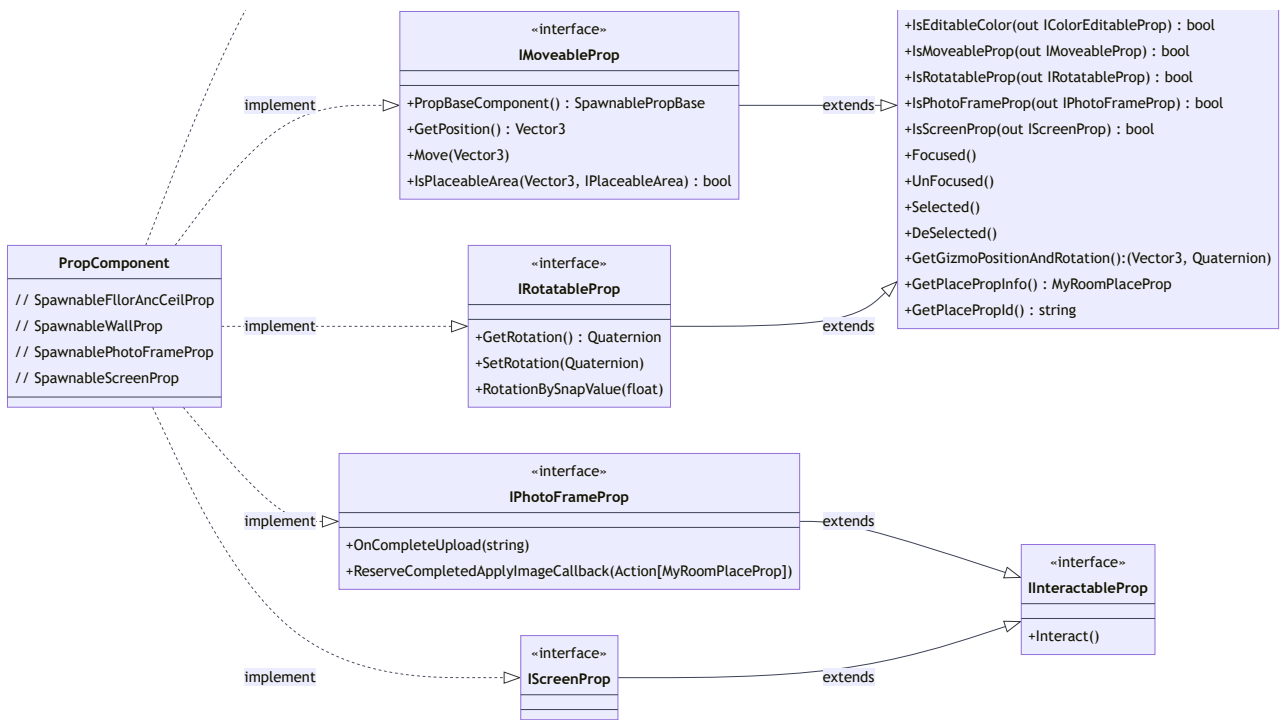
IScreenProp

💡 실시간 파일 공유 모듈 정의.

«interface»

: IInteractableProp





오브젝트 배치 구역 정의 Component 및 Interface

클래스

역할

IPlaceableArea

«interface»

💡 배치 영역 정의 및 배치 영역 기능 정의.

PlacementAreaProp

: IPlaceableArea

💡 바닥, 천장, 벽면과 같은 배치 영역을 정의하는 클래스.

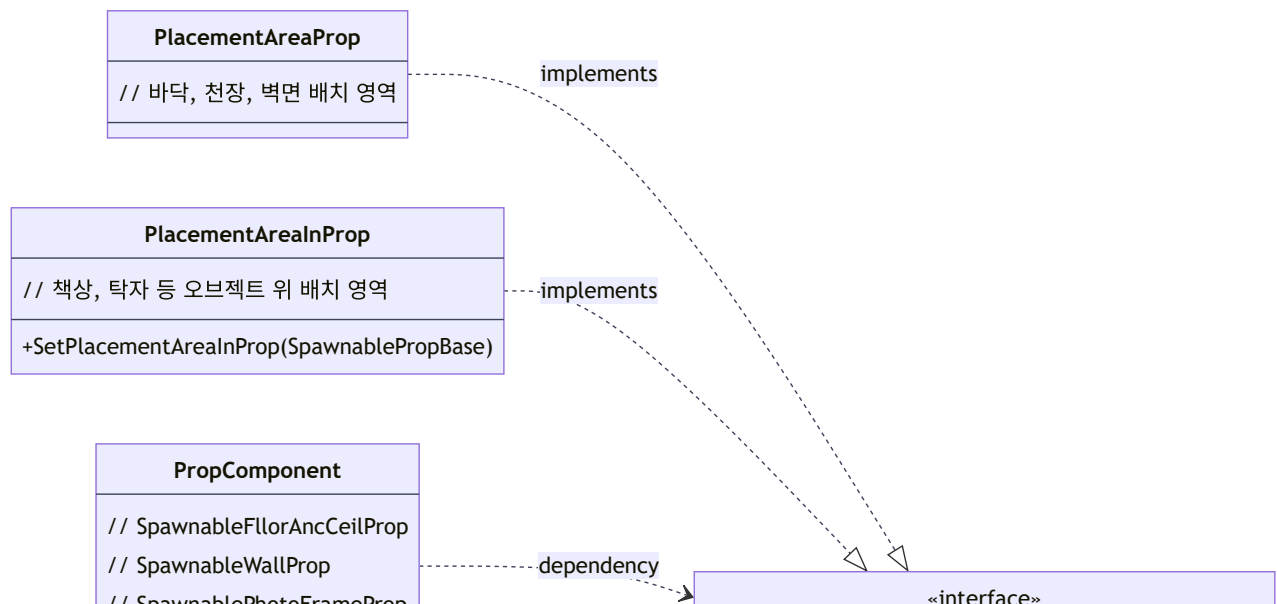
💡 오브젝트가 배치 될 수 있는 공간의 범위와 규칙 지정.

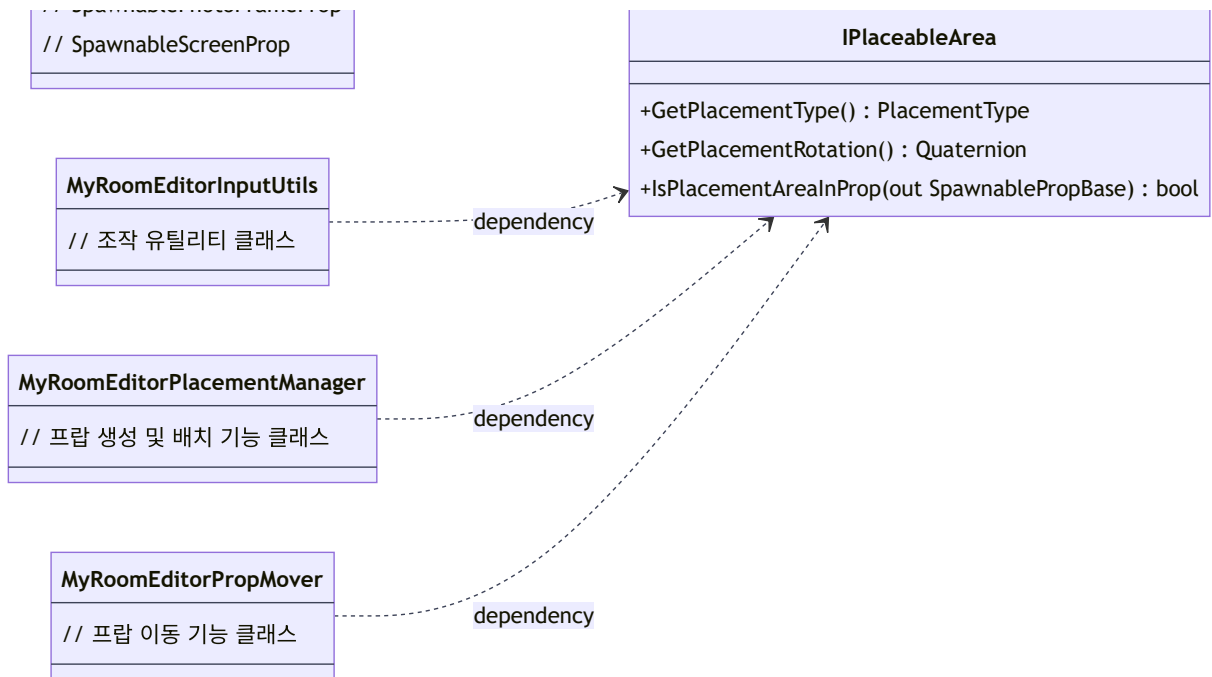
PlacementAreaInProp

: IPlaceableArea

💡 책상, 탁자, 데크 등 오브젝트 위에 배치되는 영역을 관리하는 클래스.

💡 부모 오브젝트 내 자식 오브젝트 처리.





오브젝트 편집UI Component

클래스

역할

MyRoomEditorObjectEditUI

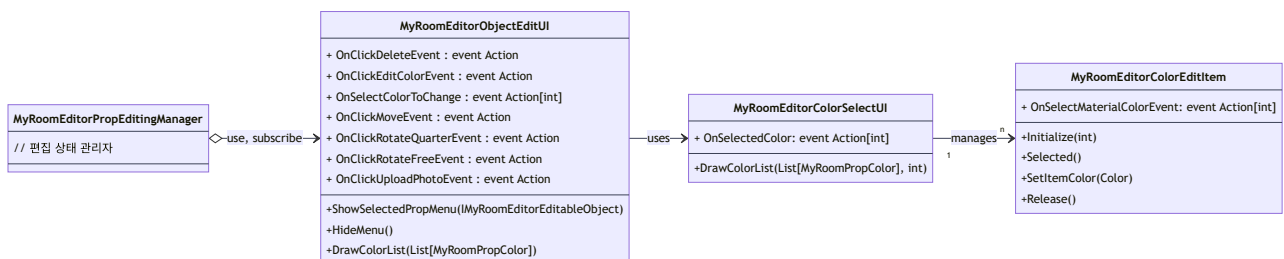
- 💡 오브젝트 편집 메뉴를 표시하고 조작하는 UI클래스.
- 💡 선택된 오브젝트의 이름 및 사용 가능한 편집 기능 표시.
- 💡 편집 기능(이동, 회전, 삭제, 색변경, 인터랙션) 등을 제어하는 버튼과 메뉴 관리.

MyRoomEditorColorSelectUI

- 💡 색상 선택 UI를 처리하는 클래스.
- 💡 사용할 수 있는 색상 옵션을 표시하고 선택 이벤트 처리.

MyRoomEditorColorEditItem

- 💡 선택 가능한 색상 항목을 나타내는 UI 컴포넌트.
- 💡 개별 색상을 표시하고 선택 상태를 관리.



5. 주요 특징

기능의 특징

- **실시간 배치 검증:** 코루틴 기반 실시간 충돌 감지 및 배치 가능성 검증
 - **모듈식 아키텍처:** 각 기능별 독립적 모듈로 구성하여 확장성 및 유지보수성 향상
 - **다양한 편집 모드:** 이동, 회전, 크기 조절, 색상 변경 등 다양한 편집 기능
 - **상태 관리:** 작업 이력을 통한 실행 취소/다시 실행 기능
 - **플랫폼 확장성 고려:** 모바일 환경 확장성을 고려한 입력 및 편집 환경 구성
 - **오브젝트 배치 검증 최적화:** Physics.RaycastNonAlloc를 사용하여 가비지 생성을 방지하고, 결과 수를 제한하여 성능을 최적화.
-

6. UseCase

개인 룸 커스터마이징 시나리오

- 1 **기본 룸 생성:** 빈 룸에서 시작하여 기본 가구 배치
- 2 **가구 배치:** 카테고리별로 가구를 선택하고 원하는 위치에 배치
- 3 **세부 조정:** 선택한 가구의 위치, 회전, 크기 조절
- 4 **색상 커스터마이징:** 가구별 색상 변경으로 개인 취향 반영
- 5 **액세서리 추가:** 사진 프레임, 장식품 등으로 공간 완성
- 6 **저장 및 공유:** 완성된 룸을 저장하고 다른 사용자와 공유

주요 사용처

- 개인 공간 커스터마이징 서비스
- 인테리어 시뮬레이션 플랫폼
- 가구 쇼핑몰의 가상 체험 서비스
- 교육용 공간 디자인 도구
- 게임 내 하우스징 시스템

MyRoomEditorState

MyRoomEditor의 상태를 관리하는 abstract 클래스

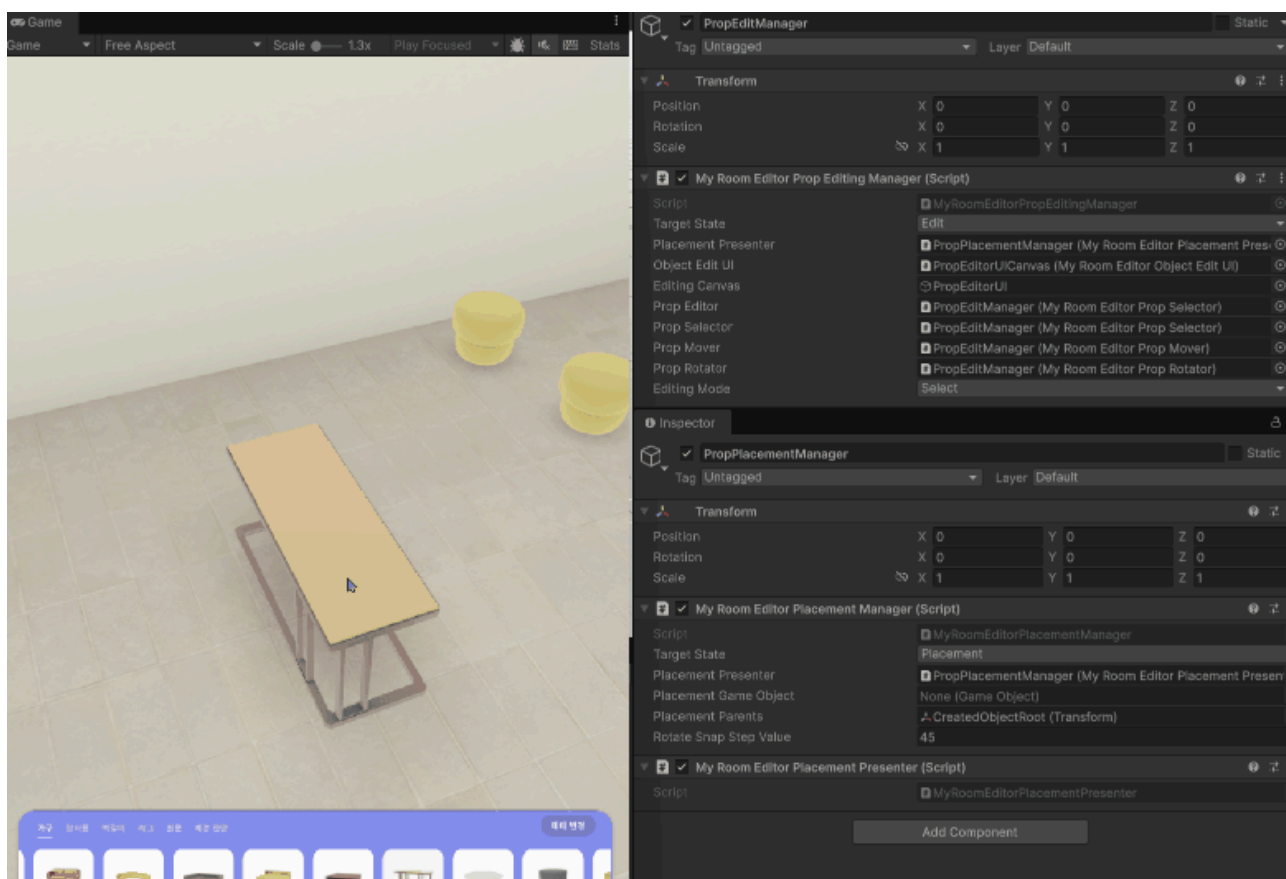
개요

MyRoomEditorState는 MyRoomEditor의 상태를 관리하는 추상 기본 클래스입니다. 이 클래스는 상태 패턴을 구현하여 다양한 편집 상태를 관리하고, 입력 이벤트를 처리하며, 상태 변화 시 입력 활성화/비활성화를 담당합니다.

역할

- MyRoomEditor의 다양한 편집 상태 관리
- 입력 이벤트 처리 및 라우팅
- 상태 변화 시 입력 활성화/비활성화 관리
- 상태 핸들러를 통한 상태 전환 관리

편집상태 전환 관리





선언

```
public abstract class MyRoomEditorState : MonoBehaviour
```

멤버

열거형

```
/// <summary>
/// MyRoomEditor의 상태를 정의하는 열거형
/// </summary>
public enum MyRoomEditorStateEnum
{
    Default,        // 기본 상태
    Placement,      // 오브젝트 배치 상태
    Editing         // 오브젝트 편집 상태
}
```

속성

```
/// <summary>
/// 상태 변경을 처리하는 핸들러. Zenject를 통해 주입되며, 상태 전환 이벤트를 관리.
/// 이 클래스가 특정 상태로 전환될 때 필요한 프로세스를 수행.
/// </summary>
[Inject]
protected MyRoomEditorStateHandle MyRoomEditorStateHandle;

/// <summary>
/// 편집 입력 디스패처. 마우스 및 키보드 입력 이벤트를 처리.
/// 상태가 활성화될 때 입력 이벤트를 구독하고, 비활성화될 때 해제.
/// </summary>
[Inject]
protected MyRoomEditorEditingInputDispatcher EditorInputDispatcher;

/// <summary>
/// 오브젝트 배치 정보 프리젠테이션.
/// 서브클래스에서 오브젝트 생성/삭제/수정 동작을 제어할 때 호출하여 오브젝트 데이터 업데이트.
/// </summary>
[SerializeField]
protected MyRoomEditorPlacementPresenter placementPresenter;
```

메서드

```

/// <summary>
/// 마우스 왼쪽 버튼이 눌렸을 때 호출되는 메서드.
/// 서브클래스에서 해당 상태의 클릭 동작을 구현 (예: 오브젝트 선택, 배치 등).
/// </summary>
protected abstract void OnPointerDown();

/// <summary>
/// 마우스 왼쪽 버튼이 떼어졌을 때 호출되는 메서드.
/// 서브클래스에서 드래그 종료나 클릭 완료 동작을 구현.
/// </summary>
protected abstract void OnPointerUp();

/// <summary>
/// 취소 입력 (예: ESC 키)이 발생했을 때 호출되는 메서드.
/// 현재 작업을 취소하거나 이전 상태로 돌아가는 프로세스를 구현.
/// </summary>
protected abstract void OnCancel();

/// <summary>
/// 마우스 오른쪽 버튼이 클릭되었을 때 호출되는 메서드.
/// 컨텍스트 메뉴 표시나 특수 동작을 수행.
/// </summary>
protected abstract void OnRightClick();

/// <summary>
/// 이 상태가 활성화될 때 호출되는 메서드.
/// 상태별 초기화 프로세스를 수행 (예: UI 표시, 입력 설정 등).
/// </summary>
protected abstract void Enable();

/// <summary>
/// 이 상태가 비활성화될 때 호출되는 메서드.
/// 상태별 정리 프로세스를 수행 (예: UI 숨김, 리소스 해제 등).
/// </summary>
protected abstract void Disable();

```

코드 스니펫

상태 변경 프로세스

```

/// <summary>
/// 서브클래스에서 상태 변경을 요청할 때 호출되는 protected 메서드.
/// 상태 머신을 통해 전역 상태를 변경하며, 모든 상태 클래스의 OnChangedState가 호출됨.
/// </summary>
/// <param name="state">전환할 목표 상태 (MyRoomEditorStateEnum 값)</param>
protected void ChangeState(MyRoomEditorStateEnum state)
{
    MyRoomEditorStateHandle.ChangeState(state);
}

/// <summary>

```

```

/// 상태 변경 이벤트가 발생할 때 호출되는 private 메서드.
/// 현재 상태가 이 클래스의 targetState와 일치하는지 확인하여
/// 입력 이벤트 구독/해제와 함께 Enable/Disable 메서드를 호출.
/// 상태 머신 패턴의 핵심 프로세스로, 상태별 동작을 자동으로 관리.
/// </summary>
/// <param name="state">새로 변경된 전역 상태</param>
private void OnChangedState(MyRoomEditorStateEnum state)
{
    // 현재 상태가 이 클래스의 대상 상태와 일치하면 활성화
    if (state == targetState)
    {
        InputEnable(); // 입력 이벤트 구독
        Enable(); // 서브클래스별 활성화 프로세스 실행
    }
    // 일치하지 않으면 비활성화
    else
    {
        InputDisable(); // 입력이벤트 해제
        Disable(); // 서브클래스별 비활성화 프로세스 실행
    }
}
}

```

기능 설명

상태 관리

- 상태 핸들러를 통해 상태 전환을 중앙에서 관리
- `ChangeState()`: 현재 상태를 변경하고 상태 핸들러에 알림
- `Enable()/Disable()`: 상태 활성화/비활성화 및 입력 디스패처 구독 관리
- 추상 메서드를 통한 상태별 입력 처리 구현

입력 이벤트 처리

- `OnPointerDown()/OnPointerUp()`: 포인터 입력 처리
- `OnCancel()`: 취소 입력 처리 (ESC 키 등)
- `OnRightClick()`: 우클릭 입력 처리

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `MyRoomEditorStateHandle`에 의존.
- `MyRoomEditorEditingInputDispatcher`에 의존.
- `MyRoomEditorPlacementPresenter`에 의존.
- `MyRoomEditorState`의 서브 클래스
 - `MyRoomEditorPlacementManager`: 오브젝트 생성 및 배치 상태 관리
 - `MyRoomEditorPropEditingManager`: 오브젝트 편집 상태 관리

사용 예시

MyRoomEditorPropEditingManager에서 상속받아 편집 상태 구현

```
public class MyRoomEditorPropEditingManager : MyRoomEditorState
{
    protected override void Enable()
    {
        // 편집 상태 활성화 로직
        Debug.Log("Edit mode enabled");
    }

    protected override void Disable()
    {
        // 편집 상태 비활성화 로직
        Debug.Log("Edit mode disabled");
    }

    protected override void OnPointerDown()
    {
        // 포인터 다운 처리 로직
    }

    // 다른 추상 메서드 구현...
}
```

관련 클래스

- MyRoomEditorStateHandle
- MyRoomEditorEditingInputDispatcher
- MyRoomEditorPlacementManager
- MyRoomEditorPropEditingManager

MyRoomEditorPlacementManager

오브젝트 생성 및 배치 상태를 관리하는 클래스

개요

`MyRoomEditorPlacementManager`는 `MyRoomEditor`에서 `MyRoomEditorState`를 상속받아 오브젝트 생성/배치 작업을 관리하는 상태 기반 클래스입니다. `MyRoomEditorInputUtils`를 사용하여 배치 가능한 영역을 감지하고, 사용자의 입력에 따라 프로퍼티의 위치와 회전을 실시간으로 조정합니다. 배치 완료 시 프로퍼티를 영구적으로 배치하고 관련 데이터를 저장합니다.

역할

- 오브젝트 배치 대상 설정 및 초기화
- 레이캐스트 기반 배치 위치 검증 및 이동 처리(`MyRoomEditorInputUtils` 사용)
- 포인터 입력을 통한 자유 회전 처리
- 배치 완료 및 취소 로직 관리
- 배치 상태와 UI 상태 간 동기화
- 배치된 프로퍼티의 영구 저장 및 부모-자식 관계 설정

선언

```
public class MyRoomEditorPlacementManager : MyRoomEditorState
```

멤버

속성

```
/// <summary>  
/// 입력 유틸리티 클래스. 마우스 입력 및 Raycast 처리를 담당.  
/// Zenject를 통해 주입되며, 입력 관련 공통 로직을 제공.  
/// </summary>  
[Inject]  
private MyRoomEditorInputUtils _inputUtils;  
  
/// <summary>  
/// 현재 배치 중인 게임 오브젝트.
```

```

/// 배치 과정에서 실시간으로 위치와 회전을 조정.
/// </summary>
private GameObject placementGameObject;

/// <summary>
/// 배치된 오브젝트들의 부모 트랜스폼.
/// 배치 완료 시 오브젝트를 이 부모 아래로 이동시켜 계층 구조 유지.
/// </summary>
private Transform placementParents;

/// <summary>
/// 우클릭 시 회전할 스냅 각도 (기본값: 45도).
/// Inspector에서 조정 가능.
/// </summary>
[SerializeField]
private float rotateSnapStepValue = 45f;

/// <summary>
/// 현재 배치 중인 SpawnablePropBase 컴포넌트.
/// 배치 타입과 속성을 확인하기 위해 사용.
/// </summary>
private SpawnablePropBase _currentSpawnedProp;

/// <summary>
/// 배치 가능 위치를 검증하는 코루틴.
/// 실시간으로 Raycast를 수행하여 배치 가능성을 체크.
/// </summary>
private Coroutine _detector;

/// <summary>
/// 회전 처리를 담당하는 코루틴.
/// 마우스 드래그 중 실시간 회전을 수행.
/// </summary>
private Coroutine _rotator;

/// <summary>
/// 현재 위치에서 배치 가능한지 여부.
/// 검증 결과에 따라 배치 가능/불가능 상태를 나타냄.
/// </summary>
private bool _canPlaceable;

/// <summary>
/// 오브젝트가 이동/회전 중인지 여부.
/// 특정 조작의 실행 조건으로 사용.
/// </summary>
private bool _isMoving;
private bool _isRotating;

/// <summary>
/// Raycast 대상 레이어 마스크.
/// Wall과 Ground 레이어만 검출하도록 설정.
/// </summary>
private int _targetLayer;

```

메서드

```
/// <summary>
/// 배치를 취소하는 public 메서드.
/// 외부에서 배치 취소를 요청할 때 사용.
/// </summary>
public void CancelPlacement()

/// <summary>
/// 배치 대상을 설정하고 배치 모드로 전환.
/// 기존 배치 오브젝트가 있을 경우 정리하고 새 오브젝트로 배치 검증 시작.
/// </summary>
/// <param name="targetObject">배치할 대상 게임 오브젝트</param>
public void SetPlacementTarget(GameObject targetObject)

/// <summary>
/// 배치 가능 위치를 실시간으로 검증하는 코루틴.
/// 오브젝트의 이동 중 배치 가능성을 지속적으로 체크.
/// </summary>
/// <param name="targetObject">검증할 대상 오브젝트</param>
private IEnumerator CoDetectPlacement(GameObject targetObject)

/// <summary>
/// 특정 이동 가능 오브젝트의 배치 가능 위치를 검증.
/// Raycast를 통해 충돌 지점을 찾고, 배치 영역 인터페이스로 검증.
/// </summary>
/// <param name="moveableProp">검증할 이동 가능 오브젝트</param>
/// <returns>배치 가능한지 여부</returns>
private bool PlaceableHitPoint(IMoveableProp moveableProp)

/// <summary>
/// 마우스 드래그 중 실시간 회전을 처리하는 코루틴.
/// 마우스 이동량에 따라 Y축 회전을 적용.
/// </summary>
private IEnumerator CoRotationHandle()

/// <summary>
/// 실제 오브젝트 배치를 수행.
/// 배치 정보를 설정하고 부모 관계를 구성한 후 편집 모드로 전환.
/// </summary>
private void Placement()

/// <summary>
/// 배치 관련 동작을 정리하고 편집 모드로 전환.
/// 코루틴 중단, 플래그 초기화, 상태 변경.
/// </summary>
private void ReleasePlacementBehaviour()
```

코드 스니펫

오브젝트 배치 프로세스

```

public void SetPlacementTarget(GameObject targetObject)
{
    ChangeState(MyRoomEditorStateEnum.Placement); // 배치 상태로 전환
    targetObject.SetActive(false); // 임시 비활성화
    if (placementGameObject != null) Destroy(placementGameObject); // 기존 오브젝트 파괴
    _isMoving = true; // 이동 모드 활성화
    _detector = StartCoroutine(CoDetectPlacement(targetObject)); // 배치 검증 코루틴 시작
}

private IEnumerator CoDetectPlacement(GameObject targetObject)
{
    targetObject.SetActive(true); // 오브젝트 활성화
    placementGameObject = targetObject; // 배치 오브젝트 설정

    // SpawnablePropBase 컴포넌트 확인
    if (placementGameObject.TryGetComponent(out _currentSpawnedProp) == false)
    {
        Debug.Log($"[MyRoomEditorPlacementManager] Cant found SpawnablePropBase Component");
        yield break; // 컴포넌트 없으면 중단
    }

    // IMoveableProp 인터페이스 확인
    if (_currentSpawnedProp.TryGetComponent(out IMoveableProp moveableProp) == false)
    {
        Debug.Log($"[MyRoomEditorPlacementManager] Can't Placement Object : {targetObject.name}");
        yield break; // 인터페이스 없으면 중단
    }

    Debug.Log($"[MyRoomEditorPlacementManager] Start PropPlacement {targetObject.name}");

    // 이동 중 배치 가능성 검증 루프
    while (_isMoving)
    {
        _canPlaceable = PlaceableHitPoint(moveableProp); // 배치 가능성 체크
        yield return null; // 다음 프레임까지 대기
    }
}

private bool PlaceableHitPoint(IMoveableProp moveableProp)
{
    // Raycast 수행
    if (!_inputUtils.RaycastHit(out var raycastHit, _targetLayer, moveableProp.transform.position, _raycastDistance))
    {
        return false;
    }

    // 배치 영역 인터페이스 확인 및 검증
    return raycastHit.transform.TryGetComponent(out IPlaceableArea placeableArea) &&
        moveableProp.IsPlaceableArea(raycastHit.point, placeableArea);
}

protected override void OnPointerUp()
{
    if (_canPlaceable) Placement(); // 배치 수행
}

```

```

private void Placement()
{
    var propBase = placementGameObject.GetComponent<SpawnablePropBase>(); //
    placementGameObject.transform.parent = placementParents; // 부모 설정

    // 배치 정보 설정 (프리젠터를 통해 데이터 생성)
    propBase.SetPlacePropInfo(_placementPresenter.CreatePlaceProp(propBase.Id,

    // 부모-자식 관계 구성
    propBase.ParentProp?.AddChildProp(propBase);

    // 선택 해제
    propBase.GetEditableObject().Deselected();

    // 배치 동작 정리
    ReleasePlacementBehaviour();

    // 이벤트 호출
    OnUpdatedPlacementProp?.Invoke();
}

```

회전 처리 코루틴

```

protected override void OnPointerDown()
{
    if (_inputUtils.IsPointerOnUI()) return; // UI 위에서 클릭 시 무시
    if (!_canPlaceable) return; // 배치 불가능 시 무시
    if (_currentSpawnedProp.PlacementType == PlacementType.Wall) return; // 벽

    _isRotating = true; // 회전 모드 활성화
    _isMoving = false; // 이동 모드 비활성화
    _rotator = StartCoroutine(CoRotationHandle()); // 회전 코루틴 시작
}

private IEnumerator CoRotationHandle()
{
    while (_isRotating) // 회전 모드 중
    {
        var rotateAngle = _inputUtils.PointerDelta().x; // 마우스 X축 이동량
        placementGameObject.transform.Rotate(0, -rotateAngle, 0); // Y축 회전
        yield return null; // 다음 프레임까지 대기
    }
}

```

기능 설명

배치 대상 설정

- SetPlacementTarget()**: 배치할 오브젝트를 설정하고 Placement 상태로 전환. 기존 배치 오브젝트를 정리하고 감지 코루틴을 시작.

실시간 배치 감지

- `CoDetectPlacement()`: 배치 중인 오브젝트의 위치를 실시간으로 업데이트. 레이캐스트를 사용하여 배치 가능한 영역을 감지.

배치 검증

- `PlaceableHitPoint()`: 레이캐스트 결과를 기반으로 배치 가능 여부를 판단. `IPlaceableArea` 인터페이스를 구현한 영역에서만 배치 가능.

입력 처리

- `OnPointerDown()`: 배치 중 클릭 시 회전 모드로 전환합니다.
- `OnPointerUp()`: 배치 가능 시 배치를 완료합니다.
- `OnRightClick()`: 90도 회전 스냅을 적용합니다.
- `OnCancel()`: 배치를 취소하고 상태를 초기화합니다.

회전 처리

- `CoRotationHandle()`: 포인터 델타를 기반으로 자유 회전을 수행합니다.

배치 완료

- `Placement()`: 배치를 완료하고 프로퍼티 정보를 저장합니다. 부모-자식 관계를 설정하고 편집 상태로 전환합니다.

의존성/상속 관계

- `MyRoomEditorState`를 상속받음.
- Zenject를 사용하여 `MyRoomEditorInputUtils`를 의존성 주입받음.
- `IMoveableProp`, `IPlaceableArea`, `SpawnablePropBase`, `MyRoomEditorInputUtils`에 의존.

사용 예시

오브젝트 생성 UI에서 오브젝트 선택시

```
private void OnSpawnProp(string downloadKey)
{
    _spawner.SpawnProp(downloadKey, obj =>
    {
        _myRoomEditorPlacementManager.SetPlacementTarget(obj);
        editorDefaultBar.SetActiveCategoryScrollView(false);
        myRoomEditorObjectEditUI.ShowSpawnPropNamePanel(scrollPanelPresenter.GetScrollPanel());
    });
}
```

관련 클래스

- MyRoomEditorState
- MyRoomEditorInputUtils
- IMoveableProp
- IPlaceableArea
- IRotatableProp

MyRoomEditorPropEditingManager

오브젝트 편집 상태를 관리하는 클래스

개요

`MyRoomEditorPropEditingManager`는 `MyRoomEditor`에서 `MyRoomEditorState`를 상속받아 오브젝트 편집 작업을 총괄하는 상태 기반 클래스입니다. 선택, 이동, 회전, 색상 변경, 삭제, 사진 업로드 등의 편집 기능을 통합 관리하며, UI 컴포넌트들과의 상호작용을 조율합니다. 이벤트 기반 아키텍처를 통해 다양한 편집 도구와의 협력을 지원합니다.

역할

- 오브젝트 선택 및 해제 상태 관리
- 상태 관리와 전략패턴을 결합하여 분리된 편집모드를 관리하고 상황에 따라 편집모드 전환.
- 전환된 편집모드에 override된 입력 동작을 전달.
- 색상 변경, 사진 업로드, 삭제 기능의 실행 및 결과 처리
- 편집 UI와의 통합 및 이벤트 중계
- 배치된 프로퍼티의 데이터 베이스 등록/해제 및 부모-자식 관계 관리
- 편집 작업의 취소 및 상태 정리
- 삭제 시 자식 프로퍼티의 연쇄 삭제 처리

선언

```
public class MyRoomEditorPropEditingManager : MyRoomEditorState
```

멤버

속성

```
/// <summary>  
/// 입력 처리 유틸리티 클래스.  
/// 마우스 입력, UI 포인터 감지 등을 담당하며, 편집 동작 중 입력 상태를 확인.  
/// </summary>  
[Inject]  
private MyRoomEditorInputUtils _inputUtils;
```



```

/// <summary>
/// 오브젝트 스폰닝 인터페이스. Zenject를 통해 주입.
/// Material 로드 및 오브젝트 생성을 담당(Addressable 관리).
/// </summary>
[Inject]
private IPropSpawner _propSpawner;

/// <summary>
/// 인터렉션 관리자.
/// 사진 업로드 등의 특수 인터렉션을 처리.
/// </summary>
[Inject]
private MyRoomEditorInteractionManager _interactionManager;

/// <summary>
/// 오브젝트 편집 UI 컴포넌트.
/// 편집 메뉴 표시, 색상 선택, 버튼 이벤트 처리.
/// </summary>
[SerializeField]
private MyRoomEditorObjectEditUI objectEditUI;

/// <summary>
/// 현재 활성화된 Prop 에디터.
/// 전략 패턴을 통해 선택, 이동, 회전 모드별로 교체됨.
/// </summary>
[SerializeField]
private MyRoomEditorPropEditor propEditor;

/// <summary>
/// 오브젝트 선택 에디터.
/// Select 모드에서 사용되며, 오브젝트 선택 및 해제를 담당.
/// </summary>
[SerializeField]
private MyRoomEditorPropSelector propSelector;

/// <summary>
/// 오브젝트 이동 에디터.
/// Move 모드에서 사용되며, 오브젝트 이동 처리.
/// </summary>
[SerializeField]
private MyRoomEditorPropMover propMover;

/// <summary>
/// 오브젝트 회전 에디터.
/// Rotate 모드에서 사용되며, 드래그를 통한 오브젝트 회전 처리.
/// </summary>
[SerializeField]
private MyRoomEditorPropRotator propRotator;

/// <summary>
/// 현재 편집 중인 오브젝트.
/// 선택된 오브젝트의 편집 작업을 수행.
/// </summary>
private IMyRoomEditorEditableObject _editingObject;

```

```

/// <summary>
/// 배치된 모든 오브젝트 리스트.
/// 편집 관리 대상 오브젝트들을 추적.
/// </summary>
private List<SpawnablePropBase> _spawnedPropBases = new();

/// <summary>
/// 삭제 스택.
/// 삭제 작업 시 부모-자식 관계를 고려한 순차적 삭제를 위해 사용.
/// </summary>
private Stack<IMyRoomEditorEditableObject> _deleteStack = new();

/// <summary>
/// 오브젝트 편집 모드 열거형.
/// 해당 열거형을 매개변수로 메서드를 호출해 편집 모드 전환.
/// </summary>
public enum PropEditMode { Disable, Select, Move, Rotate }

/// <summary>
/// 현재 편집 모드 상태.
/// Select, Move, Rotate, Disable 중 하나의 값.
/// </summary>
private PropEditMode editingMode;

```

메서드

```

/// <summary>
/// 배치된 오브젝트를 등록하여 편집 관리 대상으로 추가.
/// 새로 배치된 오브젝트가 편집 시스템에 등록되어 선택, 이동, 삭제 등의 작업이 가능해짐.
/// </summary>
/// <param name="prop">등록할 SpawnablePropBase 컴포넌트</param>
public void RegisterProp(SpawnablePropBase prop)

/// <summary>
/// 오브젝트 삭제 이벤트 처리.
/// 선택된 오브젝트와 그 자식 오브젝트들을 재귀적으로 삭제.
/// 데이터베이스에서 정보 제거 및 UI 업데이트.
/// </summary>
private void OnDeleteEvent()

/// <summary>
/// 자식 오브젝트를 재귀적으로 검색하여 삭제 스택에 추가.
/// 부모 ID를 기준으로 자식들을 찾아 순차 삭제 준비.
/// </summary>
/// <param name="parentId">부모 오브젝트의 ID</param>
private void SearchChildDeletedTarget(string parentId)

/// <summary>
/// 편집 모드를 변경하고 해당 모드의 에디터를 활성화.
/// 전략 패턴을 통해 선택, 이동, 회전 모드를 전환.
/// </summary>
/// <param name="mode">변경할 편집 모드</param>

```

```

/// <param name="mode">편집할 모드</param>
private void OnChangedEditMode(PropEditMode mode)

/// <summary>
/// Prop 에디터 정리.
/// 현재 에디터를 정리하고 비활성화.
/// </summary>
private void CleanUpPropEditor()

/// <summary>
/// 편집 모드에 따른 에디터 선택.
/// 스위치 표현식을 사용하여 적절한 에디터 반환.
/// </summary>
/// <returns>선택된 모드의 PropEditor 인스턴스</returns>
private MyRoomEditorPropEditor SwitchPropEditor()

/// <summary>
/// 배치 정보 업데이트를 처리.
/// 프리젠테를 통해 데이터베이스를 업데이트하고 변경 이벤트를 발생.
/// </summary>
/// <param name="placePropInfo">업데이트할 배치 정보</param>
private void UpdatePlacePropInfo(MyRoomPlaceProp placePropInfo)

/// <summary>
/// 색상 변경 도구 활성화.
/// 선택된 오브젝트의 색상 리스트를 UI에 표시.
/// </summary>
private void OnActiveChangeColorTool()

/// <summary>
/// 색상 선택 시 처리.
/// 선택된 색상을 로드하여 오브젝트에 적용.
/// </summary>
/// <param name="index">선택된 색상의 인덱스</param>
private void OnSelectedColorToChange(int index)

/// <summary>
/// 오브젝트 선택 이벤트 처리.
/// 선택된 오브젝트를 편집 대상으로 설정하고 편집 UI 표시.
/// </summary>
/// <param name="selectedObject">선택된 편집 가능 오브젝트</param>
private void OnObjectSelected(IMyRoomEditorEditableObject selectedObject)

/// <summary>
/// 이동 완료 처리.
/// 배치 정보 업데이트 후 선택 해제 및 모드 전환.
/// </summary>
private void OnFinishMove()

/// <summary>
/// 회전 완료 처리.
/// 배치 정보 업데이트 후 선택 해제 및 모드 전환.
/// </summary>
private void OnFinishRotate()

```

코드 스니펫

삭제 처리 프로세스

```
private void OnDeleteEvent()
{
    if (_editingObject == null) return; // 편집 중인 오브젝트 레퍼런스 체크
    var deleteCalledProp = _editingObject; // 삭제할 오브젝트 임시 캐싱
    OnReleaseSelected(); // 선택 해제 호출
    _deleteStack.Push(deleteCalledProp); // 삭제 스택에 푸시
    SearchChildDeletedTarget(deleteCalledProp.GetPlacePropId()); // 자식 검

    // 스택에 있는 모든 오브젝트 삭제
    while (_deleteStack.Count > 0)
    {
        var deleteTarget = _deleteStack.Pop(); // 스택에서 꺼냄
        RemovePlacePropInfo(deleteTarget.GetPlacePropId()); // DB 제거
        UnRegisterProp(deleteTarget.GetPlacePropId()); // 등록 해제
        deleteTarget.Delete(); // 실제 삭제
        OnUpdatedPlacementProp?.Invoke(); // 변경 이벤트
    }

    OnChangedEditMode(PropEditMode.Select); // 선택 모드 전환
    _deleteStack.Clear(); // 스택 초기화
}

private void SearchChildDeletedTarget(string parentId)
{
    // 부모를 가진 오브젝트 중 parentId와 일치하는 자식들 찾기
    var targets = _spawnedPropBases.Where
        (basePropComp => basePropComp.ParentProp != null &&
            basePropComp.ParentProp.GetPlacePropId() == parentId).ToList();
    if (targets.Count == 0) return; // 자식 없음
    foreach (var target in targets)
    {
        _deleteStack.Push(target.GetEditableObject()); // 자식 추가
        SearchChildDeletedTarget(target.GetPlacePropId()); // 재귀 호출
    }
}
```

오브젝트 색상 변경 프로세스(material변경)

```
private void OnActiveChangeColorTool()
{
    if (_editingObject == null) return; // 편집 오브젝트 확인
    if (!_editingObject.IsEditableColor(out var colorEditableProp)) return; //
    var colorList = colorEditableProp.GetColorList(); // 색상 리스트 획득
    objectEditUI.DrawColorList(colorList); // UI에 색상 리스트 표시
}
```

```

private void OnSelectedColorToChange(int index)
{
    if (_editingObject == null) return;
    if (!_editingObject.IsEditableColor(out var colorEditableProp)) return;
    var selectedColor = colorEditableProp.GetColorList()[index].materialKey;
    _propSpawner.SpawnMaterial(selectedColor, (loadedMat) => // Material 로드
    {
        colorEditableProp.SetColor(loadedMat, index); // 색상 적용
        UpdatePlacePropInfo(_editingObject.GetPlacePropInfo()); // 정보 업데이트
        OnUpdatedPlacementProp?.Invoke(); // 변경 이벤트
    });
}

```

편집 모드 전환

```

private void OnChangedEditMode(PropEditMode mode)
{
    editingMode = mode; // 모드 설정
    if (editingMode == PropEditMode.Disable)
    {
        CleanUpPropEditor(); // 에디터 정리
        return;
    }

    propEditor?.Disable(); // 기존 에디터 비활성화
    propEditor = SwitchPropEditor(); // 새 에디터 선택
    if (propEditor.Setup(_editingObject)) propEditor.Enable(); // 설정 및 활성화
}

private void CleanUpPropEditor()
{
    propEditor?.CleanUp(); // 정리
    propEditor?.Disable(); // 비활성화
    propEditor = null; // 초기화
}

private MyRoomEditorPropEditor SwitchPropEditor()
{
    return editingMode switch
    {
        PropEditMode.Select => propSelector, // 선택 에디터
        PropEditMode.Move => propMover, // 이동 에디터
        PropEditMode.Rotate => propRotator, // 회전 에디터
        _ => null // 기본값
    };
}

```

기능 설명

프로퍼티 선택 및 편집

- `OnObjectSelected()`: 오브젝트 선택 시 편집 UI를 표시하고 편집 상태를 설정.
- `OnReleaseSelected()`: 선택 해제 시 UI를 숨기고 편집 상태를 정리.

삭제 기능

- `OnDeleteEvent()`: 선택된 오브젝트와 그 자식들을 재귀적으로 삭제. 배치 정보를 제거하고 등록을 해제.

색상 변경

- `OnActiveChangeColorTool()`: 색상 변경 UI를 활성화하고 사용 가능한 색상 리스트를 표시.
- `OnSelectedColorToChange()`: 선택된 색상을 로드하여 적용하고 배치 정보를 업데이트.

이동 및 회전

- `OnActiveMoveTool()`: 이동 모드로 전환.
- `OnActiveRotateFreeTool()`: 자유 회전 모드로 전환.
- `OnRotateQuarter()`: 90도 스냅 회전을 적용.

사진 업로드

- `OnUploadPhoto()`: 사진 프레임 오브젝트에 대해 이미지 업로드를 시작.

모드 관리

- `OnChangedEditMode()`: 편집 모드를 변경하고 해당 에디터를 활성화.
- `SwitchPropEditor()`: 현재 모드에 따라 적절한 오브젝트 에디터를 반환.

오브젝트 데이터 관리

- `RegisterProp()`: 새로 배치된 오브젝트를 등록.
- `UnRegisterProp()`: 삭제된 오브젝트를 등록 해제.
- `SetPropBaseParentTransform()`: 로드된 오브젝트들의 부모-자식 관계를 설정.

의존성/상속 관계

- `MyRoomEditorState`를 상속받음.
- Zenject를 사용하여 `MyRoomEditorInputUtils`, `MyRoomEditorInteractionManager`, `IPropSpawner`를 의존성 주입받음.
- `MyRoomEditorObjectEditUI`, `MyRoomEditorPropEditor`, `MyRoomEditorPropSelector`, `MyRoomEditorPropMover`, `MyRoomEditorPropRotator`, `IMyRoomEditorEditableObject`, `SpawnablePropBase`에 의존.

관련 클래스

- `MyRoomEditorState`
- `MyRoomEditorInputUtils`

- MyRoomEditorInputUtils
- MyRoomEditorInteractionManager
- MyRoomEditorObjectEditUI
- MyRoomEditorPropSelector
- MyRoomEditorPropMover
- MyRoomEditorPropRotator

MyRoomEditorPropEditor

편집 기능에서 사용되는 공통 메서드와 멤버를 정의하는 abstract 클래스

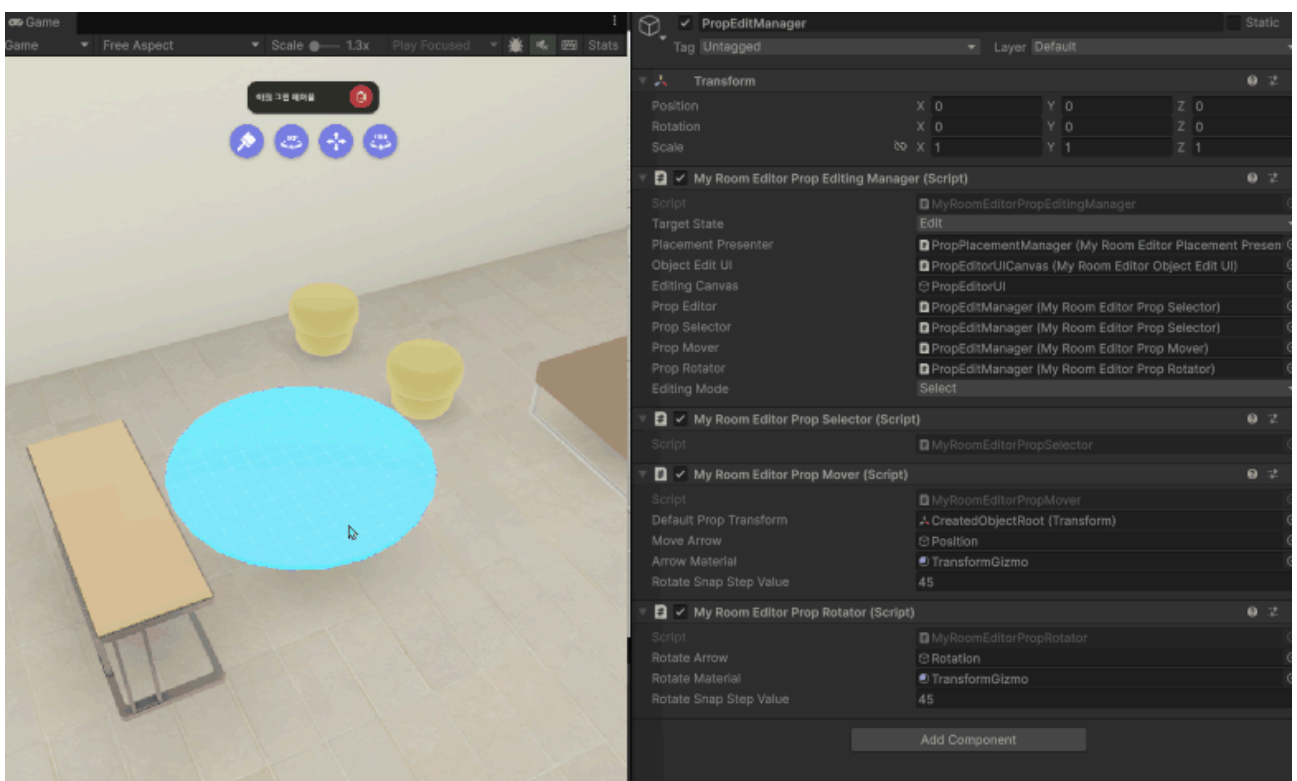
개요

MyRoomEditorPropEditor는 MyRoomEditor에서 오브젝트 편집을 위한 추상 베이스 클래스입니다. 이 클래스는 오브젝트 편집 기능의 기본 인터페이스를 제공하며, 선택된 오브젝트의 상태를 관리하고 **MyRoomEditorPropEditingManager**를 통해 받은 입력 및 UI 동작에 대한 이벤트를 서브 클래스를 통해 처리합니다.

역할

- 오브젝트 편집 기능의 기본 틀 제공
- 선택된 편집 오브젝트의 상태 캐싱 및 관리
- 입력 이벤트에 대한 추상 인터페이스 정의
- Zenject를 통한 의존성 주입 지원
- 편집도구의 라이프 사이클 관리

편집 도구 전환 및 관리





선언

```
public abstract class MyRoomEditorPropEditor : MonoBehaviour
```

멤버

속성

```
/// <summary>
/// 입력 처리 유틸리티 클래스.
/// 마우스 입력, UI 포인터 감지 등을 담당하며, 편집 동작 중 입력 상태를 확인.
/// Zenject를 통해 주입되며, protected로 서브클래스에서 접근 가능.
/// </summary>
[Inject]
protected MyRoomEditorInputUtils InputUtils;

/// <summary>
/// 현재 선택된 편집 가능한 오브젝트.
/// 편집 작업의 대상이 되는 오브젝트를 참조.
/// </summary>
protected IMyRoomEditorEditableObject SelectedObject;

/// <summary>
/// 편집 시작 시점의 색상 인덱스 캐시.
/// 실행 취소 시 원래 색상으로 복원하기 위해 저장.
/// </summary>
protected int CachedColorIndex;

/// <summary>
/// 편집 시작 시점의 위치 캐시.
/// 실행 취소 시 원래 위치로 복원하기 위해 저장.
/// </summary>
protected Vector3 CachedPosition;

/// <summary>
/// 편집 시작 시점의 회전 값 캐시.
/// 실행 취소 시 원래 회전으로 복원하기 위해 저장.
/// </summary>
protected Quaternion CachedRotation;

/// <summary>
/// 편집 시작 시점의 부모 오브젝트 캐시.
/// 실행 취소 시 원래 부모 관계로 복원하기 위해 저장.
/// </summary>
```

추상 메서드

```
/// <summary>
/// 에디터를 활성화하는 메서드.
/// 이벤트 등록, UI 표시, 편집 준비 작업을 수행.
/// 서브클래스에서 구체적인 활성화 프로세스를 구현.
/// </summary>
public abstract void Enable();

/// <summary>
/// 에디터를 비활성화하는 메서드.
/// 이벤트 해제, UI 숨김, 편집 종료 작업을 수행.
/// 서브클래스에서 구체적인 비활성화 프로세스 구현.
/// </summary>
public abstract void Disable();

/// <summary>
/// 편집할 오브젝트를 설정하고 초기화하는 메서드.
/// 선택된 오브젝트의 상태를 캐시하고 편집 준비.
/// </summary>
/// <param name="setUpObject">편집할 오브젝트</param>
/// <returns>설정이 성공했는지 여부 (true: 성공, false: 실패)</returns>
public abstract bool Setup(IMyRoomEditorEditableObject setUpObject);

/// <summary>
/// 에디터의 상태를 정리하고 리소스를 해제하는 메서드.
/// 캐시된 상태 초기화, 참조 제거 등의 정리 작업.
/// </summary>
public abstract void CleanUp();

/// <summary>
/// 마우스 왼쪽 버튼 눌림 이벤트를 처리하는 메서드.
/// 편집 시작, 드래그 모드 진입 등의 동작을 수행.
/// </summary>
public abstract void OnPointerDown();

/// <summary>
/// 마우스 왼쪽 버튼 떼어짐 이벤트를 처리하는 메서드.
/// 편집 완료, 드래그 종료 등의 동작을 수행.
/// </summary>
public abstract void OnPointerUp();

/// <summary>
/// 취소 입력 (ESC 키)을 처리하는 메서드.
/// 현재 편집 작업을 취소하고 이전 상태로 복원.
/// </summary>
public abstract void OnCancel();

/// <summary>
/// 마우스 오른쪽 버튼 클릭 이벤트를 처리하는 메서드.
/// 컨텍스트 메뉴 표시, 특수 동작 수행 등의 기능.
```

```
/// </summary>
public abstract void OnRightClick();
```

기능 설명

추상 인터페이스 제공

- 모든 오브젝트 편집기가 구현해야 하는 기본 메서드들을 정의
- Enable/Disable을 통한 편집기 라이프사이클 관리

상태 관리

- Setup 시 선택된 오브젝트의 현재 상태를 캐싱
- 편집 중 취소 시 캐시된 상태로 복원 가능
 - `CachedPosition`: 편집 전 위치 저장
 - `CachedRotation`: 편집 전 회전 값 저장
 - `CachedColorIndex`: 편집 전 색상 인덱스 저장
 - `CachedParent`: 편집 전 부모 오브젝트 저장

이벤트 처리

- 포인터 입력, 취소, 우클릭 등의 이벤트를 추상 메서드로 정의
- 서브클래스에서 구체적인 편집 로직 구현

공통 동작 순서

- 1 `Setup(IMyRoomEditorEditableObject setUpObject)`: 편집할 오브젝트 설정 및 초기화
- 2 `Enable()`: 에디터 활성화 및 이벤트 등록
- 3 편집 작업 수행: 서브클래스별 프로세스 실행
- 4 `Disable()`: 에디터 비활성화 및 이벤트 해제
- 5 `CleanUp()`: 상태 정리 및 리소스 해제

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `MyRoomEditorInputUtils`에 의존.
- Zenject 패키지를 사용하여 의존성 주입.
- `IMyRoomEditorEditableObject`, `SpawnablePropBase` 인터페이스 사용.
- `MyRoomEditorPropEditor`의 서브클래스.
 - `MyRoomPropSelector`: 오브젝트 선택 프로세스 구현.
 - `MyRoomPropMover`: 오브젝트 이동 프로세스 구현.
 - `MyRoomPropRotator`: 오브젝트 회전 프로세스 구현.

관련 클래스

- [MyRoomEditorInputUtils](#)
- [IMyRoomEditorEditableObject](#)
- [MyRoomEditorPropEditingManager](#)
- [SpawnablePropBase](#)
- [MyRoomPropSelector](#)
- [MyRoomPropMover](#)
- [MyRoomPropRotator](#)

MyRoomEditorPropSelector

오브젝트의 선택을 담당하는 클래스

개요

`MyRoomEditorPropSelector`는 MyRoomEditor에서 오브젝트 선택 기능을 제공하는 클래스입니다.

`MyRoomEditorPropEditor`를 상속받아 `MyRoomEditorInputUtils`의 레이캐스트를 통한 오브젝트 검출, 포커스 및 선택 상태 관리를 담당합니다.

역할

- `MyRoomEditorInputUtils`를 활용해 레이캐스트를 통한 편집 가능 오브젝트 검출
- 오브젝트 포커스 및 선택 상태 관리
- 우선순위 레이어를 통한 정확한 타겟팅
- 선택 해제 및 상태 정리 기능 제공

오브젝트 선택 기능 제공





선언

```
public class MyRoomEditorPropSelector : MyRoomEditorPropEditor
```

멤버

이벤트

```
/// <summary>
/// 오브젝트 선택시 호출되는 이벤트.
/// 선택된 오브젝트의 정보가 필요한 클래스에서 해당 이벤트 구독하여 사용.
/// </summary>
public event Action<IMyRoomEditorEditableObject> OnObjectSelect;

/// <summary>
/// 오브젝트 선택 해제시 호출되는 이벤트.
/// </summary>
public event Action OnReleaseSelect;
```

속성

```
/// <summary>
/// 오브젝트 감지 상태일때 실행되는 코루틴 참조.
/// </summary>
private Coroutine _detector;

/// <summary>
/// 현재 포커스된 오브젝트 참조.
/// </summary>
private IMyRoomEditorEditableObject _focusedObject;

/// 레이캐스트 타겟 레이어, 우선순위 레이어, 레이어 상수 이름
private int _targetLayer;
private int _firstPriorityLayer;
private const string MyRoomPropLayerNameKey = "MyRoomProp";
private const string MyRoomWallLayerNameKey = "Wall";
private const string MyRoomGroundLayerNameKey = "Ground";
```

메서드

```
/// <summary>
/// Raycast 오브젝트 감지 코루틴.
```

```

/// </summary>
private IEnumerator CoDetectObject()

/// <summary>
/// 오브젝트가 선택되었는지 확인하는 메서드.
/// </summary>
/// <param name="focusedObject">확인할 오브젝트</param>
/// <returns>선택 여부</returns>
private bool IsSelectedObject(IMyRoomEditorEditableObject focusedObject)

/// <summary>
/// 오브젝트 선택 해제 메서드.
/// </summary>
private void ReleaseSelect()

```

코드 스니펫

오브젝트 감지 프로세스

```

private IEnumerator CoDetectObject()
{
    while (true) // 무한 루프
    {
        yield return null; // 다음 프레임까지 대기

        if (InputUtils.RaycastHit(out RaycastHit hits, _targetLayer, _firstPersonCamera.transform.position, out focusedObject))
        {
            if (hits.transform.TryGetComponent(out IMyRoomEditorEditableObject editableObject) && InputUtils.IsPointerOnUI() == false) // UI 위가 아니면
            {
                if (focusedObject.Equals(_focusedObject)) continue; // 같은 오브젝트면
                if (IsSelectedObject(focusedObject)) // 선택된 오브젝트인지 확인
                {
                    _focusedObject?.Unfocused(); // 이전 포커스 해제
                    _focusedObject = focusedObject; // 포커스 설정
                }
                else // 선택되지 않은 오브젝트
                {
                    if (IsSelectedObject(_focusedObject) == false) // 이전 포커스 해제
                    {
                        _focusedObject?.Unfocused(); // 포커스 해제
                    }
                    _focusedObject = focusedObject; // 새로운 포커스 설정
                    _focusedObject.Focused(); // 포커스 상태로 설정
                }
            }
            else // 편집 가능 오브젝트가 아니거나 UI 위면
            {
                //Unfocus조건 // Unfocus 조건 설명
                //1. 선택된 오브젝트는 Unfocus되지 않는다. // 선택된 오브젝트는 포커스 유지
                //2. 선택된 오브젝트가 아니며 포커싱 중인 오브젝트가 있는 경우는 Unfocus 한
                if (IsSelectedObject(focusedObject)) // 포커스 오브젝트가 선택된 상태

```

```

        {
            _focusedObject = null; // 참조만 해제 (선택 상태 유지)
        }
        else // 포커스만 된 상태면
        {
            _focusedObject?.Unfocused(); // 포커스 해제
            _focusedObject = null; // 참조 해제
        }
    }
}

private bool IsSelectedObject([CanBeNull] IMyRoomEditorEditableObject focusedObject)
{
    return focusedObject != null && focusedObject.Equals(SelectedObject); // 널
}

```

선택 처리 프로세스

```

public override void OnPointerDown()
{
    if (InputUtils.IsPointerOnUI()) return; // UI 위에서 클릭했으면 무시
    //현재는 벽에대한 편집 기능이 없어 벽 클릭시 선택이 Release되게함 // 현재 벽 편집 기능이
    if (_focusedObject == null || _focusedObject.IsPlacementArea(out var area))
    {
        ReleaseSelect(); // 선택 해제
    }
    else // 편집 가능한 오브젝트가 포커스된 경우
    {
        if (_focusedObject.Equals(SelectedObject)) // 같은 오브젝트를 다시 클릭했으면
        {
            ReleaseSelect(); // 선택 해제
            return; // 메서드 종료
        }
        SelectedObject?.Deselected(); // 이전 선택 오브젝트 선택 해제
        SelectedObject = _focusedObject; // 새로운 오브젝트 선택
        SelectedObject.Selected(); // 선택 상태로 설정
        Debug.Log($"[MyRoomEditorPropSelector] Object Selected {SelectedObject}");
        OnObjectSelect?.Invoke(SelectedObject); // 선택 이벤트 호출
    }
}

private void ReleaseSelect()
{
    SelectedObject?.Deselected(); // 선택된 오브젝트 선택 해제
    SelectedObject = null; // 선택 오브젝트 참조 해제
    _focusedObject?.Deselected(); // 포커스 오브젝트 선택 해제 (안전하게)
    _focusedObject = null; // 포커스 오브젝트 참조 해제
    OnReleaseSelect?.Invoke(); // 선택 해제 이벤트 호출
}

```


기능 설명

오브젝트 검출

- 레이캐스트를 통한 타겟 레이어(MyRoomProp, Wall, Ground) 히트 검출
- 우선순위 레이어를 통한 정확한 오브젝트 타겟팅
- UI 위 포인터는 무시

포커스 관리

- 포커스된 오브젝트에 Focused() 호출
- 선택된 오브젝트는 Unfocus되지 않음
- 포커스 변경 시 이전 오브젝트 Unfocus

선택 로직

- 포인터 다운 시 포커스된 오브젝트 선택
- 같은 오브젝트 재선택 시 선택 해제
- 배치 영역(벽) 클릭 시 선택 해제

선택 해제

- 취소 키 입력 시 선택 해제
- 선택된 오브젝트 Deselected() 호출
- 포커스된 오브젝트 정리

실시간 감지

- 매 프레임마다 Raycast 수행
- 선택된 오브젝트는 계속 선택 상태 유지

의존성/상속 관계

- `MyRoomEditorPropEditor`를 상속받음.
- `IMyRoomEditorEditableObject`, `IPlaceableArea` 인터페이스 사용.
- `MyRoomEditorInputUtils`에 의존.

사용 예시

`MyRoomEditorPropEditingManager`에서 선택모드 전환

```
private void OnChangedEditMode(PropEditMode mode)
{
    editingMode = mode;
    if (editingMode == PropEditMode.Disable)
    {
        CleanUpPropEditor();
    }
}
```

```

        CleanupPropEditor(),
        return;
    }

    propEditor?.Disable();
    propEditor = SwitchPropEditor();
    if(propEditor.Setup(_editingObject)) propEditor.Enable();
}

private MyRoomEditorPropEditor SwitchPropEditor()
{
    return editingMode switch
    {
        PropEditMode.Select => propSelector,
        PropEditMode.Move => propMover,
        PropEditMode.Rotate => propRotator,
    };
}

```

관련 클래스

- [MyRoomEditorInputUtils](#)
- [IMyRoomEditorEditableObject](#)
- [MyRoomEditorPropEditingManager](#)
- [MyRoomEditorPropEditor](#)

MyRoomEditorPropMover

오브젝트의 이동을 담당하는 클래스

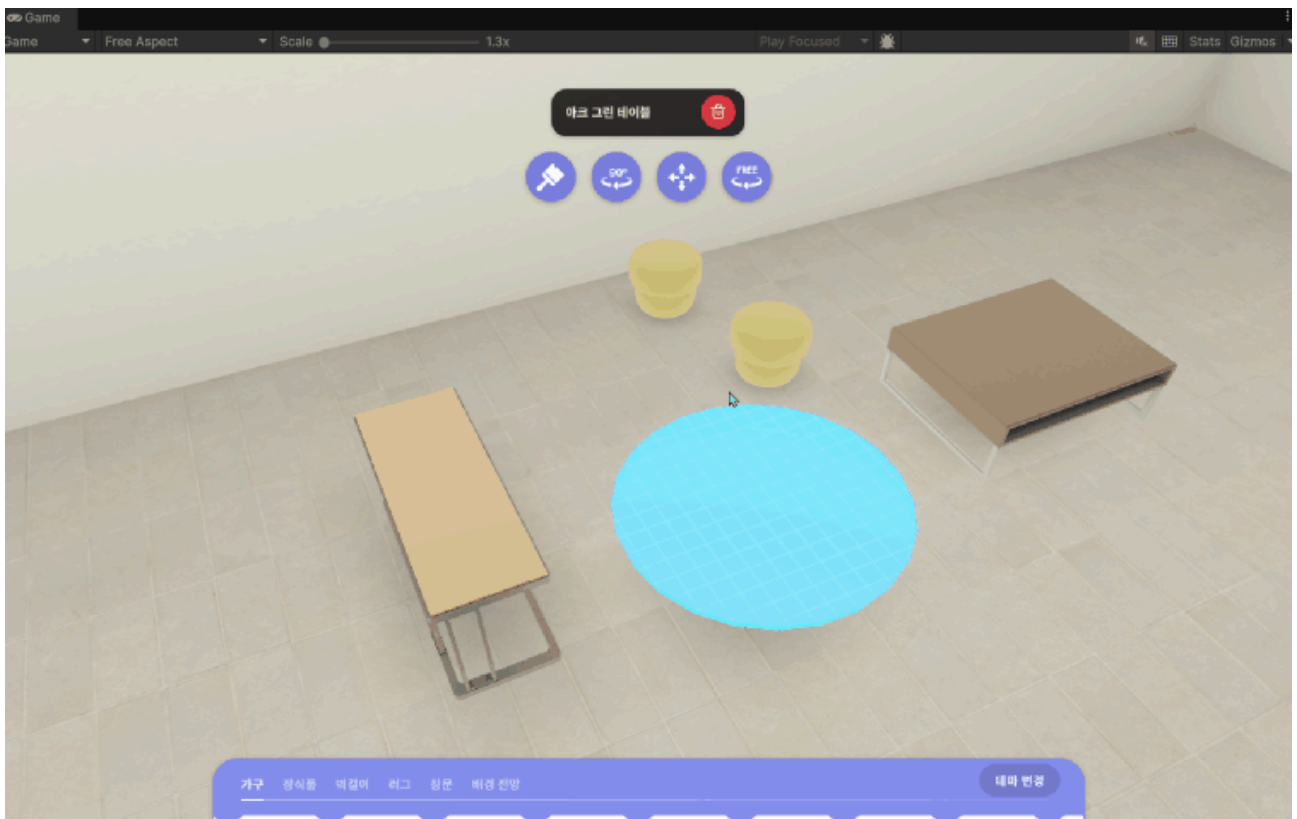
개요

`MyRoomEditorPropMover`는 `MyRoomEditor`에서 `MyRoomEditorPropEditor`를 상속 받아 오브젝트의 위치를 편집하는 클래스입니다. `MyRoomEditorInputUtils`를 활용해 `RaycastHit`을 검출하고 배치 영역을 검증하고 오브젝트의 위치를 변경합니다.

역할

- 오브젝트의 이동 및 회전(자유회전이 아닌 스냅회전만 지원) 편집 기능 제공
- `MyRoomEditorInputUtils`를 활용한 `RaycastHit`으로 배치 가능 위치 검출
- 이동 화살표 기즈모 표시 및 업데이트
- 부모 프로퍼티 관계 관리
- ESC 키로 이동 취소 및 원래 위치 복원

오브젝트 이동 기능 제공





선언

```
public class MyRoomEditorPropMover : MyRoomEditorPropEditor
```

멤버

이벤트

```
/// <summary>  
/// 오브젝트 이동 완료시 발생하는 이벤트  
/// </summary>  
public event Action OnFinishMove;
```

속성

```
/// <summary>  
/// 이동 시 부모가 없을 때 사용할 기본 트랜스폼.  
/// </summary>  
[SerializeField]  
private Transform defaultPropTransform;  
  
/// <summary>  
/// 이동 방향을 표시하는 화살표 기즈모 오브젝트.  
/// </summary>  
[SerializeField]  
private GameObject moveArrow;  
  
/// <summary>  
/// 우클릭 시 회전 스냅 값 (45도 단위).  
/// </summary>  
[SerializeField]  
private float rotateSnapStepValue = 45;  
  
/// <summary>  
/// 이동 가능한 프로퍼티 인터페이스 참조.  
/// </summary>  
private IMoveableProp _moveableProp;  
  
/// <summary>  
/// 회전 가능한 프로퍼티 인터페이스 참조 (optinal).  
/// </summary>  
private IRotatableProp _rotatableProp;
```

```

    /// <summary>
    /// 이동 감지 코루틴 참조.
    /// </summary>
    private Coroutine _detector;

    /// <summary>
    /// 레이캐스트 타겟 레이어 마스크.
    /// </summary>
    private int _targetLayer;

    /// <summary>
    /// 현재 위치가 배치 가능한지 여부.
    /// </summary>
    private bool _isPlaceable;

    /// <summary>
    /// 회전 불가능한 오브젝트의 초기 회전 값 캐시.
    /// </summary>
    private Quaternion _nonRotatablePropCachedRotation;

```

메서드

```

    /// <summary>
    /// 이동할 오브젝트 설정 메서드.
    /// </summary>
    /// <param name="setUpObject">설정할 편집 가능 오브젝트</param>
    /// <returns>설정 성공 여부</returns>
    public override bool Setup(IMyRoomEditorEditableObject setUpObject)

    /// <summary>
    /// 이동 모드 활성화 메서드.
    /// </summary>
    public override void Enable()

    /// <summary>
    /// 마우스 클릭 이벤트 핸들러.
    /// </summary>
    public override void OnPointerDown()

    /// <summary>
    /// 오브젝트 이동 상태일때 활성화 되는 이동감지 코루틴.
    /// </summary>
    private IEnumerator CoDetectMove()

    /// <summary>
    /// 배치 가능한 히트 포인트인지 확인하는 메서드.
    /// </summary>
    /// <returns>배치 가능 여부</returns>
    private bool PlaceableHitPoint()

    /// <summary>
    /// 이동 확정 메서드.

```

```

/// </summary>
private void MoveAccept()

/// <summary>
/// 오브젝트 부모 관계 설정 메서드.
/// </summary>
private void SetPropParent()

/// <summary>
/// 이동 동작 해제 메서드.
/// </summary>
private void ReleaseMovementBehaviour()

/// <summary>
/// 이동 취소 이벤트 핸들러 (ESC 키).
/// </summary>
public override void OnCancel()

```

코드 스니펫

이동 모드 시작 프로세스

```

public override bool Setup(IMyRoomEditorEditableObject setUpObject)
{
    if (!setUpObject.IsMovableProp(out _moveableProp)) return false; // 이동 가능
    SelectedObject = setUpObject; // 선택된 오브젝트 설정
    CachedPosition = _moveableProp.GetPosition(); // 초기 위치 캐시
    CachedParent = _moveableProp.PropBaseComponent.ParentProp; // 초기 부모 캐시
    if (_moveableProp.IsRotatableProp(out _rotatableProp)) // 회전 가능한 프로퍼티인
    {
        CachedRotation = _rotatableProp.GetRotation(); // 회전 값 캐시
    }
    else // 회전 불가능한 경우
    {
        _nonRotatablePropCachedRotation = _moveableProp.PropBaseComponent.trans
    }
    return true; // 설정 성공
}

public override void Enable()
{
    _detector = StartCoroutine(CoDetectMove()); // 이동 감지 코루틴 시작
    moveArrow.SetActive(true); // 이동 화살표 기즈모 활성화
    SetGizmo(); // 기즈모 위치 설정
}

```

이동 감지 및 배치검증 프로세스

```

private IEnumerator CoDetectMove()
{

```

```

while (true)
{
    _isPlaceable = PlaceableHitPoint();
    yield return null;
}

private bool PlaceableHitPoint()
{
    SetGizmo(); // 기즈모 설정
    if (!InputUtils.RaycastHit(out var raycastHit, _targetLayer, _moveableProp.transform.position, out var placeableArea)) return false;
    if (!raycastHit.transform.TryGetComponent(out IPlaceableArea placeableArea)) return false;
    if (!_moveableProp.IsPlaceableArea(raycastHit.point, placeableArea)) return false;
    return true; // 배치 가능
}

```

이동 적용 프로세스

```

public override void OnPointerDown()
{
    if (InputUtils.IsPointerOnUI()) return; // UI 위에서 클릭했으면 무시
    if (!_isPlaceable) return; // 배치 불가능한 위치면 무시
    MoveAccept(); // 이동 확정
}

private void MoveAccept()
{
    SetPropParent(); // 부모 관계 설정
    ReleaseMovementBehaviour(); // 이동 동작 해제
}

private void SetPropParent()
{
    if (CachedParent == null) // 캐시된 부모가 없는 경우
    {
        if (_moveableProp.PropBaseComponent.ParentProp == null) return; // 현재 부모가 없으면
        _moveableProp.PropBaseComponent.ParentProp.AddChildProp(_moveableProp.PropBaseComponent);
        Debug.Log($"[MyRoomEditorPropMover] {_moveableProp.PropBaseComponent.Id}가 {_moveableProp.PropBaseComponent.ParentProp.Id}에 추가되었습니다.");
    }
    else // 캐시된 부모가 있는 경우
    {
        if (_moveableProp.PropBaseComponent.ParentProp == null) // 현재 부모가 없으면
        {
            CachedParent.RemoveChildProp(_moveableProp.PropBaseComponent); // 캐시된 부모에서 제거
            _moveableProp.PropBaseComponent.transform.parent = defaultPropTransform;
            _moveableProp.PropBaseComponent.ClearPropParent(); // 부모 관계 클리어
            Debug.Log($"[MyRoomEditorPropMover] {_moveableProp.PropBaseComponent.Id}가 {_moveableProp.PropBaseComponent.ParentProp.Id}에서 제거되었습니다.");
        }
        else // 현재 부모가 있는 경우
        {
            if (_moveableProp.PropBaseComponent.ParentProp.GetPlacePropId() == CachedParent.GetPlacePropId()) return; // 같으면 리턴

```

```

        _moveableProp.PropBaseComponent.ParentProp.AddChildProp(_moveableProp);
        CachedParent.RemoveChildProp(_moveableProp.PropBaseComponent); // 캐시된 부모에서 제거
        Debug.Log($"[MyRoomEditorPropMover] {_moveableProp.PropBaseComponent}가 {_moveableProp}에 배치됨");
        Debug.Log($"[MyRoomEditorPropMover] {CachedParent.GetPlacePropId()}에 배치됨");
    }
}

private void ReleaseMovementBehaviour()
{
    StopCoroutine(_detector); // 감지 코루틴 중지
    SelectedObject.Deselected(); // 선택 해제
    CleanUp(); // 리소스 정리
    moveArrow.SetActive(false); // 화살표 기즈모 비활성화
    OnFinishMove?.Invoke(); // 이동 완료 이벤트 호출
}

```

기능 설명

이동 편집 활성화

- 이동 검출 코루틴 시작
- 이동 화살표 기즈모 활성화
- 기즈모 위치 및 회전 설정

이동 프로세스

- 1 `Setup(IMyRoomEditorEditableObject setUpObject)`: 이동 가능한 오브젝트 확인 및 초기 상태 캐시
- 2 `Enable()`: 이동 감지 코루틴 시작 및 기즈모 활성화
- 3 `CoDetectMove()`, `PlaceableHitPoint()`: 마우스 위치에 따른 오브젝트 이동 및 배치 가능성 확인
- 4 `MoveAccept()`: 마우스 클릭 시 이동 완료
- 5 `SetPropParent()`: 오브젝트 계층 구조 업데이트

배치 위치 검출

- `MyRoomEditorInputUtils`를 통한 타겟 레이어(Wall, Ground) 히트 검출
- `IPlaceableArea` 인터페이스 확인
- 배치 가능 영역 검증

기즈모 표시

- 이동 화살표를 오브젝트의 기즈모 위치에 표시
- 회전 값에 따라 화살표 방향 조정

부모 관계 관리

- 오브젝트가 다른 오브젝트 위에 배치될 때 부모-자식 관계 설정

- 부모 변경 시 기존 부모에서 제거하고 새 부모에 추가
- 독립 배치 시 부모 관계 해제

스냅 회전

- 우클릭 시 지정된 스냅 값만큼 회전
- 회전 가능 오브젝트에 한함

실행 취소

- ESC 키로 이동 취소
- 캐시된 위치, 회전, 부모 관계로 복원

의존성/상속 관계

- `MyRoomEditorPropEditor`를 상속받음.
- `IMoveableProp`, `IRotatableProp`, `IPlaceableArea` 인터페이스 사용.
- `MyRoomEditorInputUtils`에 의존.

사용 예시

`MyRoomEditorPropEditingManager`에서 이동 편집 모드 전환

```
private void OnChangedEditMode(PropEditMode mode)
{
    editingMode = mode;
    if (editingMode == PropEditMode.Disable)
    {
        CleanUpPropEditor();
        return;
    }

    propEditor?.Disable();
    propEditor = SwitchPropEditor();
    if(propEditor.Setup(_editingObject)) propEditor.Enable();
}

private MyRoomEditorPropEditor SwitchPropEditor()
{
    return editingMode switch
    {
        PropEditMode.Select => propSelector,
        PropEditMode.Move => propMover,
        PropEditMode.Rotate => propRotator,
    };
}
```

관련 클래스

- MyRoomEditorInputUtils
- MyRoomEditorPropEditor
- MyRoomEditorPropEditingManager
- IMoveableProp
- IRotatableProp
- IPlaceableArea

MyRoomEditorPropRotator

오브젝트의 회전을 담당하는 클래스

개요

`MyRoomEditorPropRotator`는 MyRoomEditor에서 오브젝트의 회전 편집을 담당하는 클래스입니다.

`MyRoomEditorPropEditor`를 상속받아 회전 편집 기능을 구현하며, PointerDelta 통한 실시간 회전과 기즈모 표시를 제공합니다.

역할

- 오브젝트의 회전 편집 기능 제공
- PointerDelta를 추적하여 실시간 회전 조작
- 회전 화살표 기즈모 표시 및 업데이트
- 스냅 회전 기능 지원
- 회전 각도에 따른 머티리얼 파라미터 업데이트(커스텀 셰이더 효과)
- ESC 키로 회전 취소 및 원래 각도 복원

오브젝트 회전 기능 제공





선언

```
public class MyRoomEditorPropRotator : MyRoomEditorPropEditor
```

멤버

이벤트

```
/// <summary>
/// 오브젝트 회전 완료시 발생하는 이벤트
/// </summary>
public event Action OnFinishRotate;
```

속성

```
/// <summary>
/// 회전 방향을 표시하는 화살표 기즈모 오브젝트.
/// </summary>
[SerializeField]
private GameObject rotateArrow;

/// <summary>
/// 회전 기즈모의 재질.
/// </summary>
[SerializeField]
private Material rotateMaterial;

/// <summary>
/// 우클릭 시 회전 스냅 값 (45도 단위).
/// </summary>
[SerializeField]
private float rotateSnapStepValue = 45;

/// <summary>
/// 회전 가능한 오브젝트 인터페이스 참조.
/// </summary>
private IRotatableProp _rotatableProp;

/// <summary>
/// 회전 감지 코루틴 참조.
/// </summary>
private Coroutine _rotator;
```

```
private Coroutine _detector;

/// <summary>
/// 셰이더 속성 ID 캐싱 (_Angle).
/// </summary>
private readonly int _rotateAnglePropertyId = Shader.PropertyToID("_Angle");
```

메서드

```
/// <summary>
/// 회전할 오브젝트 설정 메서드.
/// </summary>
/// <param name="setUpObject">설정할 편집 가능 오브젝트</param>
/// <returns>설정 성공 여부</returns>
public override bool Setup(IMyRoomEditorEditableObject setUpObject)

/// <summary>
/// 회전 모드 활성화 메서드.
/// </summary>
public override void Enable()

/// <summary>
/// 회전을 감지하는 코루틴 메서드.
/// </summary>
private IEnumerator CoDetectRotate()

/// <summary>
/// 마우스 클릭 이벤트 핸들러.
/// </summary>
public override void OnPointerDown()

/// <summary>
/// 회전 확정 메서드.
/// </summary>
private void RotateAccept()

/// <summary>
/// 회전 동작 해제 메서드.
/// </summary>
private void ReleaseRotateBehaviour()

/// <summary>
/// 회전 취소 이벤트 핸들러 (ESC 키).
/// </summary>
public override void OnCancel()
```

코드 스니펫

회전 모드 시작 프로세스

```

public override bool Setup(IMyRoomEditorEditableObject setUpObject)
{
    if (!setUpObject.IsRotatableProp(out _rotatableProp)) return false; //
    SelectedObject = setUpObject; // 선택된 오브젝트 설정
    CachedRotation = _rotatableProp.GetRotation(); // 초기 회전 값 캐시
    return true; // 설정 성공
}

public override void Enable()
{
    _detector = StartCoroutine(CoDetectRotate()); // 회전 감지 코루틴 시작
    rotateArrow.SetActive(true); // 회전 화살표 기즈모 활성화
    SetGizmo(); // 기즈모 위치 설정
}

```

회전 감지 및 적용 프로세스

```

private IEnumerator CoDetectRotate()
{
    while (true)
    {
        var delta = InputUtils.PointerDelta().x; // 마우스 포인터의 X축 델타 값 가져오기
        Quaternion deltaRotation = Quaternion.Euler(0, -delta, 0); // X축 델타를
        Quaternion rotation = _rotatableProp.GetRotation() * deltaRotation; //
        _rotatableProp.SetRotation(rotation); // 회전 설정
        rotateMaterial.SetFloat(_rotateAnglePropertyId, -rotation.eulerAngles.y);
        yield return null; // 다음 프레임까지 대기
    }
}

public override void OnPointerDown()
{
    if (InputUtils.IsPointerOnUI()) return; // UI 위에서 클릭했으면 무시
    RotateAccept(); // 회전 확정
}

private void RotateAccept()
{
    ReleaseRotateBehaviour(); // 회전 동작 해제
}

private void ReleaseRotateBehaviour()
{
    StopCoroutine(_detector); // 감지 코루틴 중지
    SelectedObject.Deselected(); // 선택 해제
    CleanUp(); // 리소스 정리
    rotateArrow.SetActive(false); // 화살표 기즈모 비활성화
    OnFinishRotate?.Invoke(); // 회전 완료 이벤트 호출
}

```

기능 설명

회전 편집 활성화

- 회전 검출 코루틴 시작
- 회전 화살표 기즈모 활성화
- 기즈모 위치 및 회전 설정

회전 프로세스

- 1 `Setup(IMyRoomEditorEditableObject setUpObject)`: 회전 가능한 오브젝트 확인 및 초기 상태 캐시
- 2 `Enable()`: 회전 감지 코루틴 시작 및 기즈모 활성화
- 3 `CoDetectRotate()`: `PointerDelta`를 통해 실시간 회전 적용
- 4 `RotateAccept()()`: 마우스 클릭 시 회전 적용

실시간 회전 조작

- 포인터 델타 값을 기반으로 실시간 회전 적용
- Y축 회전을 중심으로 한 쿼터니언 계산
- 회전 값에 따라 커스텀 셰이더 머티리얼 파라미터 업데이트

기즈모 표시

- 회전 화살표를 오브젝트의 기즈모 위치에 표시
- 기즈모 회전 방향 설정

스냅 회전

- 우클릭 시 지정된 스냅 값만큼 회전
- 정밀한 각도 조정 지원

취소 및 복원

- 취소 시 캐시된 회전 값으로 복원
- 편집 상태 해제 및 이벤트 발생

Material 효과

- Shader의 `_Angle` 속성을 사용하여 회전 각도를 시각적으로 표현
- 실시간으로 회전 값에 따라 Material 업데이트
-

의존성/상속 관계

- `MyRoomEditorPropEditor`를 상속받음.
- `IRotatableProp` 인터페이스 사용.
- `MyRoomEditorInputUtils`에 의존.

사용 예시

`MyRoomEditorPropEditingManager`에서 회전 편집 모드 전환

```
private void OnChangedEditMode(PropEditMode mode)
{
    editingMode = mode;
    if (editingMode == PropEditMode.Disable)
    {
        CleanUpPropEditor();
        return;
    }

    propEditor?.Disable();
    propEditor = SwitchPropEditor();
    if(propEditor.Setup(_editingObject)) propEditor.Enable();
}

private MyRoomEditorPropEditor SwitchPropEditor()
{
    return editingMode switch
    {
        PropEditMode.Select => propSelector,
        PropEditMode.Move => propMover,
        PropEditMode.Rotate => propRotator,
    };
}
```

관련 클래스

- `MyRoomEditorPropEditor`
- `MyRoomEditorPropEditingManager`
- `MyRoomEditorInputUtils`
- `IRotatableProp`

MyRoomEditorInteractionManager

오브젝트의 특수 인터랙션을 담당하는 클래스

개요

`MyRoomEditorInteractionManager`는 `MyRoomEditor` 안에서 오브젝트의 특수 인터랙션을 담당하는 클래스입니다. `MyRoomEditorPropEditingManager`를 통해 특정 인터랙션 호출을 처리하며, 외부 API 및 네트워크 연동을 담당합니다. 콜백 기반 비동기 처리를 지원합니다.

역할

- 이미지 파일 업로드 요청 및 응답 처리
- 플랫폼별 상호작용 로직 분기(API호출 관련)
- 업로드 완료 콜백 관리
- `MyRoomEditor`와 도메인 계층 간 상호작용 중재

선언

```
public class MyRoomEditorInteractionManager : MonoBehaviour
```

멤버

속성

```
/// <summary>  
/// 파일 업로드와 같은 인터랙션을 처리하는 도메인 레이어 인터페이스  
/// 인터랙터 패턴을 사용하여 비즈니스 로직과 프레젠테이션을 분리  
/// </summary>  
[Inject]  
private IMyRoomInteractionInteractor _interactor;  
  
/// <summary>  
/// 이미지 업로드 완료 콜백을 저장하는 필드  
/// Action<string> 델리게이트 타입으로, 업로드된 파일 경로를 매개변수로 받음  
/// 콜백을 저장하여 비동기 작업 완료 시 호출하기 위해 사용  
/// </summary>  
private Action<string> _onCompleteUploadImageCallback;
```

메서드

```
/// <summary>
/// 이미지 업로드 요청을 처리하는 public 메서드.
/// </summary>
/// <param name="imageUploadedCallback">업로드 완료 시 호출될 콜백 함수, null 가능</param>
public void UploadImage(Action<string> imageUploadedCallback)

/// <summary>
/// 파일 업로드 완료 시 호출되는 private 콜백 메서드.
/// 인터랙터의 RequestFileUpload에서 호출되며, 콜백을 실행하고 참조를 해제.
/// </summary>
/// <param name="path">업로드된 파일의 서버 경로 또는 URL</param>
private void OnCompleteUploadImage(string path)
```

코드 스니펫

이미지 업로드 프로세스

```
public void UploadImage(Action<string> imageUploadedCallback)
{
    #if !UNITY_EDITOR
        _onCompleteUploadImageCallback = imageUploadedCallback; // 비동기 작업에서 콜백
        _interactor.RequestFileUpload(OnCompleteUploadImage); // 인터랙터를 통해 파일 업로드
    #else
        ...
    #endif
}

private void OnCompleteUploadImage(string path)
{
    _onCompleteUploadImageCallback?.Invoke(path);
    _onCompleteUploadImageCallback = null;
}
```

기능 설명

이미지 업로드 요청

- `UploadImage()`: 플랫폼에 따라 다른 업로드 로직을 수행. 런타임에서는 인터랙터를 통해 실제 파일 업로드를 요청하고, 에디터에서는 테스트용 더미 URL을 반환.

콜백 관리

- 업로드 완료 시 등록된 콜백을 호출하고, 메모리 누수를 방지하기 위해 콜백을 null로 초기화.

활गत습 순기

- 컴파일 시점 조건부 컴파일을 사용하여 에디터 환경과 런타임 환경에서 다른 동작을 수행.

인터랙터 패턴

- 도메인 레이어의 인터랙터를 통해 파일 업로드 요청
- 클린 아키텍처 원칙 준수: 프레젠테이션과 도메인 분리

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- Zenject를 사용하여 `IMyRoomInteractionInteractor`를 의존성 주입받음.
- `IMyRoomInteractionInteractor` 인터페이스에 의존.

사용 예시

`MyRoomEditorPropEditingManager`에서 사진 업로드 호출시

```
// MyRoomEditorPropEditingManager에서 호출
private void OnUploadPhoto()
{
    if (_editingObject.IsPhotoFrameProp(out var photoFrameProp))
    {
        _interactionManager.UploadImage(photoFrameProp.OnCompleteUpload);
        photoFrameProp.ReserveCompletedApplyImageCallback(UpdatePlacePropInfo);
    }
    else
    {
        Debug.Log($"{_editingObject.PropNameKey} is not interactable");
    }
}
```

관련 클래스

- `MyRoomEditorPropEditingManager`
- `IPhotoFrameProp`
- `SpawnablePhotoFrameProp`

MyRoomEditorInputController

InputAction를 통해 발생하는 입력 이벤트를 각각의 Dispatcher로 전달하는 클래스

개요

`MyRoomEditorInputController`는 `InputAction`을 통해 발생하는 입력 이벤트를 각각의 `Dispatcher`로 전달하는 클래스입니다. Rfice 게임 입력과 MyRoomEditor 전용 입력을 토글하며, 카메라 이동/회전/줌과 편집 관련 입력 이벤트를 적절한 디스패처로 전달합니다. `InputSystem` [🔗](#)을 사용하여 입력을 처리하고, 안전 영역 내 포인터 입력을 보장합니다.

역할

- Rfice 입력과 MyRoom 에디터 입력의 활성화/비활성화 토글
- 카메라 제어 입력 이벤트(이동, 회전, 줌)의 구독 및 디스패처 전달
- 편집 입력 이벤트(포인터 클릭, 취소, 삭제, 우클릭)의 구독 및 디스패처 전달
- 화면 안전 영역 내 포인터 입력 검증 및 필터링

선언

```
public class MyRoomEditorInputController : MonoBehaviour
```

멤버

속성

```
/// <summary>  
/// 기본 게임 입력 액션 (Rfice 입력)  
/// </summary>  
[Inject]  
private RficeAction _rficeInput;  
  
/// <summary>  
/// MyRoom 에디터 전용 입력 액션  
/// </summary>  
[Inject]  
private MyRoomEditorInput _myRoomInput;
```

```

    /// <summary>
    /// 카메라 입력 이벤트 처리 디스패처
    /// </summary>
    [Inject]
    private MyRoomEditorCameraInputDispatcher _cameraInputDispatcher;

    /// <summary>
    /// 편집 입력 이벤트 처리 디스패처
    /// </summary>
    [Inject]
    private MyRoomEditorEditingInputDispatcher _editingDispatcher;

```

메서드

```

    /// <summary>
    /// 모든 MyRoomEditor 입력을 활성화하는 메서드.
    /// 기본 게임 입력을 비활성화하고 에디터 입력을 활성화.
    /// </summary>
    public void EnableAll()

    /// <summary>
    /// 모든 MyRoomEditor 입력을 비활성화하는 메서드.
    /// 기본 게임 입력을 활성화하고 에디터 입력을 비활성화.
    /// </summary>
    public void DisableAll()

    /// <summary>
    /// 입력 이벤트를 구독하는 메서드.
    /// 각 InputAction의 이벤트에 핸들러를 등록하여 입력 처리 시작.
    /// </summary>
    private void InputEventSubscribe()

    /// <summary>
    /// 입력 이벤트를 구독 해제하는 메서드.
    /// 각 InputAction의 이벤트에서 핸들러를 제거하여 입력 처리 중지.
    /// </summary>
    private void InputEventUnsubscribe()

    /// <summary>
    /// 포인터가 화면 안전 영역 내에 있는지 검증하는 메서드.
    /// 노치나 시스템 UI 영역을 제외한 안전 영역 내 입력만 처리.
    /// </summary>
    /// <returns>포인터가 안전 영역 내에 있으면 true, 아니면 false</returns>
    private bool IsPointerInsideScreen()

```

코드 스니펫

안전 영역 검증

```

    private bool IsPointerInsideScreen()

```

```
private bool IsPointerInsideScreen()
{
    var pointer = InputSystem.GetDevice<Pointer>();
    return pointer == null || Screen.safeArea.Contains(pointer.position.ReadValue());
}
```

이벤트 구독

```
private void InputEventSubscribe()
{
    // 카메라 이동 입력 이벤트 구독
    _myRoomInput.MyRoomEditorCamera.CameraMove.started += OnCameraMoveStart;
    _myRoomInput.MyRoomEditorCamera.CameraMove.performed += OnCameraMovePerformed;
    _myRoomInput.MyRoomEditorCamera.CameraMove.canceled += OnCameraMoveStop;

    // 카메라 회전 입력 이벤트 구독
    _myRoomInput.MyRoomEditorCamera.CameraEngage.started += OnCameraControlStart;
    _myRoomInput.MyRoomEditorCamera.CameraDisengage.performed += OnCameraControlStop;

    // 카메라 줌 입력 이벤트 구독
    _myRoomInput.MyRoomEditorCamera.CameraZoom.performed += OnCameraZoomWheel;

    // 포인터 입력 이벤트 구독
    _myRoomInput.MyRoomEditorEditing.PointerDown.started += OnPointerDown;
    _myRoomInput.MyRoomEditorEditing.PointerUp.performed += OnPointerUp;

    // 단축키 입력 이벤트 구독
    _myRoomInput.MyRoomEditorEditing.Cancel.started += OnCancel;
    _myRoomInput.MyRoomEditorEditing.RightClick.started += OnRightClick;
    _myRoomInput.MyRoomEditorEditing.Delete.started += OnDeletePress;
}
```

기능 설명

입력 시스템 토글

- **EnableAll():** Rfice 입력을 비활성화하고 MyRoomEditor 입력을 활성화. 카메라 이동, 회전, 줌, 포인터 입력을 모두 활성화.
- **DisableAll():** Rfice 입력을 활성화하고 MyRoomEditor 입력을 비활성화. 모든 에디터 입력을 비활성화.

이벤트 처리

- 카메라 입력 이벤트(이동, 회전, 줌)를 MyRoomEditorCameraInputDispatcher로 전달.
- 편집 입력 이벤트(포인터 클릭, 취소, 삭제, 우클릭)를 MyRoomEditorEditingInputDispatcher로 전달.
- Dispatcher가 실제 비즈니스 로직 처리

안전 영역 검증

- 포인터 입력 시 Screen.safeArea를 사용하여 노치나 시스템 UI 영역을 제외한 안전 영역 내 입력만 처리.

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- Zenject를 사용하여 의존성 주입: `RficeAction`, `MyRoomEditorInput`,
`MyRoomEditorCameraInputDispatcher`, `MyRoomEditorEditingInputDispatcher`.
- `InputSystem` [↗](#) 패키지를 사용.

관련 클래스

- `MyRoomEditorCameraInputDispatcher`
- `MyRoomEditorEditingInputDispatcher`
- `InputSystem` [↗](#)

MyRoomEditorCameraInputDispatcher

카메라 조작 이벤트를 처리하는 클래스

개요

`MyRoomEditorCameraInputDispatcher` 클래스는 MyRoom 에디터에서 카메라 관련 입력 이벤트를 중앙에서 관리하고 배포하는 디스패처입니다. 입력 시스템으로부터 받은 카메라 조작 이벤트를 적절한 리스너들에게 전달합니다.

역할

- 카메라 이동, 회전, 줌 입력 이벤트 처리
- 이벤트 구독 패턴을 통한 느슨한 결합 구현
- 입력 상태 관리 및 이벤트 라우팅

선언

```
public class MyRoomEditorCameraInputDispatcher
```

멤버

이벤트

```
/// <summary>  
/// 카메라 이동 방향 이벤트, 이동 방향 벡터(Vector2)를 매개변수로 전달  
/// </summary>  
public event Action<Vector2> OnCameraMoveDirection;  
  
/// <summary>  
/// 카메라 이동 상태 이벤트. 이동 시작/종료를 bool 값으로 전달.  
/// </summary>  
public event Action<bool> OnCameraMoved;  
  
/// <summary>  
/// 카메라 회전 모드 시작 이벤트.  
/// </summary>  
public event Action OnCameraEngage;
```

```

/// <summary>
/// 카메라 회전 모드 종료 이벤트.
/// </summary>
public event Action OnCameraDisengage;

/// <summary>
/// 카메라 줌 값 변경 이벤트.
/// </summary>
public event Action<float> OnCameraZoom;

```

속성

```

/// <summary>
/// 이동 시작 시 true, 이동 종료 시 false로 설정.
/// </summary>
private bool _isActiveCameraMove;

```

메서드

```

/// <summary>
/// 카메라 이동이 시작되었을 때 호출되는 메서드.
/// 이동 상태를 활성화하고 이동 시작 이벤트를 발생시킴.
/// </summary>
public void OnCameraMoveStarted()

/// <summary>
/// 카메라 이동 중 방향 값이 변경되었을 때 호출되는 메서드.
/// 이동이 활성화된 경우에만 이동 방향 이벤트를 발생시킴.
/// </summary>
/// <param name="moveVec2">카메라 이동 방향 벡터</param>
public void OnCameraMovePerformed(Vector2 moveVec2)

/// <summary>
/// 카메라 이동이 종료되었을 때 호출되는 메서드.
/// 이동 상태를 비활성화하고 이동 종료 이벤트를 발생시킴.
/// </summary>
public void OnCameraMoveStopped()

/// <summary>
/// 카메라 회전 모드가 시작되었을 때 호출되는 메서드.
/// 회전 시작 이벤트를 발생시킴.
/// </summary>
public void OnCameraEngageStarted()

/// <summary>
/// 카메라 회전 모드가 종료되었을 때 호출되는 메서드.
/// 회전 종료 이벤트를 발생시킴.
/// </summary>
public void OnCameraDisengaged()

```

```
/// <summary>
/// 카메라 줌 값이 변경되었을 때 호출되는 메서드.
/// 줌 값 변경 이벤트를 발생시킴.
/// </summary>
/// <param name="value">줌 델타 값 (양수: 줌인, 음수: 줌아웃)</param>
public void OnCameraZoomed(float value)
```

코드 스니펫

카메라 이동 처리

```
public void OnCameraMoveStarted()
{
    _isActiveCameraMove = true;
    OnCameraMoved?.Invoke(_isActiveCameraMove);
}

public void OnCameraMovePerformed(Vector2 moveVec2)
{
    if (!_isActiveCameraMove) return;
    OnCameraMoveDirection?.Invoke(moveVec2);
}

public void OnCameraMoveStopped()
{
    _isActiveCameraMove = false;
    OnCameraMoved?.Invoke(_isActiveCameraMove);
}
```

카메라 회전 처리

```
public void OnCameraEngageStarted()
{
    OnCameraEngage?.Invoke();
}

public void OnCameraDisengaged()
{
    OnCameraDisengage?.Invoke();
}
```

카메라 줌 처리

```
public void OnCameraZoomed(float value)
{
    OnCameraZoom?.Invoke(value);
}
```

기능 설명

카메라 이동 이벤트

- `OnCameraMoveStarted()`: 이동 모드 시작, 이동 활성화 이벤트 발생
- `OnCameraMovePerformed()`: 이동 방향 전달, 활성화 상태일 때만 동작
- `OnCameraMoveStopped()`: 이동 모드 종료, 이동 비활성화 이벤트 발생

카메라 회전 및 줌 이벤트

- `OnCameraEngageStarted()`: 회전 모드 시작 이벤트 발생
- `OnCameraDisengaged()`: 회전 모드 종료 이벤트 발생
- `OnCameraZoomed()`: 줌 값 변경 이벤트 발생

이벤트 구독 패턴

- Observer 패턴 구현으로 이벤트 기반 통신
- 입력 처리 로직과 비즈니스 로직 분리

상태 추적

- `_isActiveCameraMove`: 현재 카메라 이동이 활성화되어 있는지 추적

의존성/상속 관계

- `InputSystem`  사용.

이벤트 흐름

카메라 이동

- 1 `OnCameraMoveStarted()` → `OnCameraMoved(true)`
- 2 `OnCameraMovePerformed(Vector2)` → `OnCameraMoveDirection(Vector2)` (반복)
- 3 `OnCameraMoveStopped()` → `OnCameraMoved(false)`

카메라 회전

- 1 `OnCameraEngageStarted()` → `OnCameraEngage()`
- 2 회전 동작 수행
- 3 `OnCameraDisengaged()` → `OnCameraDisengage()`

카메라 줌

- 1 `OnCameraZoomed(float)` → `OnCameraZoom(float)`

관련 클래스

- MyRoomEditorInputController
- MyRoomCameraController

MyRoomEditorCameraController

카메라 조작 동작 클래스

개요

MyRoomCameraController 클래스는 MyRoomEditor에서 카메라의 이동, 회전, 줌 기능을 관리하는 컨트롤러입니다. Cinemachine 컴포넌트를 활용하여 부드러운 카메라 조작을 제공하며, 경계 제한과 입력 이벤트 처리를 포함합니다.

역할

- 카메라 이동, 회전, 줌 기능 구현
- Cinemachine 컴포넌트 초기화 및 설정
- 입력 이벤트에 따른 카메라 제어
- 카메라 이동 경계 제한 적용

카메라 조작 기능 제공





선언

```
public class MyRoomCameraController : MonoBehaviour
```

멤버

속성

```
/// <summary>
/// 입력 이벤트를 받아 카메라 조작을 수행하는 디스패처
/// </summary>
[Inject]
private MyRoomEditorCameraInputDispatcher _dispatcher;

/// <summary>
/// 카메라 팔로워 트랜스폼: 카메라가 따라다니는 대상
/// 이동 시 이 오브젝트의 위치를 변경하여 카메라를 이동시킴
/// </summary>
[SerializeField]
private Transform _cameraFollower;

/// <summary>
/// 카메라 이동 속도: WASD 키 이동 시 적용되는 속도
/// </summary>
[SerializeField]
private float moveSpeed = 5f;

/// <summary>
/// 카메라 회전 속도: 마우스 회전 시 적용되는 속도
/// </summary>
[SerializeField]
private float rotateSpeed = 2f;

/// <summary>
/// 카메라 줌 최소 거리: 줌 인 시 최소 거리 제한
/// </summary>
[SerializeField]
private float zoomDistanceMin = 1f;

/// <summary>
/// 카메라 줌 최대 거리: 줌 아웃 시 최대 거리 제한
/// </summary>
[SerializeField]
private float zoomDistanceMax = 10f;

/// <summary>
/// 카메라 줌 속도: 마우스 휠 입력에 대한 줌 반응 민감도
```

```

/// 카메라 줌 감도: 마우스 휠 입력에 대한 줌 반응 민감도
/// </summary>
[SerializeField]
private float zoomSensitive = 0.3f;

/// <summary>
/// Cinemachine Virtual Camera: 카메라 조작 기본 컴포넌트
/// </summary>
[SerializeField]
private CinemachineVirtualCamera virtualCamera;

/// <summary>
/// Cinemachine Input Provider: 회전 입력을 처리하는 컴포넌트
/// </summary>
[SerializeField]
private CinemachineInputProvider inputProvider;

/// <summary>
/// Cinemachine POV 컴포넌트: 마우스 회전을 처리 (런타임에 초기화)
/// </summary>
private CinemachinePOV _cinemachinePOV;

/// <summary>
/// Cinemachine 프레이밍 트랜스포저: 줌 거리를 제어 (런타임에 초기화)
/// </summary>
private CinemachineFramingTransposer _cinemachineFramingTransposer;

/// <summary>
/// 카메라 이동 경계 콜라이더: 카메라 이동 범위를 제한하는 박스 콜라이더
/// </summary>
[SerializeField]
private BoxCollider cameraBoundary;

/// <summary>
/// 이동 방향 캐시: 현재 입력된 이동 방향
/// </summary>
private Vector3 _moveDir;

/// <summary>
/// 이동 활성화 상태: 현재 이동이 활성화되어 있는지 여부
/// </summary>
private bool _activeMove;

/// <summary>
/// 이동 코루틴: 카메라 이동을 처리하는 코루틴 참조
/// </summary>
private Coroutine _moveCoroutine;

```

메서드

```

/// <summary>
/// Cinemachine 카메라 컴포넌트를 초기화하는 프라이빗 메서드.
/// POV와 FramingTransposer 컴포넌트를 가져와 설정하고, 회전 속도를 적용.

```



```

/// </summary>
private void InitializeCinemachineCameraComponents()

/// <summary>
/// 카메라 이동 방향을 캐시하는 이벤트 핸들러.
/// 입력된 2D 벡터를 3D 이동 방향으로 변환하여 저장.
/// </summary>
/// <param name="moveVec2">입력된 이동 방향 벡터 (X, Y)</param>
private void OnCameraMoveDirectionCache(Vector2 moveVec2)

/// <summary>
/// 카메라 이동 상태 변경 이벤트 핸들러.
/// 이동 활성화 시 코루틴을 시작하고, 비활성화 시 코루틴을 중지.
/// </summary>
/// <param name="isActive">이동 활성화 상태</param>
private void OnCameraMoving(bool isActive)

/// <summary>
/// 카메라 이동을 처리하는 코루틴.
/// 카메라 전방/우측 방향을 고려하여 이동 방향을 계산하고, 경계 내에서 위치를 업데이트.
/// </summary>
private IEnumerator CoMove()

/// <summary>
/// 카메라 위치를 경계 박스 내로 클램핑하는 프라이빗 메서드.
/// X축과 Z축을 경계 범위 내로 제한.
/// </summary>
/// <param name="targetPos">클램핑할 목표 위치</param>
/// <returns>경계 내로 클램핑된 위치</returns>
private Vector3 ClampedPosition(Vector3 targetPos)

/// <summary>
/// 카메라 회전 시작 이벤트 핸들러.
/// Cinemachine 입력 제공자를 활성화하여 마우스 회전 입력을 허용.
/// </summary>
private void OnCameraEngaged()

/// <summary>
/// 카메라 회전 종료 이벤트 핸들러.
/// Cinemachine 입력 제공자를 비활성화하여 마우스 회전 입력을 차단.
/// </summary>
private void OnCameraDisengaged()

/// <summary>
/// 카메라 줌 이벤트 핸들러.
/// 줌 거리를 계산하여 최소/최대 범위 내에서 조정.
/// </summary>
/// <param name="value">줌 입력 값 (양수: 줌아웃, 음수: 줌인)</param>
private void OnCameraZoomed(float value)

```

주요 코드 스니펫

카메라 이동 프로세스

```

private void OnCameraMoving(bool isActive)
{
    _activeMove = isActive;
    if (isActive)
    {
        // 이동 코루틴 시작
        _moveCoroutine = StartCoroutine(CoMove());
    }
    else
    {
        // 이동 코루틴 중지 및 방향 초기화
        StopCoroutine(_moveCoroutine);
        _moveDir = Vector3.zero;
    }
}

private void OnCameraMoveDirectionCache(Vector2 moveVec2)
{
    _moveDir = new Vector3(moveVec2.x, 0, moveVec2.y);
}

private IEnumerator CoMove()
{
    if (Camera.main == null) yield break; // 메인 카메라가 존재하지 않으면 코루틴 종료

    var camTransform = Camera.main.transform;

    while (_activeMove) // 이동이 활성화된 동안 반복
    {
        var camForward = camTransform.TransformDirection(Vector3.forward); // 카메라 앞 방향
        camForward.y = 0;

        var camRight = camTransform.TransformDirection(Vector3.right); // 카메라 오른쪽 방향

        var targetDir = _moveDir.x * camRight + _moveDir.z * camForward; // 최종 이동 방향
        targetDir = targetDir.normalized;

        var targetPos = (targetDir * moveSpeed * Time.deltaTime) + _cameraFollower.position;
        _cameraFollower.position = ClampedPosition(targetPos); // 경계 내 위치로 클램핑

        yield return null;
    }
}

```

카메라 이동 경계 클램핑

```

private Vector3 ClampedPosition(Vector3 targetPos)
{

```

```

var minBounds = cameraBoundary.bounds.min;
var maxBounds = cameraBoundary.bounds.max;
return new Vector3(
    Mathf.Clamp(targetPos.x, minBounds.x, maxBounds.x),
    targetPos.y,
    Mathf.Clamp(targetPos.z, minBounds.z, maxBounds.z)
);
}

```

카메라 회전 프로세서

```

private void OnCameraEngaged()
{
    inputProvider.enabled = true;
}

private void OnCameraDisengaged()
{
    inputProvider.enabled = false;
}

```

카메라 줌 프로세서

```

private void OnCameraZoomed(float value)
{
    // 줌 거리가 유효 범위 내에 있는지 확인
    if (!(_cinemachineFramingTransposer.m_CameraDistance ≥ zoomDistanceMin) &&
        !(_cinemachineFramingTransposer.m_CameraDistance ≤ zoomDistanceMax)) return;

    // 줌 거리 조정
    _cinemachineFramingTransposer.m_CameraDistance -= value * zoomSensitive;

    // 최소/최대 범위 클램핑
    if (_cinemachineFramingTransposer.m_CameraDistance < zoomDistanceMin)
    {
        _cinemachineFramingTransposer.m_CameraDistance = zoomDistanceMin;
    }
    else if (_cinemachineFramingTransposer.m_CameraDistance > zoomDistanceMax)
    {
        _cinemachineFramingTransposer.m_CameraDistance = zoomDistanceMax;
    }
}

```

기능 설명

카메라 이동

- 입력 방향에 따라 카메라 팔로워 이동
- 카메라 저바/오프 바하으 기즈오르 사대저 이도

- 카메라 이동 / 회전 / 줌 / 포커스 / 렌즈 / 셔터 / 조리개 / ISO

- 박스 콜라이더 경계를 벗어나지 않도록 제한
- 코루틴을 사용하여 부드러운 이동 구현

카메라 회전

- 마우스 드래그 입력을 [Cinemachine POV](#) 컴포넌트로 처리
- 입력 제공자의 활성화/비활성화로 회전 모드 제어

카메라 줌

- [Framing Transposer](#)를 통한 거리 조절
- 최소/최대 줌 거리 제한
- 입력 값에 따른 감도 적용

이벤트 처리

- [MyRoomEditorCameraInputDispatcher](#) 이벤트 구독
- 카메라 참여/비참여 상태 관리

의존성/상속 관계

- [MonoBehaviour](#)를 상속받음.
- [MyRoomEditorCameraInputDispatcher](#)에 의존.
- Cinemachine 패키지를 사용 ([Virtual Camera](#), [Input Provider](#), [Cinemachine POV](#), [Framing Transposer](#)).

관련 클래스

- [MyRoomEditorCameraInputDispatcher](#)
- [Cinemachine](#)
- [MyRoomEditorInputController](#)

MyRoomEditorEditingInputDispatcher

편집 관련 입력 이벤트를 처리하는 클래스

개요

`MyRoomEditorEditingInputDispatcher` 클래스는 MyRoom 에디터에서 편집 관련 입력 이벤트를 중앙에서 관리하고 배포하는 디스패처입니다. 포인터 클릭, 취소, 삭제 등의 편집 입력을 적절한 리스너들에게 전달합니다.

역할

- 편집 관련 입력 이벤트 처리
- 이벤트 구독 패턴을 통한 느슨한 결합 구현
- 포인터, 키보드 입력 상태 관리 및 이벤트 중계

선언

```
public class MyRoomEditorEditingInputDispatcher
```

멤버

이벤트

```
/// <summary>  
/// 마우스 버튼이 눌렸을 때 발생  
/// </summary>  
public event Action OnPointerDownEvent;  
  
/// <summary>  
/// 마우스 버튼이 떴을 때 발생  
/// </summary>  
public event Action OnPointerUpEvent;  
  
/// <summary>  
/// ESC 키 등 취소 입력 시 발생  
/// </summary>  
public event Action OnCancelEvent;
```

```

/// <summary>
/// 마우스 우클릭 시 발생
/// </summary>
public event Action OnRightClickEvent;

/// <summary>
/// Delete 키 등 삭제 입력 시 발생
/// </summary>
public event Action OnDeletePressEvent;

```

메서드

```

/// <summary>
/// 마우스 버튼이 눌렸음을 이벤트로 전달.
/// </summary>
public void OnPointerDown()

/// <summary>
/// 마우스 버튼이 떴음을 이벤트로 전달.
/// </summary>
public void OnPointerUp()

/// <summary>
/// ESC 키 등 취소 입력을 이벤트로 전달.
/// </summary>
public void OnCanceled()

/// <summary>
/// 마우스 우클릭 입력을 이벤트로 전달.
/// </summary>
public void OnRightClicked()

/// <summary>
/// Delete 키 등 삭제 입력을 이벤트로 전달.
/// </summary>
public void OnDeletePressed()

```

기능 설명

포인터 입력 이벤트

- `OnPointerDown()`: 마우스 왼쪽 버튼이나 터치 시작 시 이벤트 발생
- `OnPointerUp()`: 마우스 왼쪽 버튼이나 터치 종료 시 이벤트 발생
- `OnRightClicked()`: 마우스 우클릭 시 이벤트 발생

키보드 입력 이벤트

- `OnCanceled()`: ESC 키나 취소 입력 시 이벤트 발생
- `OnDeletePressed()`: Delete 키나 Backspace 키 입력 시 이벤트 발생

이벤트 구독 패턴

- Observer 패턴 구현으로 입력 처리와 비즈니스 로직 분리
- 다중 리스너 지원으로 확장성 제공

이벤트 흐름

- 1 `MyRoomEditorInputController`에서 입력 감지
- 2 해당 이벤트 메서드 호출
- 3 등록된 이벤트 핸들러 실행

관련 클래스

- `MyRoomEditorInputController`
- `MyRoomEditorState`
- `InputSystem` [↗](#)

MyRoomEditorInputUtils

입력 유틸리티 클래스

개요

MyRoomEditorInputUtils는 MyRoomEditor에서 입력 관련 유틸리티 기능을 제공하는 클래스입니다. Unity의 Physics Raycast를 사용하여 포인터 위치 기반 레이캐스트를 수행하고, 결과를 필터링하며, UI 요소 클릭 감지와 포인터 입력 값 제공 기능을 포함합니다. 편집 및 배치 작업에서 정확한 타겟 감지를 지원합니다.

역할

- 포인터 기반 레이캐스트 수행 및 결과 반환
- 무시할 오브젝트 필터링을 통한 정확한 타겟 감지
- 우선순위 레이어 기반 레이캐스트 결과 선택
- 포인터 위치 및 델타 값 제공
- UI 요소 위 포인터 감지

선언

```
public class MyRoomEditorInputUtils : MonoBehaviour
```

멤버

속성

```
/// <summary>  
/// Raycast를 수행할 카메라: 화면 좌표를 3D 공간으로 변환  
/// </summary>  
[SerializeField]  
private Camera raycastCamera;  
  
/// <summary>  
/// Raycast 최대 거리: 이 거리 이상의 충돌은 무시  
/// </summary>  
[SerializeField]  
private float raycastMaxDistance = 50f;
```



```

/// <summary>
/// Raycast 최소 거리: 이 거리 이하의 충돌은 무시 (너무 가까운 충돌 방지)
/// </summary>
[SerializeField]
private float raycastMinDistance = 0.5f;

/// <summary>
/// 현재 이벤트 시스템: UI 검증에 사용
/// </summary>
private EventSystem _eventSystem = EventSystem.current;

/// <summary>
/// Raycast 결과 최대 개수 제한
/// </summary>
private const int RaycastLimit = 10;

```

메서드

```

/// <summary>
/// 특정 오브젝트를 무시한 Raycast 결과를 반환하는 메서드.
/// ignoreObject를 제외한 가장 가까운 충돌을 찾아 반환.
/// </summary>
/// <param name="raycastHit">반환받을 RaycastHit 구조체</param>
/// <param name="targetLayerMask">Raycast 대상 레이어 마스크</param>
/// <param name="ignoreObject">Raycast 결과에서 무시할 SpawnablePropBase 오브젝트</param>
/// <returns>유효한 충돌이 있으면 true, 없으면 false</returns>
public bool GetRaycastHit(out RaycastHit raycastHit, LayerMask targetLayerMask,
                           SpawnablePropBase ignoreObject)

/// <summary>
/// 우선순위 레이어를 기반으로 Raycast 결과를 반환하는 메서드.
/// priorityLayer의 오브젝트를 우선 선택하며, 없으면 가장 가까운 오브젝트 선택.
/// </summary>
/// <param name="raycastHit">반환받을 RaycastHit 구조체</param>
/// <param name="targetLayerMask">Raycast 대상 레이어 마스크</param>
/// <param name="priorityLayer">우선시할 레이어 (-1이면 우선순위 없음)</param>
/// <returns>유효한 충돌이 있으면 true, 없으면 false</returns>
public bool GetRaycastHit(out RaycastHit raycastHit, LayerMask targetLayerMask,
                           int priorityLayer)

/// <summary>
/// Raycast 결과에서 무시할 오브젝트를 제거하는 메서드.
/// IPlaceableArea 인터페이스를 구현한 오브젝트 중 ignoreObject와 다른 경우만 유지.
/// </summary>
/// <param name="raycastHits">필터링할 RaycastHit 배열</param>
/// <param name="ignoreObject">무시할 SpawnablePropBase 오브젝트</param>
/// <returns>필터링된 RaycastHit 배열</returns>
private RaycastHit[] RemoveIgnoring(RaycastHit[] raycastHits, SpawnablePropBase ignoreObject)

/// <summary>
/// 현재 포인터의 화면 좌표를 반환하는 메서드.
/// Raycast를 위한 시작점으로 사용.
/// </summary>

```

```

/// <returns>포인터 화면 좌표</returns>
public Vector2 PointerDelta()

/// <summary>
/// 포인터가 UI 요소 위에 있는지 검증하는 메서드.
/// GraphicRaycaster를 사용하여 UI 요소에 대한 Raycast 수행.
/// </summary>
/// <returns>포인터가 UI 위에 있으면 true, 없으면 false</returns>
public bool IsPointerOnUI()

```

코드 스니펫

우선순위 기반 Raycast

```

public bool GetRaycastHit(out RaycastHit raycastHit, LayerMask targetLayerMask,
{
    // 포인터 위치에서 카메라를 통해 Ray 생성
    var ray = raycastCamera.ScreenPointToRay(PointerPosition());

    // 가비지 생성을 방지하기 위해 비할당 Raycast 사용
    RaycastHit[] results = new RaycastHit[RaycastLimit];
    var hitLength = Physics.RaycastNonAlloc(ray, results, raycastMaxDistance, t

    // 충돌이 없으면 실패 반환
    if (hitLength == 0)
    {
        raycastHit = new RaycastHit();
        return false;
    }

    // 유효한 콜라이더를 가진 결과만 필터링하고 최소 거리 이상 확인
    results = Array.FindAll(results, raycastHit =>
        raycastHit.collider != null
        && raycastHit.distance > raycastMinDistance);

    // 필터링 후 결과가 없으면 실패 반환
    if (results.Length == 0)
    {
        raycastHit = new RaycastHit();
        return false;
    }

    // 거리순으로 정렬
    Array.Sort(results, (raycast1, raycast2) => raycast1.distance.CompareTo(ray

    // 우선순위 레이어가 지정되지 않은 경우
    if (priorityLayer == -1)
    {
        raycastHit = results.First();
    }
    else
    {

```

```

        // 우선순위 레이어의 오브젝트들만 필터링
        var priorityRaycastHits = results.Where(hits => hits.transform.gameObject.layer == priorityLayer);
        // 우선순위 레이어 오브젝트가 있으면 첫 번째 선택, 없으면 전체 결과의 첫 번째 선택
        raycastHit = priorityRaycastHits.Length != 0 ? priorityRaycastHits.First() : results.First();
    }
    return true;
}

```

특정 오브젝트를 무시한 가장 가까운 오브젝트를 반환하는 Raycast

```

public bool GetRaycastHit(out RaycastHit raycastHit, LayerMask targetLayerMask, Vector3 start, Vector3 end)
{
    // 포인터 위치에서 카메라를 통해 Ray 생성
    var ray = raycastCamera.ScreenPointToRay(PointerPosition());

    // 가비지 생성을 방지하기 위해 비할당 Raycast 사용
    RaycastHit[] results = new RaycastHit[RaycastLimit];
    var hitLength = Physics.RaycastNonAlloc(ray, results, raycastMaxDistance, targetLayerMask);

    // 충돌이 없으면 실패 반환
    if (hitLength == 0)
    {
        raycastHit = new RaycastHit();
        return false;
    }

    // BoxCollider만 허용하고 최소 거리 이상인 결과만 필터링
    results = Array.FindAll(results, raycastHit => raycastHit.collider is BoxCollider && raycastHit.distance > raycastMinDistance);

    // 필터링 후 결과가 없으면 실패 반환
    if (results.Length == 0)
    {
        raycastHit = new RaycastHit();
        return false;
    }

    // 무시할 오브젝트 제거
    results = RemoveIgnoring(results, ignoreObject);

    // 무시 후 결과가 없으면 실패 반환
    if (results.Length == 0)
    {
        raycastHit = new RaycastHit();
        return false;
    }

    // 거리순으로 정렬하여 가장 가까운 충돌 선택
    Array.Sort(results, (raycast1, raycast2) => raycast1.distance.CompareTo(raycast2));
    raycastHit = results.First();
    return true;
}

```

UI 포인터 검증

```
public bool IsPointerOnUI()
{
    // 씬의 모든 GraphicRaycaster를 찾음
    var graphicRaycaster = FindObjectsOfType<GraphicRaycaster>();

    // 포인터 이벤트 데이터 생성
    var pointerEvent = new PointerEventData(_eventSystem);
    var raycastResult = new List<RaycastResult>();
    pointerEvent.position = PointerPosition();

    // 각 Raycaster에 대해 Raycast 수행
    foreach (var raycaster in graphicRaycaster)
    {
        raycaster.Raycast(pointerEvent, raycastResult);
    }

    // Raycast 결과가 있으면 UI 위에 있는 것으로 판단
    return raycastResult.Count > 0;
}
```

무시 오브젝트 필터링

```
private RaycastHit[] RemoveIgnoring(RaycastHit[] raycastHits, SpawnablePropBase[] propBase)
{
    var result = Array.FindAll(raycastHits, hit =>
        hit.collider.TryGetComponent<IPlaceableArea>(out var area)
        && (area.IsPlacementAreaInProp(out var propBase) == false || propBase == null));
    return result;
}
```

주요 기능 설명

Raycast 수행

- `Physics.RaycastNonAlloc` [🔗](#): 가비지 생성 방지를 위한 비할당 Raycast
- 결과 제한: 최대 10개의 충돌 결과만 처리
- 거리 필터링: 최소/최대 거리 기반 필터링
- `GetRaycastHit` (무시 오브젝트 버전): 포인터 위치에서 레이캐스트를 수행하고, 지정된 ignoreObject를 제외한 가장 가까운 BoxCollider를 가진 오브젝트를 반환. 최소/최대 거리 필터링을 적용.
- `GetRaycastHit` (우선순위 레이어 버전): 레이캐스트 결과를 우선순위 레이어에 따라 선택. 우선순위 레이어의 오브젝트가 있으면 이를, 없으면 가장 가까운 오브젝트를 반환.

포인터 입력 처리

- `PointerDelta()`: 마우스 또는 터치 입력의 델타 값을 반환.
- `PointerPosition()`: 현재 포인터의 화면 좌표를 반환.

UI 감지

- `IsPointerOnUI()`: `GraphicRaycaster` [🔗](#)를 사용하여 포인터가 UI 요소 위에 있는지 확인. UI 클릭을 방지하기 위해 편집 작업에서 사용.

결과 필터링

- `RemoveIgnoring()`: `IPlaceableArea` 인터페이스를 구현한 오브젝트 중 `ignoreObject`와 다른 경우만 유지. 배치 작업에서 자기 자신을 무시하기 위해 사용.

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `InputSystem` [🔗](#), `UGUI` [🔗](#) 사용.
- `IPlaceableArea`, `SpawnablePropBase`에 의존.

사용 예시

`MyRoomEditorPropSelector`에서 레이캐스트를 통해 오브젝트 선택

```
private IEnumerator CoDetectObject()
{
    while (true)
    {
        yield return null;

        if (InputUtils.GetRaycastHit(out RaycastHit hits, _targetLayer, _firstPersonCamera) == false)
        {
            if (hits.transform.TryGetComponent(out IMyRoomEditorEditableObject editableObject)
                && InputUtils.IsPointerOnUI() == false)
            {
                if (focusedObject.Equals(_focusedObject)) continue;
                if (IsSelectedObject(focusedObject))
                {
                    _focusedObject?.Unfocused();
                    _focusedObject = focusedObject;
                }
                else
                {
                    if (IsSelectedObject(_focusedObject) == false)
                    {
                        _focusedObject?.Unfocused();
                    }
                    _focusedObject = focusedObject;
                }
            }
        }
    }
}
```


SpawnablePropBase

모든 배치 가능한 오브젝트의 abstract 클래스

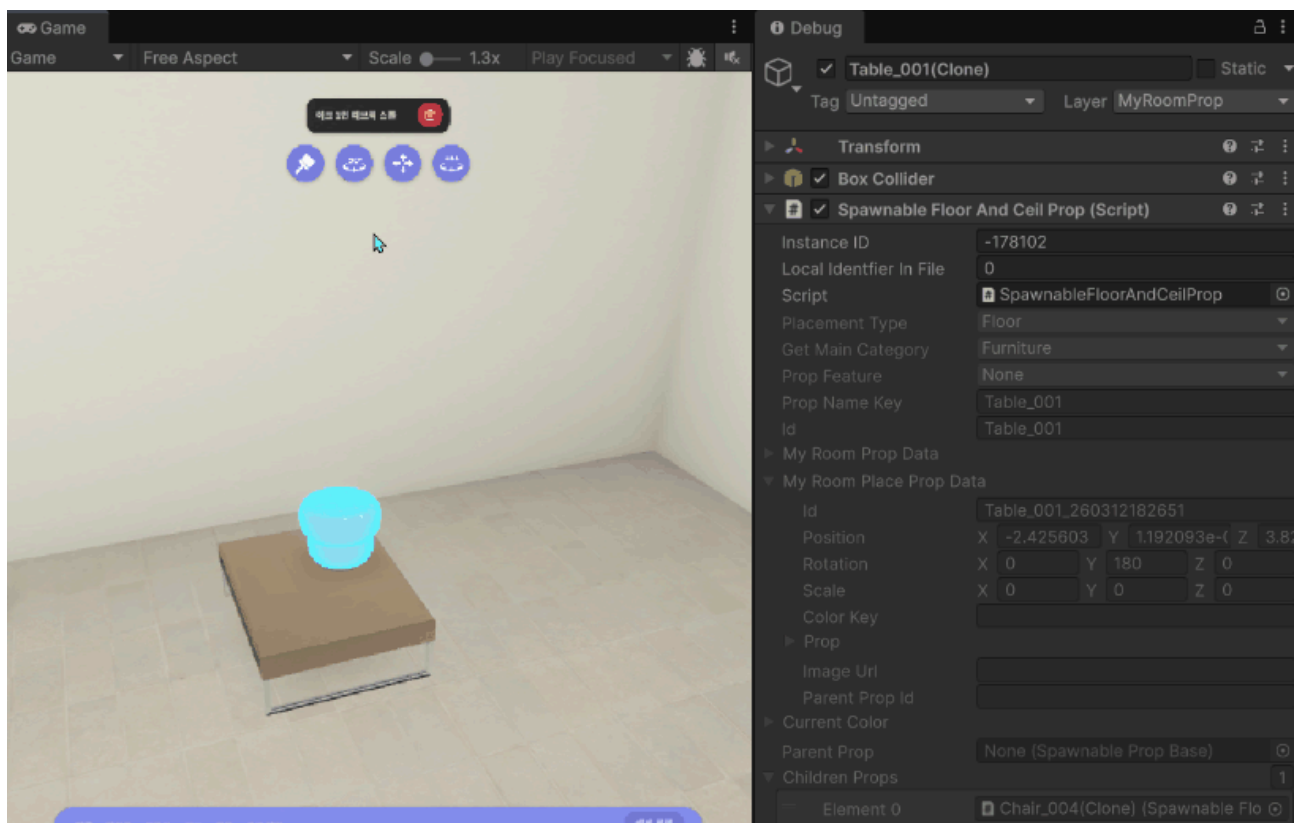
개요

SpawnablePropBase는 MyRoomEditor에서 모든 배치 가능한 오브젝트의 추상 기본 클래스입니다. 이 클래스는 배치 가능한 오브젝트들의 공통 기능을 제공하며, **IMyRoomEditorEditableObject** 인터페이스를 구현하여 편집 가능한 오브젝트로 동작합니다.

역할

- 모든 배치 가능한 오브젝트들의 기본 기능을 제공
- 공통 속성 및 상태 관리
- 부모-자식 오브젝트 관계 구성
- 하이라이트 상태 표현
- 배치 정보 관리

배치 정보 관리





선언

```
public abstract class SpawnablePropBase : MonoBehaviour
```

멤버

속성

```
/// <summary>
/// 배치 타입: Floor, Ceiling, Wall 등 오브젝트가 배치될 수 있는 표면 타입
/// </summary>
public PlacementType PlacementType { get; private set; }

/// <summary>
/// 로컬라이징용 오브젝트 이름 키
/// </summary>
public string PropNameKey { get; private set; }

/// <summary>
/// 오브젝트의 고유 식별자
/// </summary>
public string Id { get; private set; }

/// <summary>
/// 부모 오브젝트 참조
/// </summary>
public SpawnablePropBase ParentProp { get; private set; }

/// <summary>
/// 자식 오브젝트 리스트: 이 오브젝트 슬롯에 배치된 자식 오브젝트들
/// </summary>
private List<SpawnablePropBase> _childrenProps = new();

/// <summary>
/// 오브젝트 기본 데이터 : ID, 타입, 카테고리, 색상 리스트 등
/// </summary>
protected MyRoomProp MyRoomPropData { get; private set; }

/// <summary>
/// 오브젝트 배치 데이터 : 위치, 회전, 색상, 부모 관계 등
/// </summary>
protected MyRoomPlaceProp MyRoomPlacePropData { get; private set; }

/// <summary>
/// 현재 색상 정보
/// </summary>
private Color _currentColor;
```



```

/// </summary>
protected MyRoomPropColor CurrentColor { get; set; }

```

추상 메서드

```

/// <summary>
/// 편집 가능한 오브젝트 인터페이스를 반환하는 추상 메서드
/// 서브클래스에서 구체적인 구현체를 반환
/// </summary>
public abstract IMyRoomEditorEditableObject GetEditableObject();

/// <summary>
/// 하이라이트 설정을 위한 추상 메서드
/// 서브클래스에서 PropEditingState 등의 시각적 피드백을 설정
/// </summary>
protected abstract void SetupPropHighlight();

```

메서드

```

/// <summary>
/// 오브젝트 기본 정보를 설정하는 메서드.
/// MyRoomProp 데이터로부터 ID, 타입, 카테고리 등의 속성을 초기화.
/// </summary>
/// <param name="propInfo">설정할 오브젝트 기본 정보</param>
public void SetPropInfo(MyRoomProp propInfo)

/// <summary>
/// 배치 정보를 설정하는 메서드.
/// 위치, 회전, 색상 등의 배치 데이터를 적용하고, 관련 컴포넌트를 초기화.
/// </summary>
/// <param name="placePropInfo">설정할 배치 정보</param>
public void SetPlacePropInfo(MyRoomPlaceProp placePropInfo)

/// <summary>
/// 현재 배치 정보를 조회하는 메서드.
/// Transform으로부터 최신 위치/회전을 가져와 배치 정보를 업데이트하여 반환.
/// </summary>
/// <returns>업데이트된 배치 정보</returns>
public MyRoomPlaceProp GetPlacePropInfo()

/// <summary>
/// 자식 오브젝트를 추가하는 메서드.
/// 이미 추가된 오브젝트는 중복 추가하지 않음.
/// </summary>
/// <param name="childProp">추가할 자식 오브젝트</param>
public void AddChildProp(SpawnablePropBase childProp)

/// <summary>
/// 자식 오브젝트를 제거하는 메서드.
/// </summary>
/// <param name="childProp">제거할 자식 오브젝트</param>

```

```
// <param name="childProp">추가된 자식 오브젝트</param>  
public void RemoveChildProp(SpawnablePropBase childProp)
```

코드 스니펫

배치된 오브젝트 초기화

```
public void SetPlacePropInfo(MyRoomPlaceProp placePropInfo)  
{  
    MyRoomPlacePropData = placePropInfo;  
  
    // 색상 변형을 지원하는 경우 현재 색상 설정  
    if (MyRoomPropData.CheckColorVariation())  
    {  
        if (string.IsNullOrEmpty(placePropInfo.colorKey))  
        {  
            CurrentColor = null;  
        }  
        else  
        {  
            // 색상 키에 해당하는 색상 정보를 찾아 설정  
            CurrentColor = MyRoomPropData.colorList.FirstOrDefault(key => key.r  
        }  
    }  
  
    // 하이라이트 설정 (서브클래스 구현)  
    SetupPropHighlight();  
  
    // 추가 배치 영역 설정  
    SetupAdditivePlaceArea();  
  
    // 인터랙터블 컴포넌트가 있으면 초기화 실행  
    if (TryGetComponent(out IInteractableProp interactableProp))  
    {  
        interactableProp.Interact();  
    }  
}  
  
private void SetupAdditivePlaceArea()  
{  
    var placementAreaInProps = GetComponentsInChildren<PlacementAreaInProp>();  
    if (placementAreaInProps == null) return;  
  
    foreach (var areaInProp in placementAreaInProps)  
    {  
        areaInProp.SetPlacementAreaInProp(this);  
    }  
}
```

자식 오브젝트 추가/제거

```

public void AddChildProp(SpawnablePropBase childProp)
{
    // 이미 자식으로 등록된 경우 중복 추가 방지
    if (_childrenProps.Any(x => x.MyRoomPlacePropData.id == childProp.MyRoomPlacePropData.id))
    {
        Debug.Log($"Already Set Child Prop : {childProp.GetPlacePropId()}");
        return;
    }

    // 자식 리스트에 추가
    _childrenProps.Add(childProp);

    // Transform 부모 설정
    childProp.transform.parent = transform;

    // 배치 데이터에 부모 ID 설정
    childProp.MyRoomPlacePropData.parentPropId = MyRoomPlacePropData.id;

    // 자식의 부모 참조 설정
    childProp.ParentProp = this;

    Debug.Log($"[SpawnablePropBase] Prop: {GetPlacePropId()} Add Child Prop : {childProp.GetPlacePropId()}");
}

public void RemoveChildProp(SpawnablePropBase childProp)
{
    // 존재하지 않는 자식은 무시
    if (_childrenProps.All(x => x.MyRoomPlacePropData.id != childProp.MyRoomPlacePropData.id))
    {
        return;
    }

    _childrenProps.Remove(childProp);
    Debug.Log($"[SpawnablePropBase] Prop: {GetPlacePropId()} Remove Child Prop : {childProp.GetPlacePropId()}");
}

```

기능 설명

오브젝트 정보 관리

- `SetPropInfo()`: 오브젝트의 기본 데이터 설정
- `SetPlacePropInfo()`: 오브젝트의 배치 데이터 설정 및 초기화
- `GetPlacePropInfo()`/`GetPlacePropId()`: 배치 정보를 조회

부모-자식 관계 관리

- `AddChildProp()`/`RemoveChildProp()`: 자식 오브젝트 추가/제거
- `SetPropParent()`/`ClearPropParent()`: 부모 오브젝트 설정/해제
- • `PlacementAreaInProp` 초기화 및 동작 지원
- `gameobject.transform` 관리

편집 기능

- `RegisterProp()`: 편집 매니저에 오브젝트 등록
- `SetupPropHighlight()`: 하이라이트 상태 초기화 (서브클래스에서 구현)

오브젝트 삭제

- `Delete()`: 오브젝트 삭제 및 리소스 정리

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `MyRoomProp`, `MyRoomPlaceProp` 사용(데이터베이스 클래스/구조체).
- `PlacementAreaInProp`에 의존.
- 서브클래스
 - `SpawnableFloorAndCeilProp`: 바닥/천장 배치 오브젝트
 - `SpawnableWallProp`: 벽 배치 오브젝트
 - `SpawnablePhotoFrameProp`: 사진 프레임 오브젝트
 - `SpawnableScreenProp`: 스크린 오브젝트

관련 클래스

- `IInteractableProp`
- `MyRoomEditorPropEditingManager`
- `IMyRoomEditorEditableObject`
- `IInteractableProp`
- `PlacementAreaInProp`
- `SpawnableFloorAndCeilProp`
- `SpawnableWallProp`
- `SpawnablePhotoFrameProp`
- `SpawnableScreenProp`

SpawnableFloorAndCeilProp

바닥과 천장에 배치되는 일반적인 오브젝트 클래스

개요

`SpawnableFloorAndCeilProp`는 MyRoomEditor에서 바닥과 천장에 배치되는 오브젝트를 위한 클래스입니다. `SpawnablePropBase`를 상속받아 오브젝트 기본 기능과 이동, 회전, 색상 편집 등의 동작을 구체적으로 구현합니다.

역할

- 바닥/천장 오브젝트의 기본 기능 제공
- 이동, 회전, 색상 변경 인터페이스 구현
- 하이라이트 상태 관리
- 배치 검증 및 부모 관계 설정
- 오브젝트 기즈모 위치 반환

구현 인터페이스에 따른 편집 기능 표시



구현 인터페이스

- `IColorEditableProp`: 색상 편집 기능
- `IMoveableProp`: 이동 기능

- **IRotatableProp**: 회전 기능
- **IMyRoomEditorEditableObject**: 편집 가능한 오브젝트

선언

```
public class SpawnableFloorAndCeilProp : SpawnablePropBase, IColorEditableProp,
```

멤버

속성

```
/// <summary>
/// 편집 상태 컴포넌트: 하이라이트 상태 관리
/// </summary>
private PropEditingState _propEditingState;

/// <summary>
/// Material 변경 헬퍼: 색상 변경 처리
/// </summary>
private MyRoomMaterialChangeHelper _materialChangeHelper;
```

메서드

```
/// <summary>
/// 오브젝트 삭제 메서드.
/// 이벤트 구독 해제 후 게임 오브젝트를 파괴.
/// </summary>
public void Delete()

/// <summary>
/// 하이라이트 상태를 설정하는 추상 메서드 구현.
/// PropEditingState의 초기 설정을 수행.
/// </summary>
protected override void SetupPropHighlight()

/// <summary>
/// 포커스 상태로 변경하는 메서드.
/// 유효한 상태의 하이라이트를 표시.
/// </summary>
public void Focused()

/// <summary>
/// 포커스 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Unfocused()
```

```

/// <summary>
/// 선택 상태로 변경하는 메서드.
/// 선택된 상태의 하이라이트를 표시.
/// </summary>
public void Selected()

/// <summary>
/// 선택 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Deselected()

/// <summary>
/// 배치 가능 영역인지 검증하고 배치하는 메서드.
/// 히트 포인트로 이동 후 배치 타입 검증 및 부모 관계 설정.
/// </summary>
/// <param name="hitPosition">배치할 히트 포인트 위치</param>
/// <param name="area">배치 영역 인터페이스</param>
/// <returns>배치 가능한 경우 true</returns>
public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)

/// <summary>
/// 오브젝트를 지정된 위치로 이동하는 메서드.
/// </summary>
/// <param name="position">이동할 목표 위치</param>
public void Move(Vector3 position)

/// <summary>
/// 사용 가능한 색상 리스트를 반환하는 메서드.
/// </summary>
/// <returns>색상 리스트 (없으면 빈 리스트)</returns>
public List<MyRoomPropColor> GetColorList()

/// <summary>
/// 색상을 변경하는 메서드.
/// Material을 복제하여 적용하고 선택 상태로 하이라이트 변경.
/// </summary>
/// <param name="mat">적용할 Material</param>
/// <param name="colorIndex">색상 인덱스</param>
public void SetColor(Material mat, int colorIndex)

/// <summary>
/// 기즈모의 위치와 회전을 계산하여 반환하는 메서드.
/// 배치 타입에 따라 오브젝트 위쪽(Floor) 또는 아래쪽(Ceiling)에 기즈모 배치.
/// </summary>
/// <returns>기즈모 위치와 회전 튜플</returns>
public (Vector3, Quaternion) GetGizmoPositionAndRotation()

```

코드 스니펫

배치 가능성 검증

```

public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)
{
    // 히트 포인트로 즉시 이동
    Move(hitPosition);

    // 배치 타입 검증
    if (area.GetPlacementType() != PlacementType)
    {
        _propEditingState.Invalid();
        return false;
    }

    // 부모 관계 설정 (배치 영역이 다른 오브젝트 위인 경우)
    if (area.IsPlacementAreaInProp(out var parent))
    {
        SetPropParent(parent);
    }
    else
    {
        ClearPropParent();
    }

    // 유효한 배치 상태 표시
    _propEditingState.Valid();
    return true;
}

public void Move(Vector3 position)
{
    transform.position = position;
}

```

스냅 회전

```

public void RotationBySnapValue(float snapStepValue)
{
    var currentRot = transform.eulerAngles.y;
    var targetRot = Mathf.Round((currentRot + snapStepValue) / snapStepValue) * snapStepValue;
    transform.rotation = Quaternion.Euler(0, targetRot, 0);
}

```

색상 변경 가능 검증

```

public bool IsEditableColor(out IColorEditableProp prop)
{
    if (MyRoomPropData.CheckColorVariation()
        && MyRoomPropData.colorList.Count > 0
        && _materialChangeHelper != null)
    {
        prop = this;
    }
}

```



```

        return true;
    }
    else
    {
        prop = null;
        return false;
    }
}

```

색상 변경

```

public void SetColor(Material mat, int colorIndex)
{
    var cloneMat = new Material(mat);
    _materialChangeHelper.ChangeMaterial(cloneMat);
    _propEditingState.Selected();
    CurrentColor = GetColorList()[colorIndex];
}

```

기즈모 위치 계산

```

public (Vector3, Quaternion) GetGizmoPositionAndRotation()
{
    var bounds = _collider.bounds;
    var gizmoRotation = Quaternion.Euler(0, 0, 0);

    // 배치 타입에 따른 오프셋 방향 계산
    Vector3 offsetDirection = PlacementType switch
    {
        PlacementType.Floor => (Vector3.up * bounds.size.y), // 오브젝트 위쪽
        PlacementType.Ceiling => (Vector3.down * bounds.size.y), // 오브젝트 아래쪽
        _ => Vector3.zero
    };

    Vector3 offset = transform.position + offsetDirection;
    return (offset, gizmoRotation);
}

```

기능 설명

편집 기능

- 이동, 회전, 색상 변경 지원
- 하이라이트 상태 표시

편집 가능성 검증

- 색상 편집: 색상 변경 인터페이스 지원 지원 + 오브젝트 데이터에 색상 리스트 존재

- **이동 가능:** 이동 인터페이스 지원
- **회전 가능:** 회전 인터페이스 지원

배치 프로세스

- 1 **위치 이동:** 히트 포인트로 즉시 이동
- 2 **타입 검증:** 배치 영역 타입과 PlacementType 일치 확인
- 3 **부모 설정:** 배치 영역이 다른 오브젝트 위면 부모 관계 설정
 - 오브젝트 슬롯 영역에 배치 시 부모 설정
 - 룸 레벨 배치 시 부모 해제
- 4 **시각 피드백:** 유효/무효에 따라 하이라이트 변경

하이라이트 상태

- **Focused:** 녹색 (Valid)
- **Unfocused:** 기본 (Default)
- **Selected:** 파란색 (Selected)
- **Deselected:** 기본 (Default)

기즈모 위치

- **Floor:** 오브젝트 위쪽 (bounds.height만큼 위)
- **Ceiling:** 오브젝트 아래쪽 (bounds.height만큼 아래)
- 기즈모는 수직 방향으로 표시

의존성/상속 관계

- `SpawnablePropBase`를 상속받음.
- `IColorEditableProp`, `IMoveableProp`, `IRotatableProp` 인터페이스 구현.
- `PropEditingState`, `MyRoomMaterialChangeHelper`에 의존.

관련 클래스

- `PropEditingState`
- `SpawnablePropBase`
- `IColorEditableProp`
- `IMoveableProp`
- `IRotatableProp`

SpawnableWallProp

벽에 배치되는 일반적인 오브젝트 클래스

개요

`SpawnableWallProp`는 MyRoomEditor에서 벽을 배치되는 일반적인 오브젝트를 위한 클래스입니다. `SpawnablePropBase`를 상속받아 오브젝트 기본 기능과 이동, 색상 편집 등의 동작을 구체적으로 구현합니다.

역할

- 벽 오브젝트의 기본 기능 제공
- 이동, 색상 변경 인터페이스 구현
- 하이라이트 상태 관리
- 배치 검증 및 부모 관계 설정
- 오브젝트 기즈모 위치 반환

구현 인터페이스에 따른 편집 기능 표시





구현 인터페이스

- `IColorEditableProp`: 색상 편집 기능
- `IMoveableProp`: 이동 기능
- `IMyRoomEditorEditableObject`: 편집 가능한 오브젝트

선언

```
public class SpawnableWallProp : SpawnablePropBase, IMoveableProp, IColorEditable
```

멤버

속성

```
/// <summary>  
/// 편집 상태 컴포넌트: 하이라이트 상태 관리  
/// </summary>  
private PropEditingState _propEditingState;  
  
/// <summary>  
/// Material 변경 헬퍼: 색상 변경 처리  
/// </summary>  
private MyRoomMaterialChangeHelper _materialChangeHelper;
```

메서드

```
/// <summary>  
/// 오브젝트 삭제 메서드.  
/// 이벤트 구독 해제 후 게임 오브젝트를 파괴.  
/// </summary>  
public void Delete()  
  
/// <summary>  
/// 하이라이트 상태를 설정하는 추상 메서드 구현.  
/// PropEditingState의 초기 설정을 수행.  
/// </summary>  
protected override void SetupPropHighlight()  
  
/// <summary>  
/// 포커스 상태로 변경하는 메서드.  
/// 유효한 상태의 하이라이트를 표시.
```

```

/// </summary>
public void Focused()

/// <summary>
/// 포커스 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Unfocused()

/// <summary>
/// 선택 상태로 변경하는 메서드.
/// 선택된 상태의 하이라이트를 표시.
/// </summary>
public void Selected()

/// <summary>
/// 선택 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Deselected()

/// <summary>
/// 배치 가능 영역인지 검증하고 배치하는 메서드.
/// 히트 포인트로 이동 후 배치 타입 검증 및 부모 관계 설정.
/// </summary>
/// <param name="hitPosition">배치할 히트 포인트 위치</param>
/// <param name="area">배치 영역 인터페이스</param>
/// <returns>배치 가능한 경우 true</returns>
public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)

/// <summary>
/// 오브젝트를 지정된 위치로 이동하는 메서드.
/// </summary>
/// <param name="position">이동할 목표 위치</param>
public void Move(Vector3 position)

/// <summary>
/// 사용 가능한 색상 리스트를 반환하는 메서드.
/// </summary>
/// <returns>색상 리스트 (없으면 빈 리스트)</returns>
public List<MyRoomPropColor> GetColorList()

/// <summary>
/// 색상을 변경하는 메서드.
/// Material을 복제하여 적용하고 선택 상태로 하이라이트 변경.
/// </summary>
/// <param name="mat">적용할 Material</param>
/// <param name="colorIndex">색상 인덱스</param>
public void SetColor(Material mat, int colorIndex)

/// <summary>
/// 기즈모의 위치와 회전을 계산하여 반환하는 메서드.
/// 배치 타입에 따라 오브젝트 위쪽(Floor) 또는 아래쪽(Ceiling)에 기즈모 배치.
/// </summary>
/// <returns>기즈모 위치와 회전 튜플</returns>

```

```
public (Vector3, Quaternion) GetGizmoPositionAndRotation()
```

코드 스니펫

배치 검증

```
public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)
{
    // 히트 포인트로 즉시 이동
    Move(hitPosition);

    // 배치 영역의 회전을 적용 (벽면 방향 맞춤)
    transform.rotation = area.GetPlacementRotation();

    // 배치 타입 검증
    if (area.GetPlacementType() != PlacementType)
    {
        _propEditingState.Invalid();
        return false;
    }

    // 부모 관계 설정 (배치 영역이 다른 오브젝트 위인 경우)
    if (area.IsPlacementAreaInProp(out var parent))
    {
        SetPropParent(parent);
    }
    else
    {
        ClearPropParent();
    }

    // 유효한 배치 상태 표시
    _propEditingState.Valid();
    return true;
}

public void Move(Vector3 position)
{
    transform.position = position;
}
```

색상 변경 가능 검증

```
public bool IsEditableColor(out IColorEditableProp prop)
{
    if (MyRoomPropData.CheckColorVariation()
        && MyRoomPropData.colorList.Count > 0
        && _materialChangeHelper != null)
    {
        prop = this;
    }
}
```

```

        prop = this;
        return true;
    }
    else
    {
        prop = null;
        return false;
    }
}

```

색상 변경

```

public void SetColor(Material mat, int colorIndex)
{
    var cloneMat = new Material(mat);
    _materialChangeHelper.ChangeMaterial(cloneMat);
    _propEditingState.Selected();
    CurrentColor = GetColorList()[colorIndex];
}

```

기즈모 위치 계산

```

public (Vector3, Quaternion) GetGizmoPositionAndRotation()
{
    var bounds = _collider.bounds;
    var fixedRotation = transform.rotation.eulerAngles.y - 90;
    var gizmoRotation = new Vector3(0, fixedRotation, 90);

    // 벽 방향에 따른 오프셋 방향 계산
    var offsetDirection = fixedRotation switch
    {
        RightDirectionKey => (Vector3.right * bounds.size.x) + (Vector3.right * 0.5f),
        BackDirectionKey => (Vector3.back * bounds.size.z) + (Vector3.back * 0.5f),
        ForwardDirectionKey => (Vector3.forward * bounds.size.z) + (Vector3.forward * 0.5f),
        LeftDirectionKey => (Vector3.left * bounds.size.x) + (Vector3.left * 0.5f),
        _ => Vector3.zero
    };

    Vector3 offset = bounds.center + offsetDirection;
    return (offset, Quaternion.Euler(gizmoRotation));
}

```

기능 설명

편집 기능

- 이동, 색상 변경 지원
- 하이라이트 상태 표시

편집 가능성 검증

- **색상 편집:** 색상 변경 인터페이스 지원 지원 + 오브젝트 데이터에 색상 리스트 존재
- **이동 가능:** 이동 인터페이스 지원

배치 프로세스

- 1 **위치 이동:** 히트 포인트로 즉시 이동
- 2 **타입 검증:** 배치 영역 타입과 PlacementType 일치 확인
- 3 **부모 설정:** 배치 영역이 다른 오브젝트 위면 부모 관계 설정
 - 오브젝트 슬롯 영역에 배치 시 부모 설정
 - 룸 레벨 배치 시 부모 해제
- 4 **시각 피드백:** 유효/무효에 따라 하이라이트 변경

하이라이트 상태

- **Focused:** 녹색 (Valid)
- **Unfocused:** 기본 (Default)
- **Selected:** 파란색 (Selected)
- **Deselected:** 기본 (Default)

기즈모 위치

- 벽 구조물의 회전값에 맞춰 적합한 위치 반환
- 기즈모는 수평 방향으로 표시

기즈모 위치 계산

```
public (Vector3,Quaternion) GetGizmoPositionAndRotation()
{
    var bounds = _collider.bounds;
    var fixedRotation = transform.rotation.eulerAngles.y - 90;
    var gizmoRotation = new Vector3(0, fixedRotation, 90);
    var offsetDirection = fixedRotation switch
    {
        RightDirectionKey => (Vector3.right * bounds.size.x) + (Vector3.right * 0.5f),
        BackDirectionKey => (Vector3.back * bounds.size.z) + (Vector3.back * 0.5f),
        ForwardDirectionKey => (Vector3.forward * bounds.size.z) + (Vector3.forward * 0.5f),
        LeftDirectionKey => (Vector3.left * bounds.size.x) + (Vector3.left * 0.5f),
        _ => Vector3.zero
    };
    Vector3 offset = bounds.center + offsetDirection;
    return (offset, Quaternion.Euler(gizmoRotation));
}
```

벽 배치 특화

- 배치 시 `area.GetPlacementRotation()`으로 벽면 방향 자동 설정

- 기즈모는 벽 방향에 따라 오브젝트 측면에 배치
- 회전은 불가능하여 벽면 고정 유지

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `SpawnablePropBase`를 상속받음.
- `IColorEditableProp`, `IMoveableProp` 인터페이스 구현.
- `PropEditingState`에 의존.

관련 클래스

- `PropEditingState`
- `SpawnablePropBase`
- `IColorEditableProp`
- `IMoveableProp`

SpawnablePhotoFrameProp

특수한 인터랙션을 포함한 오브젝트 클래스

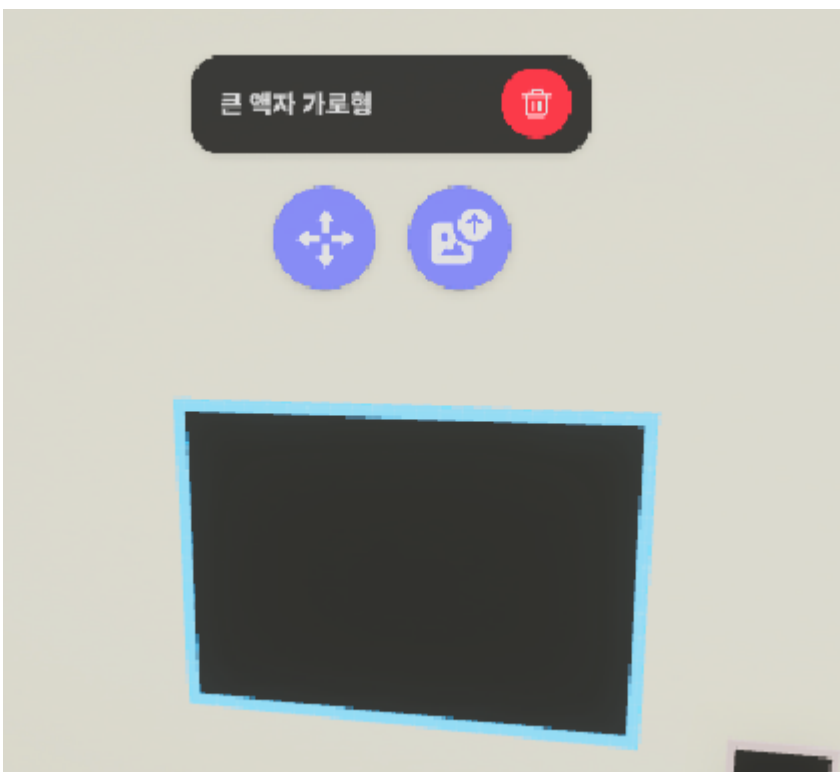
개요

`SpawnablePhotoFrameProp`는 MyRoomEditor에서 사진 프레임을 나타내는 스폰 가능한 오브젝트입니다. 이 클래스는 사용자가 사진을 업로드하고 표시할 수 있는 기능을 제공하며, `IMoveableProp`과 `IPhotoFrameProp` 인터페이스를 구현합니다.

역할

- 사진 프레임 오브젝트의 이동 및 편집 기능 제공
- 이미지 업로드 명령 및 콜백 받은 URL을 오브젝트 텍스처 적용 처리
- 오브젝트 하이라이트 및 선택 상태 관리
- 기즈모 위치 및 회전 계산
- 이미지 URL 데이터 관리
- 벽에 배치 할 수 있는 오브젝트

구현 인터페이스에 따른 편집 기능 표시



구현 인터페이스

- `IMoveableProp`: 이동 기능
- `IMyRoomEditorEditableObject`: 편집 가능한 오브젝트
- `IPhotoFrameProp`: 사진 프레임 특수 기능
- `IInteractableProp`: 상호작용 기능

선언

```
public class SpawnablePhotoFrame : SpawnablePropBase, IMoveableProp, IPhotoFrameProp
```

멤버

이벤트

- `OnCompleteAppliedImageCallback`: 이미지 적용 완료 시 호출되는 이벤트

속성

```
/// <summary>
/// Zenject를 통한 의존성 주입: 인터랙션 매니저
/// 이미지 업로드 처리를 담당하는 매니저
/// </summary>
[Inject]
private MyRoomEditorInteractionManager _interactionManager;

/// <summary>
/// 이미지 적용이 완료되었을 때 호출되는 이벤트
/// </summary>
private event Action<MyRoomPlaceProp> OnCompleteAppliedImageCallback;

/// <summary>
/// 이미지 렌더러 컴포넌트: 프레임에 이미지 표시 담당
/// </summary>
private PropImageRenderer _propImageRenderer;

/// <summary>
/// 텍스처 로더 컴포넌트: URL로부터 텍스처 로드 담당(프로젝트 유틸리티 클래스)
/// </summary>
private TextureLoaderFromURL _textureLoader;

/// <summary>
/// 현재 이미지 경로: 로드된 이미지의 URL
/// </summary>
private string _imgPath;

/// <summary>
```

```

/// <summary>
/// 편집 상태 컴포넌트: 하이라이트 상태 관리
/// </summary>
private PropEditingState _propEditingState;

```

메서드

```

/// <summary>
/// 오브젝트 삭제 메서드.
/// 이벤트 구독 해제 후 게임 오브젝트를 파괴.
/// </summary>
public void Delete()

/// <summary>
/// 하이라이트 상태를 설정하는 추상 메서드 구현.
/// PropEditingState의 초기 설정을 수행.
/// </summary>
protected override void SetupPropHighlight()

/// <summary>
/// 포커스 상태로 변경하는 메서드.
/// 유효한 상태의 하이라이트를 표시.
/// </summary>
public void Focused()

/// <summary>
/// 포커스 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Unfocused()

/// <summary>
/// 선택 상태로 변경하는 메서드.
/// 선택된 상태의 하이라이트를 표시.
/// </summary>
public void Selected()

/// <summary>
/// 선택 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Deselected()

/// <summary>
/// 배치 가능 영역인지 검증하고 배치하는 메서드.
/// 히트 포인트로 이동 후 배치 타입 검증 및 부모 관계 설정.
/// </summary>
/// <param name="hitPosition">배치할 히트 포인트 위치</param>
/// <param name="area">배치 영역 인터페이스</param>
/// <returns>배치 가능한 경우 true</returns>
public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)

/// <summary>

```

```

/// 오브젝트를 지정된 위치로 이동하는 메서드.
/// </summary>
/// <param name="position">이동할 목표 위치</param>
public void Move(Vector3 position)

/// <summary>
/// 기즈모의 위치와 회전을 계산하여 반환하는 메서드.
/// 배치 타입에 따라 오브젝트 위쪽(Floor) 또는 아래쪽(Ceiling)에 기즈모 배치.
/// </summary>
/// <returns>기즈모 위치와 회전 튜플</returns>
public (Vector3, Quaternion) GetGizmoPositionAndRotation()

/// <summary>
/// 이미지 업로드 완료 이벤트 핸들러.
/// 업로드된 이미지 경로를 받아 텍스처 로드 시작.
/// </summary>
/// <param name="imgPath">업로드된 이미지의 경로/URL</param>
public void OnCompleteUpload(string imgPath)

/// <summary>
/// 로드된 텍스처를 프레임에 적용하는 프라이빗 메서드.
/// 이미지 렌더러에 이미지를 추가하고 배치 데이터에 URL 저장.
/// </summary>
/// <param name="img">적용할 텍스처</param>
private void ApplyImage(Texture2D img)

/// <summary>
/// 이미지 적용 완료 콜백을 예약하는 메서드.
/// 이미지 적용이 완료되었을 때 호출할 콜백을 설정.
/// </summary>
/// <param name="callback">이미지 적용 완료 시 호출할 콜백</param>
public void ReserveCompletedApplyImageCallback(Action<MyRoomPlaceProp> callback)

/// <summary>
/// IInteractableProp 인터페이스의 Interact 메서드 구현.
/// 룸 진입 시 저장된 이미지 URL로부터 이미지를 로드.
/// </summary>
public void Interact()

```

코드 스니펫

이미지 업로드 완료 처리

```

public void OnCompleteUpload(string imgPath)
{
    _imgPath = imgPath;
    _textureLoader.GetTexture(imgPath, ApplyImage);
}

```

이미지 적용

```
private void ApplyImage(Texture2D img)
{
    _propImageRenderer.AddImage(img);
    MyRoomPlacePropData.imageUrl = _imgPath;
    OnCompleteAppliedImageCallback?.Invoke(GetPlacePropInfo());
}
```

콜백 예약

```
public void ReserveCompletedApplyImageCallback(Action<MyRoomPlaceProp> callback)
{
    OnCompleteAppliedImageCallback = callback;
}
```

초기화 및 로드

```
public void Interact()
{
    if (string.IsNullOrEmpty(MyRoomPlacePropData.imageUrl)) return;
    Debug.Log($"[SpawnablePhotoFrame] {GetPlacePropId()} Load Image => {MyRoomPlacePropData.imageUrl}");
    _imgPath = MyRoomPlacePropData.imageUrl;
    _textureLoader.GetTexture(_imgPath, ApplyImage);
}
```

기능 설명

이미지 관리

- 1 업로드 요청: **MyRoomEditorInteractionManager**를 통해 이미지 업로드
- 2 완료 콜백: **OnCompleteUpload(string imgPath)**에서 이미지 경로 수신
- 3 텍스처 로드: **TextureLoaderFromURL** (외부 유틸리티 클래스)로 URL로부터 텍스처 로드
- 4 이미지 적용: **ApplyImage(Texture2D img)**에서 프레임에 이미지 표시
- 5 데이터 저장: 배치 데이터에 이미지 URL 저장

편집 기능

- 이동, 이미지 텍스처 변경 지원
- 하이라이트 상태 표시

편집 가능성 검증

- 이미지 변경 가능: 이미지 변경 인터페이스 지원
- 이동 가능: 이동 인터페이스 지원

배치 프로세스

- 1 **위치 이동:** 히트 포인트로 즉시 이동
- 2 **타입 검증:** 배치 영역 타입과 PlacementType 일치 확인
- 3 **부모 설정:** 배치 영역이 다른 오브젝트 위면 부모 관계 설정
 - 오브젝트 슬롯 영역에 배치 시 부모 설정
 - 룸 레벨 배치 시 부모 해제
- 4 **시각 피드백:** 유효/무효에 따라 하이라이트 변경

하이라이트 상태

- **Focused:** 녹색 (Valid)
- **Unfocused:** 기본 (Default)
- **Selected:** 파란색 (Selected)
- **Deselected:** 기본 (Default)

기즈모 위치

- 벽 구조물의 회전값에 맞춰 적합한 위치 반환
- 기즈모는 수평 방향으로 표시

기즈모 위치 계산

```
public (Vector3, Quaternion) GetGizmoPositionAndRotation()
{
    var bounds = _collider.bounds;
    var fixedRotation = transform.rotation.eulerAngles.y - 90;
    var gizmoRotation = new Vector3(0, fixedRotation, 90);
    var offsetDirection = fixedRotation switch
    {
        RightDirectionKey => (Vector3.right * bounds.size.x) + (Vector3.right * 0.5f),
        BackDirectionKey => (Vector3.back * bounds.size.z) + (Vector3.back * 0.5f),
        ForwardDirectionKey => (Vector3.forward * bounds.size.z) + (Vector3.forward * 0.5f),
        LeftDirectionKey => (Vector3.left * bounds.size.x) + (Vector3.left * 0.5f),
        _ => Vector3.zero
    };
    Vector3 offset = bounds.center + offsetDirection;
    return (offset, Quaternion.Euler(gizmoRotation));
}
```

인터랙션 초기화

- `Interact()` 메서드로 룸 진입 시 자동 이미지 로드
- 저장된 URL로부터 텍스처를 로드하여 프레임에 적용

벽 배치 특화

- 배치 시 `area.GetPlacementRotation()`으로 벽면 방향 자동 설정
- 기즈모는 벽 방향에 따라 오브젝트 측면에 배치

- 회전은 불가능하여 벽면 고정 유지

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `SpawnablePropBase`를 상속받음.
- `IMoveableProp`, `IPhotoFrameProp` 인터페이스를 구현.
- `MyRoomEditorInteractionManager`에 의존.
- `PropImageRenderer`, `TextureLoaderFromURL`, `PropEditingState` 컴포넌트를 사용.

관련 클래스

- `MyRoomEditorInteractionManager`
- `PropEditingState`
- `SpawnablePropBase`
- `IMoveableProp`
- `IInteractableProp`

SpawnableScreenProp

특수한 인터랙션을 포함한 오브젝트 클래스

개요

`SpawnableScreenProp`는 MyRoomEditor에서 스크린을 나타내는 스폰 가능한 오브젝트입니다. 이 클래스는 스크린 핸들러를 제공하며, `IMoveableProp`와 `IScreenProp` 인터페이스를 구현합니다.

역할

- 스크린 오브젝트(`NetworkedScreen`)의 이동 기능 제공
- 오브젝트 하이라이트 및 선택 상태 관리
- 기즈모 위치 및 회전 계산
- 벽에 배치 할 수 있는 오브젝트

구현 인터페이스

- `IMoveableProp`: 이동 기능
- `IMyRoomEditorEditableObject`: 편집 가능한 오브젝트
- `IInteractableProp`: 상호작용 기능
- `IScreenProp`: 스크린 특수 기능

선언

```
public class SpawnableScreenProp : SpawnablePropBase, IMoveableProp, IScreenProp
```

멤버

속성

```
/// <summary>  
/// 스크린 표면 트랜스폼: 파일 공유 UI 표시 위치  
/// </summary>  
[SerializeField]  
private Transform screenPropTr;
```

```

public Transform ScreenHandlerTr ⇒ screenPropTr;

/// <summary>
/// 편집 상태 컴포넌트: 하이라이트 상태 관리
/// </summary>
private PropEditingState _propEditingState;

```

메서드

```

/// <summary>
/// 오브젝트 삭제 메서드.
/// 이벤트 구독 해제 후 게임 오브젝트를 파괴.
/// </summary>
public void Delete()

/// <summary>
/// 하이라이트 상태를 설정하는 추상 메서드 구현.
/// PropEditingState의 초기 설정을 수행.
/// </summary>
protected override void SetupPropHighlight()

/// <summary>
/// 포커스 상태로 변경하는 메서드.
/// 유효한 상태의 하이라이트를 표시.
/// </summary>
public void Focused()

/// <summary>
/// 포커스 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Unfocused()

/// <summary>
/// 선택 상태로 변경하는 메서드.
/// 선택된 상태의 하이라이트를 표시.
/// </summary>
public void Selected()

/// <summary>
/// 선택 해제 상태로 변경하는 메서드.
/// 기본 하이라이트 상태로 복원.
/// </summary>
public void Deselected()

/// <summary>
/// 배치 가능 영역인지 검증하고 배치하는 메서드.
/// 히트 포인트로 이동 후 배치 타입 검증 및 부모 관계 설정.
/// </summary>
/// <param name="hitPosition">배치할 히트 포인트 위치</param>
/// <param name="area">배치 영역 인터페이스</param>
/// <returns>배치 가능한 경우 true</returns>
public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)

```

```

public bool IsAcceptableArea(Vector3 hitPosition, float acceptableArea)

/// <summary>
/// 오브젝트를 지정된 위치로 이동하는 메서드.
/// </summary>
/// <param name="position">이동할 목표 위치</param>
public void Move(Vector3 position)

/// <summary>
/// 기즈모의 위치와 회전을 계산하여 반환하는 메서드.
/// 배치 타입에 따라 오브젝트 위쪽(Floor) 또는 아래쪽(Ceiling)에 기즈모 배치.
/// </summary>
/// <returns>기즈모 위치와 회전 튜플</returns>
public (Vector3, Quaternion) GetGizmoPositionAndRotation()

```

기능 설명

편집 기능

- 이동 변경 지원
- 하이라이트 상태 표시

편집 가능성 검증

- 이동 가능: 이동 인터페이스 지원

배치 프로세스

- 1 위치 이동: 히트 포인트로 즉시 이동
- 2 타입 검증: 배치 영역 타입과 PlacementType 일치 확인
- 3 부모 설정: 배치 영역이 다른 오브젝트 위면 부모 관계 설정
 - 오브젝트 슬롯 영역에 배치 시 부모 설정
 - 룸 레벨 배치 시 부모 해제
- 4 시각 피드백: 유효/무효에 따라 하이라이트 변경

하이라이트 상태

- **Focused:** 녹색 (Valid)
- **Unfocused:** 기본 (Default)
- **Selected:** 파란색 (Selected)
- **Deselected:** 기본 (Default)

기즈모 위치

- 벽 구조물의 회전값에 맞춰 적합한 위치 반환
- 기즈모는 수평 방향으로 표시

기즈모 위치 계산

```

public (Vector3,Quaternion) GetGizmoPositionAndRotation()
{
    var bounds = _collider.bounds;
    var fixedRotation = transform.rotation.eulerAngles.y - 90;
    var gizmoRotation = new Vector3(0, fixedRotation, 90);
    var offsetDirection = fixedRotation switch
    {
        RightDirectionKey => (Vector3.right * bounds.size.x) + (Vector3.right * 0.5f),
        BackDirectionKey => (Vector3.back * bounds.size.z) + (Vector3.back * 0.5f),
        ForwardDirectionKey => (Vector3.forward * bounds.size.z) + (Vector3.forward * 0.5f),
        LeftDirectionKey => (Vector3.left * bounds.size.x) + (Vector3.left * 0.5f),
        _ => Vector3.zero
    };
    Vector3 offset = bounds.center + offsetDirection;
    return (offset, Quaternion.Euler(gizmoRotation));
}

```

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `SpawnablePropBase`를 상속받음.
- `IMoveableProp`, `IScreenProp` 인터페이스를 구현.
- `PropEditingState` 컴포넌트를 사용.

관련 클래스

- `NetworkedScreen`: 실시간 파일 공유 모듈
- `PropEditingState`: 하이라이트 상태 관리
- `SpawnablePropBase`: 부모 클래스
- `IMoveableProp`: 구현된 인터페이스

PropEditingState

오브젝트 편집 상태에 따라 오브젝트의 Material 효과를 표현하는 클래스

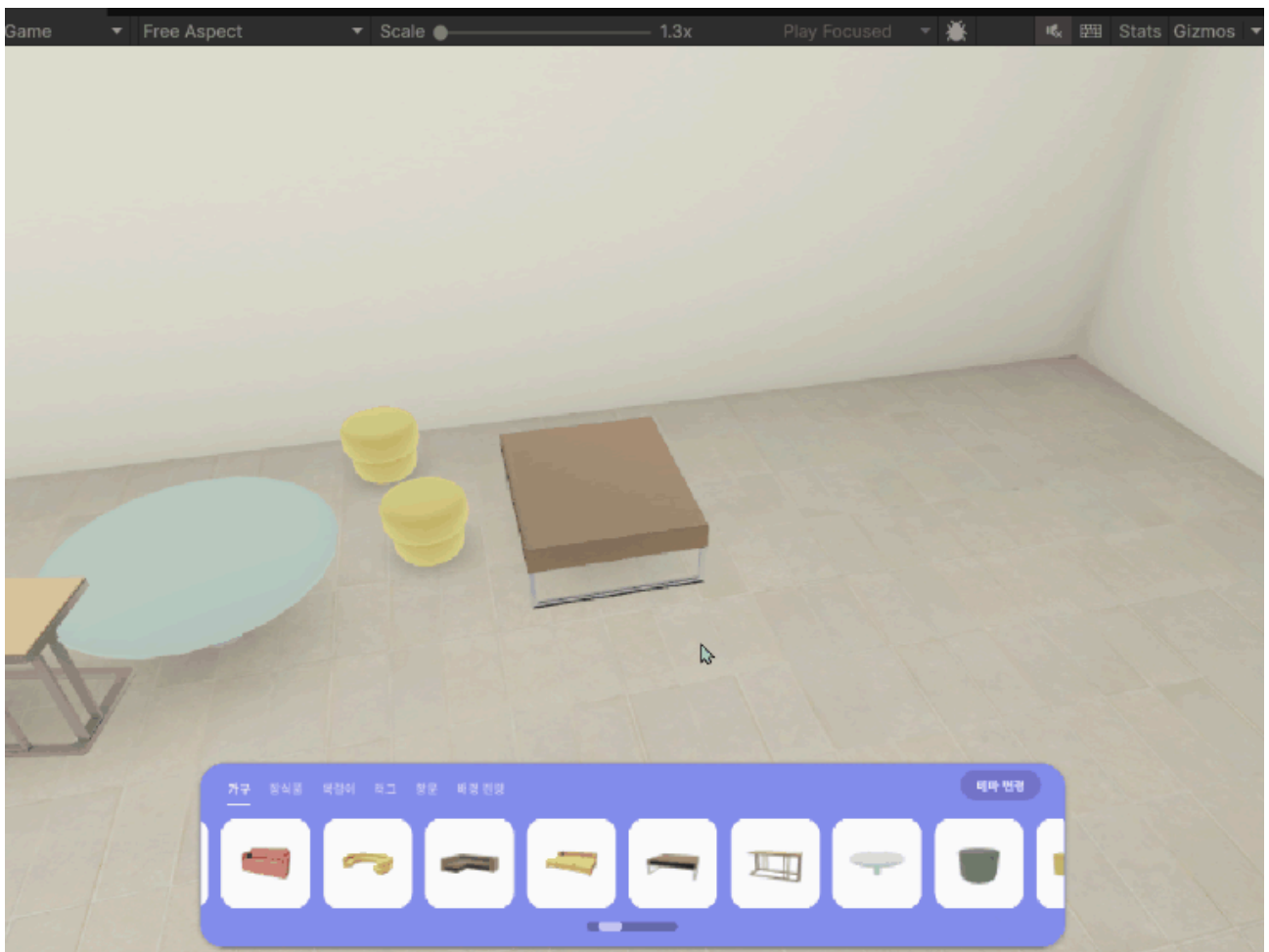
개요

PropEditingState는 MyRoomEditor에서 오브젝트의 시각적 하이라이트 상태를 관리하는 클래스입니다. 머티리얼의 셰이더 키워드를 통해 선택, 유효성 등의 시각적 피드백을 제공합니다.

역할

- 오브젝트의 하이라이트 상태 관리
- 셰이더 키워드를 통한 시각적 피드백 제공
- 편집 중 상태 표시 (선택됨, 유효함, 무효함 등)

편집 상태 표시



선언

```
public class PropEditingState : MonoBehaviour
```

멤버

열거형

```
/// <summary>
/// 하이라이트 상태를 정의하는 열거형
/// None: 기본 상태, Selected: 선택됨, Valid: 유효함, Invalid: 유효하지 않음
/// </summary>
public enum HighlightState
{
    None,          // 기본 상태 (하이라이트 없음)
    Selected,      // 선택된 상태 (파란색)
    Valid,         // 유효한 배치 위치 (녹색)
    Invalid        // 유효하지 않은 배치 위치 (빨간색)
}
```

속성

```
/// <summary>
/// 하이라이트 효과를 적용할 대상 Material 리스트
/// Setup() 메서드에서 _HIGHLIGHTSTATE 속성을 가진 Material들만 수집
/// </summary>
private readonly List<Material> targetMaterials = new List<Material>();

/// <summary>
/// 하이라이트 상태를 확인하는 Material 속성 이름
/// 이 속성이 있는 Material만 하이라이트 효과 적용 대상으로 간주
/// </summary>
private const string StatePropertyName = "_HIGHLIGHTSTATE";

/// <summary>
/// 하이라이트 상태와 Shader 키워드를 매핑하는 사전
/// 각 상태에 해당하는 Shader 키워드를 정의
/// </summary>
private static readonly Dictionary<HighlightState, string> StateKeywords = new()
{
    { HighlightState.None, "_HIGHLIGHTSTATE_NONE" },
    { HighlightState.Selected, "_HIGHLIGHTSTATE_BLUE" },
    { HighlightState.Valid, "_HIGHLIGHTSTATE_GREEN" },
    { HighlightState.Invalid, "_HIGHLIGHTSTATE_RED" },
};
```

메서드

```
/// <summary>
/// 하이라이트 시스템을 초기화하는 메서드.
/// 오브젝트의 모든 자식 Renderer에서 하이라이트 가능한 Material들을 찾아 수집.
/// </summary>
public void Setup()

    /// <summary>
    /// 하이라이트 상태를 기본 상태(None)로 변경하는 메서드.
    /// 하이라이트 효과를 해제.
    /// </summary>
public void Default()

    /// <summary>
    /// 하이라이트 상태를 선택됨(Selected)으로 변경하는 메서드.
    /// 파란색 하이라이트 효과 적용.
    /// </summary>
public void Selected()

    /// <summary>
    /// 하이라이트 상태를 유효함(Valid)으로 변경하는 메서드.
    /// 녹색 하이라이트 효과 적용.
    /// </summary>
public void Valid()

    /// <summary>
    /// 하이라이트 상태를 유효하지 않음(Invalid)으로 변경하는 메서드.
    /// 빨간색 하이라이트 효과 적용.
    /// </summary>
public void Invalid()

    /// <summary>
    /// 하이라이트 상태를 설정하는 프라이빗 메서드.
    /// 지정된 상태에 해당하는 Shader 키워드를 활성화.
    /// </summary>
    /// <param name="newState">설정할 새로운 하이라이트 상태</param>
private void SetState(HighlightState newState)

    /// <summary>
    /// Shader 키워드를 변경하는 프라이빗 메서드.
    /// 기존 하이라이트 키워드들을 모두 비활성화하고 새로운 키워드만 활성화.
    /// </summary>
    /// <param name="keyword">활성화할 Shader 키워드</param>
private void ChangeKeyword(string keyword)
```

코드 스니펫

Material 초기화

```

public void Setup()
{
    // 자식 오브젝트들의 모든 Renderer 컴포넌트 가져오기
    var renderers = GetComponentsInChildren<Renderer>();

    foreach (var renderer in renderers)
    {
        // 각 Renderer의 모든 Material 순회
        foreach (var material in renderer.materials)
        {
            // Material이 존재하고 하이라이트 속성을 가지고 있는 경우만 수집
            if (material != null && material.HasProperty(StatePropertyName))
            {
                targetMaterials.Add(material);
            }
        }
    }
}

```

하이라이트 변경 호출 프로세스

```

public void Selected()
{
    SetState(HighlightState.Selected);
}

private void SetState(HighlightState newState)
{
    // 상태에 해당하는 키워드를 찾아 활성화
    ChangeKeyword(StateKeywords[newState]);
}

```

키워드 변경 로직

```

private void ChangeKeyword(string keyword)
{
    // 수집된 모든 대상 Material에 대해 키워드 변경 수행
    foreach (var material in targetMaterials)
    {
        // 현재 활성화된 모든 키워드 중 하이라이트 관련 키워드만 비활성화
        foreach (var enabledKeyword in material.enabledKeywords)
        {
            // "_HIGHLIGHTSTATE"로 시작하는 키워드만 대상으로 함
            if (enabledKeyword.ToString().StartsWith("_HIGHLIGHTSTATE"))
            {
                material.DisableKeyword(enabledKeyword);
            }
        }
    }

    // 새로운 키워드 활성화

```



```
        material.EnableKeyword(keyword);
    }
}
```

기능 설명

하이라이트 상태 시스템

- **None:** 기본 상태, 하이라이트 없음 (`_HIGHLIGHTSTATE_NONE`)
- **Selected:** 선택된 오브젝트를 파란색으로 표시 (`_HIGHLIGHTSTATE_BLUE`)
- **Valid:** 유효한 배치 위치를 녹색으로 표시 (`_HIGHLIGHTSTATE_GREEN`)
- **Invalid:** 유효하지 않은 배치 위치를 빨간색으로 표시 (`_HIGHLIGHTSTATE_RED`)

머티리얼 관리

- `Setup()` 메서드에서 자식 Renderer들의 Material 중 `_HIGHLIGHTSTATE` 속성을 가진 Material들을 수집
- 하이라이트 효과를 지원하는 Material만 대상으로 함

키워드 기반 렌더링

- 기존에 활성화된 `_HIGHLIGHTSTATE`로 시작하는 키워드들을 모두 비활성화
- 새로운 상태에 해당하는 키워드만 활성화
- Material의 `EnableKeyword()`와 `DisableKeyword()`를 사용하여 동적 변경

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `SpawnablePropBase` 등에서 동적으로 컴포넌트 추가.
- 셰이더에서 `_HIGHLIGHTSTATE_*` 키워드 사용.
- `IMyRoomEditorEditableObject.Focused()`, `Selected()` 등에서 호출.

Shader 요구사항

- 하이라이트 효과를 사용하려면 Material의 Shader가 다음 키워드를 지원해야함
 - `_HIGHLIGHTSTATE_NONE`
 - `_HIGHLIGHTSTATE_BLUE`
 - `_HIGHLIGHTSTATE_GREEN`
 - `_HIGHLIGHTSTATE_RED`

사용 예시

하이라이트 기능 셋업

```

// 배치 오브젝트 클래스(SpawnablePropBase의서브 클래스)에서 Setup() 호출
protected override void SetupPropHighlight()
{
    _propEditingState.Setup();
}

// PropEditingState에서 Material및 렌더러 검색하여 캐싱
public void Setup()
{
    var renderers = GetComponentInChildren<Renderer>();
    foreach (var renderer in renderers)
    {
        foreach (var material in renderer.materials)
        {
            if (material != null && material.HasProperty(StatePropertyName))
            {
                targetMaterials.Add(material);
            }
        }
    }
}

```

하이라이트 표시

```

//오브젝트 이동시 배치 가능한 위치일때 Valid 하이라이트 처리
public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)
{
    Move(hitPosition);
    transform.rotation = area.GetPlacementRotation();

    if (area.GetPlacementType() != PlacementType)
    {
        _propEditingState.Invalid();
        return false;
    }
    if (area.IsPlacementAreaInProp(out var parent))
    {
        SetPropParent(parent);
    }
    else
    {
        ClearPropParent();
    }

    _propEditingState.Valid();
    return true;
}

// 오브젝트 선택/포커싱 할때 Focused/Unfocused 하이라이트 처리
if (InputUtils.RaycastHit(out RaycastHit hits, _targetLayer, _firstPriority)
{
    if (hits.transform.TryGetComponent(out IMyRoomEditorEditableObject focusedObject)
    {
        _propEditingState.Focused(focusedObject);
    }
}

```

```

        && InputUtils.IsPointerOnUI() == false)
    {
        if (focusedObject.Equals(_focusedObject)) continue;
        if (IsSelectedObject(focusedObject))
        {
            _focusedObject?.Unfocused();
            _focusedObject = focusedObject;
        }
        else
        {
            if (IsSelectedObject(_focusedObject) == false)
            {
                _focusedObject?.Unfocused();
            }
            _focusedObject = focusedObject;
            _focusedObject.Focused();
        }
    }
    else
    {
        //Unfocus조건
        //1. 선택된 오브젝트는 Unfocus되지 않는다.
        //2. 선택된 오브젝트가 아니며 포커싱 중인 오브젝트가 있는 경우는 Unfocus 한다.
        if (IsSelectedObject(_focusedObject))
        {
            _focusedObject = null;
        }
        else
        {
            _focusedObject?.Unfocused();
            _focusedObject = null;
        }
    }
}
}

```

관련 클래스

- [SpawnableFloorAndCeilProp](#)
- [SpawnableWallProp](#)
- [SpawnablePhotoFrameProp](#)
- [SpawnableScreenProp](#)

IMyRoomEditorEditableObject

편집 가능한 오브젝트 정의 및 기능 정의 인터페이스

개요

`IMyRoomEditorEditableObject` 인터페이스는 MyRoomEditor에서 편집 가능한 객체의 기본 동작을 정의합니다.

역할

- MyRoomEditor에서 편집 가능한 객체의 표준 인터페이스 제공
- 객체의 선택/해제, 포커스 상태 관리
- 다양한 편집 기능(색상, 이동, 회전 등)의 지원 여부 검사

선언

```
public interface IMyRoomEditorEditableObject : IEquatable<IMyRoomEditorEditableObject>
```

멤버

속성

```
/// <summary>  
/// 로컬라이징용 객체의 이름 키.  
/// </summary>  
string PropNameKey { get; }
```

메서드

```
/// <summary>  
/// 객체를 삭제합니다.  
/// </summary>  
void Delete()  
  
/// <summary>
```

```

/// 색상 편집이 가능한지 검사하고 인터페이스를 반환합니다.
/// </summary>
/// <param name="prop">색상 편집 인터페이스 출력</param>
/// <returns>색상 편집 가능한 경우 true</returns>
bool IsEditableColor(out IColorEditableProp prop)

/// <summary>
/// 이동이 가능한지 검사하고 인터페이스를 반환합니다.
/// </summary>
/// <param name="prop">이동 인터페이스 출력</param>
/// <returns>이동 가능한 경우 true</returns>
bool IsMovableProp(out IMoveableProp prop)

/// <summary>
/// 회전이 가능한지 검사하고 인터페이스를 반환합니다.
/// </summary>
/// <param name="prop">회전 인터페이스 출력</param>
/// <returns>회전 가능한 경우 true</returns>
bool IsRotatableProp(out IRotatableProp prop)

/// <summary>
/// 포토프레임 프로퍼티인지 검사하고 인터페이스를 반환합니다.
/// </summary>
/// <param name="prop">포토프레임 인터페이스 출력</param>
/// <returns>포토프레임인 경우 true</returns>
bool IsPhotoFrameProp(out IPhotoFrameProp prop)

/// <summary>
/// 배치 영역인지 검사하고 인터페이스를 반환합니다.
/// </summary>
/// <param name="area">배치 영역 인터페이스 출력</param>
/// <returns>배치 영역인 경우 true</returns>
bool IsPlacementArea(out IPlaceableArea area)

/// <summary>
/// 객체가 포커스되었을 때 호출됩니다.
/// </summary>
void Focused()

/// <summary>
/// 객체의 포커스가 해제되었을 때 호출됩니다.
/// </summary>
void Unfocused()

/// <summary>
/// 객체가 선택되었을 때 호출됩니다.
/// </summary>
void Selected()

/// <summary>
/// 객체의 선택이 해제되었을 때 호출됩니다.
/// </summary>
void Deselected()

/// <summary>

```

```

/// 기즈모의 위치와 회전을 반환합니다.
/// </summary>
/// <returns>위치와 회전의 튜플</returns>
(Vector3, Quaternion) GetGizmoPositionAndRotation()

/// <summary>
/// 배치된 오브젝트 정보를 반환합니다.
/// </summary>
/// <returns>MyRoomPlaceProp 정보</returns>
MyRoomPlaceProp GetPlacePropInfo()

/// <summary>
/// 배치된 오브젝트 ID를 반환합니다.
/// </summary>
/// <returns>오브젝트 ID 문자열</returns>
string GetPlacePropId()

```

기능 설명

오브젝트 관리

- `Delete()`로 오브젝트 삭제
- `PropNameKey`로 로컬라이징된 이름 제공

편집 기능 검사

- 각 편집 기능(색상, 이동, 회전 등)에 대한 지원 여부 검사
- 지원되는 경우 해당 인터페이스 반환
- `IsEditableColor(out IColorEditableProp prop)`: 색상 편집 가능 여부 및 인터페이스 반환
- `IsMovableProp(out IMoveableProp prop)`: 이동 가능 여부 및 인터페이스 반환
- `IsRotatableProp(out IRotatableProp prop)`: 회전 가능 여부 및 인터페이스 반환
- `IsPhotoFrameProp(out IPhotoFrameProp prop)`: 사진 프레임 여부 및 인터페이스 반환
- `IsPlacementArea(out IPlaceableArea area)`: 배치 영역 여부 및 인터페이스 반환

상태 관리

- `Focused()/Unfocused()`: 포커스 상태 전환
- `Selected()/Deselected()`: 선택 상태 전환

기즈모 및 배치 정보

- `GetGizmoPositionAndRotation()`: 편집 기즈모 위치/회전 반환
- `GetPlacePropInfo()/GetPlacePropId()`: 배치 정보 제공

의존성/상속 관계

- `IEquatable<IMyRoomEditorEditableObject>`를 구현.

- 확장 인터페이스들:

- [IColorEditableProp](#): 색상 변경 기능
- [IMoveableProp](#): 이동 기능
- [IRotatableProp](#): 회전 기능
- [IPhotoFrameProp](#): 사진 프레임 기능
- [IPlaceableArea](#): 배치 영역 기능

사용 예시

인터페이스 검증 및 사용

```
if (editableObject.IsMovableProp(out IMoveableProp movableProp))
{
    // 이동 가능한 오브젝트 처리
    Vector3 position = movableProp.GetPosition();
    // ...
}

if (editableObject.IsEditableColor(out IColorEditableProp colorProp))
{
    // 색상 편집 가능한 오브젝트 처리
    var colors = colorProp.GetColorList();
    // ...
}
```

상태 변경

```
// 오브젝트 선택
editableObject.Focused();
editableObject.Selected();

// 오브젝트 선택 해제
editableObject.Unfocused();
editableObject.Deselected();
```

관련 클래스 및 인터페이스

- [SpawnablePropBase](#)
- [SpawnableFloorAndCeilProp](#)
- [SpawnableWallProp](#)
- [SpawnablePhotoFrameProp](#)
- [SpawnableScreenProp](#)
- [IColorEditableProp](#)
- [IMoveableProp](#)

- IRotatableProp
- IPhotoFrameProp
- IPlaceableArea
- SpawnablePropBase
- MyRoomEditorInteractionManager
- MyRoomEditorPlacementManager
- MyRoomEditorPropEditingManager
- MyRoomEditorPropSelector

IColorEditableProp

색상 편집 가능한 오브젝트 정의

개요

IColorEditableProp 인터페이스는 MyRoom 에디터에서 색상 편집이 가능한 프로퍼티를 정의합니다. 이 인터페이스를 구현하는 클래스는 색상 목록을 제공하고 색상을 설정할 수 있습니다.

역할

- MyRoomEditor에서 색상 편집 기능을 제공하는 인터페이스 정의
- 색상 목록 조회 및 색상 적용 기능 표준화

선언

```
public interface IColorEditableProp : IMyRoomEditorEditableObject
```

멤버

메서드

```
/// <summary>  
/// 사용할 수 있는 색상 목록을 반환합니다.  
/// </summary>  
List<MyRoomPropColor> GetColorList()  
  
/// <summary>  
/// 지정된 색상 인덱스로 머티리얼의 색상을 설정합니다.  
/// </summary>  
/// <param name="mat">색상을 적용할 머티리얼</param>  
/// <param name="colorIndex">색상 목록의 인덱스</param>  
void SetColor(Material mat, int colorIndex)
```

기능 설명

색상 목록 제공

- `GetColorList()` 메서드로 편집 가능한 색상 목록을 반환
- `MyRoomPropColor` 객체 리스트 형태로 색상 정보 제공

색상 적용

- `SetColor()` 메서드로 지정된 머티리얼에 색상 적용
- 색상 인덱스를 통해 목록에서 특정 색상을 선택하여 적용

의존성/상속 관계

- `IMyRoomEditorEditableObject`를 상속받음.
- 색상 편집이 가능한 오브젝트들에서 구현(`SpawnableFloorAndCeilProp`, `SpawnableWallProp`)

사용 예시

색상 편집 가능 여부 확인

```
if (prop.TryGetComponent(out IMyRoomEditorEditableObject editableObject))
{
    if (editableObject.IsEditableColor(out IColorEditableProp colorProp))
    {
        var colors = colorProp.GetColorList();
        // 색상 목록을 UI에 표시
    }
}
```

색상 적용

```
colorProp.SetColor(renderer.material, selectedColorIndex);
```

관련 클래스

- `SpawnableWallProp`
- `SpawnableFloorAndCeilProp`
- `IMyRoomEditorEditableObject`
- `MyRoomEditorPropEditingManager`

IMoveableProp

이동 가능한 오브젝트 정의

개요

IMoveableProp 인터페이스는 MyRoom 에디터에서 이동 가능한 프로퍼티를 정의합니다. 이 인터페이스를 구현하는 클래스는 위치 이동 및 배치 가능 영역 검사를 수행할 수 있습니다.

역할

- MyRoomEditor에서 오브젝트의 이동 기능 제공
- 배치 가능 영역 내에서의 위치 검증

선언

```
public interface IMoveableProp : IMyRoomEditorEditableObject
```

멤버

속성

```
/// <summary>  
/// 이동 가능한 프로퍼티의 기본 컴포넌트 참조.  
/// </summary>  
SpawnablePropBase PropBaseComponent { get; }
```

메서드

```
/// <summary>  
/// 프로퍼티의 현재 위치를 반환합니다.  
/// </summary>  
/// <returns>현재 위치 Vector3</returns>  
Vector3 GetPosition()  
  
/// <summary>  
/// 프로퍼티를 지정된 위치로 이동합니다.
```

```

/// </summary>
/// <param name="pos">이동할 목표 위치</param>
void Move(Vector3 pos)

/// <summary>
/// 지정된 위치가 배치 가능 영역 내에 있는지 검사합니다.
/// </summary>
/// <param name="targetPos">검사할 목표 위치</param>
/// <param name="placeableArea">배치 가능 영역 인터페이스</param>
/// <returns>배치 가능한 경우 true</returns>
bool IsPlaceableArea(Vector3 targetPos, IPlaceableArea placeableArea)

```

기능 설명

위치 관리

- `GetPosition()`으로 현재 위치 조회
- `Move()`로 새로운 위치로 이동

배치 영역 검사

- `IsPlaceableArea()`로 목표 위치의 유효성 검증
- 배치 가능 영역 인터페이스와 연동하여 안전한 이동 보장

프로퍼티 참조

- `SpawnablePropBase` 속성으로 기본 프로퍼티 컴포넌트 접근

의존성/상속 관계

- `IMyRoomEditorEditableObject`를 상속받음.
- `SpawnablePropBase`, `IPlaceableArea` 클래스에 의존.

상속 관계 및 구현체

- `IMyRoomEditorEditableObject`를 확장
- 이동이 가능한 오브젝트들에서 구현:
 - `SpawnableFloorAndCeilProp`
 - `SpawnableWallProp`
 - `SpawnablePhotoFrameProp`
 - `SpawnableScreenProp`

사용 예시

이동 가능 여부 확인

```

if (editableObject.IsMovableProp(out IMoveableProp movableProp))
{
    Vector3 currentPos = movableProp.GetPosition();
    // 이동 UI 표시
}

```

위치 이동

```

movableProp.Move(targetPosition);

```

배치 영역 검사

```

if (movableProp.IsPlaceableArea(newPosition, placeableArea))
{
    movableProp.Move(newPosition);
}

```

관련 클래스

- [SpawnableFloorAndCeilProp](#)
- [SpawnableWallProp](#)
- [SpawnablePropBase](#)
- [IPlaceableArea](#)
- [MyRoomEditorPlacementManager](#)
- [MyRoomEditorPropEditingManager](#)
- [IMyRoomEditorEditableObject](#)

IRotatableProp

회전 가능한 오브젝트 정의

개요

IRotatableProp 인터페이스는 MyRoomEditor에서 회전 가능한 오브젝트의 회전 기능을 정의합니다. 이 인터페이스를 구현하는 클래스는 자유 회전과 스냅 회전을 지원합니다.

역할

- MyRoomEditor에서 오브젝트의 회전 상태 관리
- 자유 회전 및 스냅 회전 기능 제공
- 회전 값의 설정과 조회 지원

선언

```
public interface IRotatableProp : IMyRoomEditorEditableObject
```

멤버

메서드

```
/// <summary>  
/// 현재 회전 값을 반환합니다.  
/// </summary>  
/// <returns>현재 회전 Quaternion</returns>  
Quaternion GetRotation()  
  
/// <summary>  
/// 회전 값을 설정합니다.  
/// </summary>  
/// <param name="rotation">설정할 회전 Quaternion</param>  
void SetRotation(Quaternion rotation)  
  
/// <summary>  
/// 스냅 값에 따라 회전을 적용합니다.  
/// </summary>  
/// <param name="snapStepValue">스냅 단계 값 (도 단위)</param>
```

```
/// <param name= "snapStepValue" >스텝 간격 값 (도 단위)</param>  
void RotationBySnapValue(float snapStepValue)
```

기능 설명

회전 값 관리

- `GetRotation()`: 객체의 현재 회전 상태를 Quaternion으로 반환
- `SetRotation()`: 지정된 회전 값으로 객체 회전 설정

스냅 회전

- `RotationBySnapValue()`: 지정된 각도 간격으로 회전을 스냅
- 주로 사용자 인터페이스에서 정밀한 회전 조정을 위해 사용

의존성/상속 관계

- `IMyRoomEditorEditableObject`를 상속받음.
- 이동이 가능한 오브젝트들에서 구현(`SpawnableFloorAndCeilProp`)

사용 예시

회전 값 설정

```
if (editableObject.IsRotatableProp(out IRotatableProp rotatableProp))  
{  
    Quaternion newRotation = Quaternion.Euler(0, 45, 0);  
    rotatableProp.SetRotation(newRotation);  
}
```

스냅 회전 적용

```
rotatableProp.RotationBySnapValue(15f); // 15도 단위로 스냅 회전
```

회전 값 조회

```
Quaternion currentRotation = rotatableProp.GetRotation();  
float yAngle = currentRotation.eulerAngles.y;  
// 회전 각도를 UI에 표시
```

관련 클래스

- `MyRoomEditorPropRotator`
- `SpawnableFloorAndCeilProp`

SpamBotCommanderProp

- MyRoomEditorPropEditingManager

IInteractableProp

특수한 인터랙션 가능한 오브젝트 정의

개요

IInteractableProp 인터페이스는 MyRoom 에디터에서 사용자와 상호작용할 수 있는 프로퍼티를 정의합니다. 이 인터페이스를 구현하는 클래스는 Interact 메서드를 통해 특정 동작을 수행할 수 있습니다.

역할

- MyRoomEditor에서 사용자 상호작용을 처리
- 상호작용 이벤트에 대한 응답 로직 제공

선언

```
public interface IInteractableProp
```

멤버

메서드

```
/// <summary>  
/// 프로퍼티와의 상호작용을 처리하는 메서드.  
/// 구현 클래스에서 특정 상호작용 로직을 정의.  
/// </summary>  
void Interact()
```

기능 설명

상호작용 처리

- Interact()** 메서드를 통해 특정 인터랙션 오브젝트의 동작을 실행
- 구현 클래스에서 상호작용의 구체적인 동작을 정의

의존성/상속 관계

- 특수 인터렉션이 존재하는 오브젝트들에서 구현([SpawnablePhotoFrameProp](#), [SpawnableScreenProp](#))
- 확장 인터페이스들:
 - [IPhotoFrameProp](#): 사진 프레임 기능
 - [IScreenProp](#): 공유 오브젝트 기능

사용 예시

[SpawnablePropBase](#)에서 상호작용 가능 여부 확인 용도로 사용

```
// SpawnablePropBase.cs 에서 상호작용 처리
if (TryGetComponent(out IInteractableProp interactableProp))
{
    interactableProp.Interact();
}
```

관련 클래스

- [SpawnablePropBase](#)
- [MyRoomEditorInteractionManager](#)
- [IPhotoFrameProp](#)
- [IScreenProp](#)
- [SpawnablePhotoFrameProp](#)
- [SpawnableScreenProp](#)

IPhotoFrameProp

사진 프레임 정의 및 이미지 설정 기능

개요

IPhotoFrameProp 인터페이스는 MyRoomEditor에서 사진 프레임 프로퍼티의 기능을 정의합니다. 이 인터페이스를 구현하는 클래스는 사진 업로드 및 적용 기능을 제공합니다.

역할

- 사진 프레임 객체의 사진 업로드 처리
- 업로드 완료 후 이미지 적용 콜백 관리
- 사용자 상호작용을 통한 사진 변경 기능 제공

선언

```
public interface IPhotoFrameProp : IInteractableProp
```

멤버

메서드

```
/// <summary>  
/// 사진 업로드가 완료되었을 때 호출되는 메서드.  
/// 업로드된 이미지 경로를 받아서 처리.  
/// </summary>  
/// <param name="imagePath">업로드된 이미지의 파일 경로</param>  
void OnCompleteUpload(string imagePath)  
  
/// <summary>  
/// 이미지 적용 완료 콜백을 예약하는 메서드.  
/// 사진 적용이 완료되었을 때 호출될 콜백 함수를 등록.  
/// </summary>  
/// <param name="callback">적용 완료 시 호출될 콜백 함수</param>  
void ReserveCompletedApplyImageCallback(Action<MyRoomPlaceProp> callback)
```

기능 설명

사진 업로드

- `OnCompleteUpload()`로 업로드 완료된 이미지 처리
- 파일 경로를 받아 텍스트로 변환하여 프레임에 적용

콜백 관리

- `ReserveCompletedApplyImageCallback()`로 적용 완료 시 호출될 콜백 등록
- 사진 적용 후 배치 정보 업데이트 및 동기화에 사용

의존성/상속 관계

- `IInteractableProp`를 확장
- 이미지 업로드 기능을 포함한 오브젝트에서 구현(`SpawnablePhotoFrameProp`)

사용 예시

```
private void OnUploadPhoto()
{
    if (_editingObject.IsPhotoFrameProp(out var photoFrameProp))
    {
        _interactionManager.UploadImage(photoFrameProp.OnCompleteUpload);
        photoFrameProp.ReserveCompletedApplyImageCallback(UpdatePlacePropInfo);
    }
    else
    {
        Debug.Log($"{_editingObject.PropNameKey} is not interactable");
    }
}
```

관련 클래스

- `SpawnablePhotoFrameProp`
- `IInteractableProp`
- `SpawnablePhotoFrameProp`
- `MyRoomEditorPropEditingManager`
- `MyRoomEditorInteractionManager`

IScreenProp

실시간 파일 공유 모듈 정의

개요

IScreenProp 인터페이스는 MyRoomEditor에서 스크린 타입의 오브젝트를 식별하기 위한 마커 인터페이스입니다. 이 인터페이스는 **IInteractableProp**를 상속받아 상호작용 기능을 제공합니다.

역할

- 스크린 타입 프로퍼티의 식별자 역할
- 스크린 관련 오브젝트 공통 인터페이스 제공
- 상호작용 가능한 스크린 객체들의 표준화

선언

```
public interface IScreenProp : IInteractableProp
```

멤버

(상속받은 **IInteractableProp**의 메서드 사용)

기능 설명

마커 인터페이스

- IScreenProp**는 특정 기능을 정의하지 않고 타입 식별만 수행
- 스크린 관련 오브젝트들을 구분하기 위한 용도로 사용
- 컴포넌트 검색 시 타입 체크에 활용

상속된 기능

- IInteractableProp**의 **Interact()** 메서드 상속
- 스크린 오브젝트들의 기본 상호작용 기능 제공

의존성/상속 관계

- [IInteractableProp](#)를 상속받음.
- [SpawnableScreenProp](#): 실시간 파일 공유가 가능한 스크린 오브젝트들에서 구현

사용 예시

스크린 프로퍼티 타입 확인

```
if (editableObject is IScreenProp screenProp)
{
    // 스크린 타입 프로퍼티로 처리
    screenProp.Interact(); // 상호작용 실행
}
```

관련 클래스

- [SpawnableScreenProp](#)
- [IInteractableProp](#)
- [MyRoomEditorInteractionManager](#)

IPlaceableArea

배치 영역 정의 및 배치 영역 기능

개요

IPlaceableArea 인터페이스는 MyRoomEditor에서 오브젝트를 배치할 수 있는 영역을 정의합니다. 이 인터페이스를 구현하는 클래스는 배치 타입, 회전, 오브젝트 위의 배치영역 검사 기능을 제공합니다.

역할

- 배치 가능한 영역의 타입 정보 제공
- 배치 시 적용될 회전 값 정의
- 오브젝트 위의 배치영역 검사

선언

```
public interface IPlaceableArea
```

멤버

메서드

```
/// <summary>
/// 배치 영역의 타입을 반환합니다.
/// </summary>
/// <returns>배치 타입 enum 값</returns>
PlacementType GetPlacementType()

/// <summary>
/// 배치 시 적용될 회전 값을 반환합니다.
/// </summary>
/// <returns>배치 회전 Quaternion</returns>
Quaternion GetPlacementRotation()

/// <summary>
/// 해당 영역이 오브젝트에 추가된 슬롯 영역임을 체크하고, 오브젝트 슬롯 영역 일시 영역을 소유한 s
/// </summary>
/// <param name="baseProp">영역 내 프로퍼티 출력</param>
```

```
/// <param name= "baseProp" >공식 배 스폰퍼의 플렉스</param>  
/// <returns>프로퍼티가 존재하면 true</returns>  
bool IsPlacementAreaInProp(out SpawnablePropBase baseProp)
```

기능 설명

배치 타입 관리

- `GetPlacementType()`으로 영역의 배치 타입 반환
- 벽, 바닥, 천장 등 배치 가능한 표면 타입 구분

회전 적용

- `GetPlacementRotation()`으로 배치 시 회전 값 제공
- 표면 방향에 맞는 적절한 회전 적용

슬롯 배치 오브젝트 검사

- `IsPlacementAreaInProp()`으로 해당 영역이 오브젝트 위의 배치 가능한 슬롯 영역인지 판단하고 슬롯 영역일시, 해당 슬롯을 소유한 오브젝트 반환
- 중복 배치 및 충돌 검사에 사용

의존성/상속 관계

- `PlacementType`, `SpawnablePropBase` 클래스에 의존.
- `PlacementAreaProp`, `PlacementAreaInProp`: 배치 가능한 공간을 정의하는 오브젝트에서 구현

사용 예시

배치가능영역 검증 및 타입 확인

```
if (raycastHit.transform.TryGetComponent(out IPlaceableArea placeableArea)  
{  
    var type = placeableArea.GetPlacementType();  
}
```

배치영역이 오브젝트가 소유한 슬롯 영역인지 확인

```
if (editableObject.IsPlacementArea(out IPlaceableArea placeableArea))  
{  
    if (placeableArea.IsPlacementAreaInProp(out SpawnablePropBase prop))  
    {  
        // 오브젝트 위 배치 영역 처리  
        Debug.Log("배치 영역 타입: " + type + ", 오브젝트: " + prop.Id);  
    }  
}
```

관련 클래스

- [PlacementAreaProp](#)
- [PlacementAreaInProp](#)
- [MyRoomEditorPlacementManager](#)
- [MyRoomEditorPropEditingManager](#)
- [IMoveableProp](#)

PlacementAreaProp

바닥, 천장, 벽면과 같은 배치 영역을 정의하는 클래스

개요

`PlacementAreaProp`는 MyRoomEditor에서 룸 레벨의 배치 가능 영역을 정의하는 클래스입니다. 벽, 바닥, 천장 등의 룸 구조물을 나타내며, `IPlaceableArea` 인터페이스를 구현합니다. 오브젝트 배치의 기반이 되는 영역입니다.

역할

- 룸 레벨 배치 영역 정의 (벽, 바닥, 천장)
- 배치 타입 및 회전 정보 제공
- 프로퍼티 배치 정보 저장/로드
- 편집 가능 인터페이스 구현 (제한적)

선언

```
public class PlacementAreaProp : MonoBehaviour, IPlaceableArea, IColorEditable
```

멤버

속성

```
/// <summary>  
/// 배치 영역 타입 (바닥, 벽, 천장 등)  
/// </summary>  
[SerializeField]  
private PlacementType placementAreaType;  
  
/// <summary>  
/// 배치 시 적용될 회전 값  
/// </summary>  
[SerializeField]  
private Quaternion propRotation;
```

메서드

```
/// <summary>
/// 배치 영역 타입을 반환.
/// </summary>
public PlacementType GetPlacementType()

/// <summary>
/// 배치 시 적용될 회전 값을 반환.
/// </summary>
public Quaternion GetPlacementRotation()

/// <summary>
/// 해당 컴포넌트가 `PlacementAreaInProp`(오브젝트 중복배치 영역)인지 확인하고 부모 오브젝트
/// </summary>
public bool IsPlacementAreaInProp(out SpawnablePropBase baseProp)
```

기능 설명

룸 구조물 표현

- 벽, 바닥, 천장 등의 룸 레벨 배치 영역 정의
- 배치 타입과 회전에 따라 다른 오브젝트 배치 가능

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `IPlaceableArea` 인터페이스 구현.
- `Collider` 컴포넌트 필요 (RequireComponent).
- `PlacementType` 열거형 사용.

사용 예시

오브젝트 영역 검증 단계에서 회전 값 및 타입/슬롯영역인지 확인

```
public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)
{
    Move(hitPosition);
    transform.rotation = area.GetPlacementRotation();

    if (area.GetPlacementType() != PlacementType)
    {
        _propEditingState.Invalid();
        return false;
    }
    if (area.IsPlacementAreaInProp(out var parent))
    {
        SetPropParent(parent);
    }
}
```

```
    }  
    else  
    {  
        ClearPropParent();  
    }  
  
    _propEditingState.Valid();  
    return true;  
}
```

관련 클래스

- PlacementType: 배치 타입 열거형
- IPlaceableArea: 배치 영역 인터페이스

PlacementAreaInProp

책상, 탁자 등 오브젝트 위에 배치되는 영역을 관리하는 클래스

개요

`PlacementAreaInProp`는 오브젝트에 추가 배치 가능한 슬롯 영역(책상 위 등)을 정의하는 클래스입니다.
`IPlaceableArea` 인터페이스를 구현하여 슬롯 위치에 다른 오브젝트를 배치할 수 있도록 합니다. 배치 타입에 따라 적절한 레이어로 설정됩니다.

역할

- 오브젝트에 추가 배치 슬롯 영역 정의
- 배치 타입에 따른 레이어 설정
- 부모 프로퍼티 참조 관리
- 배치 가능 여부 및 위치 정보 제공

선언

```
public class PlacementAreaInProp : MonoBehaviour, IPlaceableArea
```

멤버

속성

```
/// <summary>  
/// 배치 영역 타입 (Floor, Wall, Ceiling 등)  
/// </summary>  
[SerializeField]  
private PlacementType _placementType;  
  
/// <summary>  
/// 이 배치 영역이 속한 오브젝트  
/// </summary>  
private SpawnablePropBase _baseParent;
```

메서드

```

/// <summary>
/// 슬롯을 가진 오브젝트와 배치 영역을 연결하고 레이어를 설정.
/// </summary>
public void SetPlacementAreaInProp(SpawnablePropBase parent)

/// <summary>
/// 배치 영역 타입을 반환.
/// </summary>
public PlacementType GetPlacementType()

/// <summary>
/// 배치 시 적용될 회전 값을 반환.
/// </summary>
public Quaternion GetPlacementRotation()

/// <summary>
/// 해당 배치 가능 영역이 `PlacementAreaInProp`(오브젝트 중복배치 영역)인지 확인하고 오브젝트
/// </summary>
public bool IsPlacementAreaInProp(out SpawnablePropBase baseProp)

```

기능 설명

배치 정보 제공

- 배치 타입 정보 제공
- 배치 회전 정보 제공 (transform.rotation)
- 해당 영역을 소유한 슬롯 오브젝트 반환.

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `IPlaceableArea` 인터페이스 구현.
- `PlacementType` 열거형 사용.
- `SpawnablePropBase`에 의존.
- - `Collider` 컴포넌트 필요 (RequireComponent).

사용 예시

오브젝트 영역 검증 단계에서 회전 값 및 타입/슬롯영역인지 확인

```

public bool IsPlaceableArea(Vector3 hitPosition, IPlaceableArea area)
{
    Move(hitPosition);
    transform.rotation = area.GetPlacementRotation();

    if (area.GetPlacementType() != PlacementType)

```

```
{
    _propEditingState.Invalid();
    return false;
}
if (area.IsPlacementAreaInProp(out var parent))
{
    SetPropParent(parent);
}
else
{
    ClearPropParent();
}

_propEditingState.Valid();
return true;
}
```

관련 클래스

- [PlacementAreaProp](#)
- [IPlaceableArea](#)
- [SpawnablePropBase](#)
- [PlacementType](#)

MyRoomEditorObjectEditUI

오브젝트 편집 메뉴를 표시하고 조작하는 UI 클래스

개요

`MyRoomEditorObjectEditUI`는 MyRoomEditor에서 선택된 오브젝트의 편집 UI를 관리하는 클래스입니다. Unity UI 시스템을 사용하여 편집 메뉴를 표시하고, 색상 변경, 이동, 회전, 삭제, 이미지 업로드 등의 기능을 제공합니다. 이벤트 기반 아키텍처를 통해 외부 매니저와 통신하며, 툴팁과 시각적 피드백을 지원합니다.

역할

- 선택된 프로퍼티의 편집 메뉴 표시 및 숨김
- 색상 선택 UI와의 통합 관리
- 편집 도구 상태 관리 (색상 편집, 이동, 자유 회전)
- 버튼 호버/클릭 이벤트 처리 및 시각적 피드백 제공
- 프로퍼티 타입에 따른 메뉴 아이템 활성화/비활성화
- 오브젝트 삭제 기능 제공
- 툴팁 표시 및 위치 조정

선언

```
public class MyRoomEditorObjectEditUI : MonoBehaviour
```

멤버

열거형

```
/// <summary>  
/// 활성 편집 도구 열거형.  
/// 현재 활성화된 편집 도구의 상태를 나타냄.  
/// </summary>  
public enum ActiveEditingTool  
{  
    None = 0,        // 활성 도구 없음  
    EditColor,       // 색상 편집 모드  
    Move,            // 이동 모드
```



```

        RotateFree,    // 자유 회전 모드
    }
}

```

이벤트(각각의 편집 기능 버튼 클릭시 발생하는 이벤트)

```

public event Action OnClickDeleteEvent;
public event Action OnClickEditColorEvent;
public event Action<int> OnSelectColorToChange;
public event Action OnClickMoveEvent;
public event Action OnClickRotateQuarterEvent;
public event Action OnClickRotateFreeEvent;
public event Action OnClickUploadPhotoEvent;

```

속성

```

/// <summary>
/// 메뉴 게임 오브젝트: 편집 메뉴의 루트 오브젝트
/// </summary>
[SerializeField]
private GameObject menuGameObject;

/// <summary>
/// 색상 선택 UI: 색상 변경을 위한 UI 컴포넌트
/// </summary>
[SerializeField]
private MyRoomEditorColorSelectUI colorSelectUI;

/// <summary>
/// 편집 버튼들: 삭제, 색상 편집, 이동, 회전, 이미지 업로드 버튼
/// </summary>
[SerializeField]
private Button deleteBtn, editColorBtn, moveBtn, rotateQuarterBtn, rotateFreeBtn;

/// <summary>
/// 현재 선택한 오브젝트 레퍼런스 캐싱.
/// </summary>
private IMyRoomEditorEditableObject _selectedProp;

/// <summary>
/// 편집 버튼 리스트
/// </summary>
private List<Button> _editButtons;

/// <summary>
/// 메뉴 상태별 스프라이트
/// </summary>
[SerializeField]
private Sprite normalSprite, hoverSprite, pressedSprite, selectedSprite, disabledSprite;

/// <summary>

```

```

/// 툴팁 오브젝트 RectTransform
/// </summary>
[SerializeField]
private RectTransform hoverTooltip, activeTooltip;

```

메서드

```

/// <summary>
/// 선택된 오브젝트의 편집 메뉴를 표시하는 메서드.
/// 오브젝트의 편집 가능 기능에 따라 UI 요소들을 활성화/비활성화.
/// </summary>
/// <param name="selectedProp">선택된 편집 가능한 오브젝트</param>
public void ShowSelectedPropMenu(IMyRoomEditorEditableObject selectedProp)

/// <summary>
/// 편집 도구 변경 시 UI 상태를 업데이트하는 프라이빗 메서드.
/// 활성 도구에 따라 버튼 상태와 툴팁을 변경.
/// </summary>
private void EditingToolChanged()

/// <summary>
/// 사용 가능한 메뉴 항목들을 활성화하는 프라이빗 메서드.
/// 오브젝트의 기능 지원 여부에 따라 버튼들을 활성화/비활성화.
/// </summary>
/// <param name="colorChangeable">색상 변경 가능 여부</param>
/// <param name="rotatable">회전 가능 여부</param>
/// <param name="moveable">이동 가능 여부</param>
/// <param name="imgUploadAble">이미지 업로드 가능 여부</param>
/// <param name="useNamePanel">이름 패널 사용 여부</param>
private void ActiveUsableMenuItem(bool colorChangeable, bool rotatable, bool moveable, bool imgUploadAble, bool useNamePanel)

/// <summary>
/// 색상 리스트를 그리는 메서드.
/// 현재 선택된 색상을 찾아 색상 선택 UI에 표시.
/// </summary>
/// <param name="colorList">표시할 색상 리스트</param>
public void DrawColorList(List<MyRoomPropColor> colorList)

```

코드 스니펫

선택 오브젝트 편집 메뉴 활성화

```

public void ShowSelectedPropMenu(IMyRoomEditorEditableObject selectedProp)
{
    // 메뉴 활성화 및 초기화
    menuGameObject.SetActive(true);
    _activeTool = ActiveEditingTool.None;
    EditingToolChanged();

    // 선택된 오브젝트 저장
}

```

```

        _selectedProp = selectedProp;

        // 오브젝트의 편집 가능 기능 확인
        var colorChangeable = _selectedProp.IsEditableColor(out _);
        var rotatable = _selectedProp.IsRotatableProp(out _);
        var moveable = _selectedProp.IsMovableProp(out _);
        var imgUploadAble = _selectedProp.IsPhotoFrameProp(out _);
        var useNamePanel = _selectedProp.IsPlacementArea(out _) == false;

        // 이름 패널 설정 (배치 영역이 아닌 경우)
        if (useNamePanel)
        {
            _localizedText.TranslationName = _selectedProp.PropNameKey;
        }

        // 사용 가능한 메뉴 항목 활성화
        ActiveUsableMenuItem(colorChangeable, rotatable, moveable, imgUploadAble, useNamePanel);
    }

    private void EditingToolChanged()
    {
        // 툴팁 비활성화
        hoverTooltip.gameObject.SetActive(false);

        // 도구가 없는 경우
        if (_activeTool == ActiveEditingTool.None)
        {
            // 모든 버튼 활성화
            EnableAllButtons();
            activeTooltip.gameObject.SetActive(false);
            hoverTooltip.gameObject.SetActive(false);
            colorSelectUI.gameObject.SetActive(false);
            return;
        }

        // 활성 도구에 따라 대상 버튼 설정
        var targetBtn = _activeTool switch
        {
            ActiveEditingTool.EditColor => editColorBtn,
            ActiveEditingTool.RotateFree => rotateFreeBtn,
            ActiveEditingTool.Move => moveBtn,
        };

        // 대상 버튼 선택 상태로 설정
        targetBtn.image.sprite = selectedSprite;
        targetBtn.interactable = true;

        // 다른 버튼들은 비활성화
        var disableBtn = _editButtons.FindAll(btn => btn != targetBtn);
        DisableButtons(disableBtn);

        // 활성 툴팁 표시
        ShowToolTip(activeTooltip, targetBtn.transform.position);
    }

```

```
private void ActiveUsableMenuItem(bool colorChangeable, bool rotatable, bool moveable)
{
    // 각 버튼의 활성화 상태 설정
    editColorBtn.gameObject.SetActive(colorChangeable);
    rotateQuarterBtn.gameObject.SetActive(rotatable);
    moveBtn.gameObject.SetActive(moveable);
    rotateFreeBtn.gameObject.SetActive(rotatable);
    imgUploadBtn.gameObject.SetActive(imgUploadAble);
    propNamePanel.gameObject.SetActive(userNamePanel);

    // 모든 버튼 활성화
    EnableAllButtons();
}
```

컬러 변경 UI 활성화

```
public void DrawColorList(List<MyRoomPropColor> colorList)
{
    var currentColorKey = _selectedProp.GetPlacePropInfo().colorKey;
    var currentColorIndex = colorList.FindIndex(key => key.materialKey == currentColorKey);
    colorSelectUI.OnDrawColorList(colorList, currentColorIndex);
    colorSelectUI.gameObject.SetActive(true);
}
```

버튼 상태 관리

```
private void EnableAllButtons()
{
    _editButtons.ForEach(button =>
    {
        button.interactable = true;
        button.image.sprite = normalSprite;
        SetChildImageAlpha(button, _enabledIconColor);
    });
}

private void DisableButtons(List<Button> buttons)
{
    buttons.ForEach(button =>
    {
        button.interactable = false;
        button.image.sprite = disabledSprite;
        SetChildImageAlpha(button, _disabledIconColor);
    });
}
```

이벤트 트리거 추가 (툴팁 표시 및 숨김)

```

private void AddListeners(Button button)
{
    EventTrigger trigger = button.gameObject.AddComponent<EventTrigger>();

    EventTrigger.Entry pointerEnterEntry = new EventTrigger.Entry { eventID = EventID.PointerEnter;
    pointerEnterEntry.callback.AddListener(data => OnPointerEnterEvent(button));
    trigger.triggers.Add(pointerEnterEntry);

    EventTrigger.Entry pointerExitEntry = new EventTrigger.Entry { eventID = EventID.PointerExit;
    pointerExitEntry.callback.AddListener(data => OnPointerExitEvent(button));
    trigger.triggers.Add(pointerExitEntry);
}

private void OnPointerEnterEvent(Button targetBtn)
{
    if (_activeTool != ActiveEditingTool.None) return;
    targetBtn.image.sprite = hoverSprite;
    if (_additionalStatusButtons.Contains(targetBtn))
    {
        ShowToolTip(hoverToolTip, targetBtn.transform.position);
    }
}

private void ShowToolTip(RectTransform targetToolTip, Vector2 position)
{
    targetToolTip.gameObject.SetActive(true);
    targetToolTip.position = new Vector3(position.x, targetToolTip.position.y, 0);
}

private void OnPointerExitEvent(Button targetBtn)
{
    if (_activeTool != ActiveEditingTool.None) return;
    targetBtn.image.sprite = normalSprite;
    hoverToolTip.gameObject.SetActive(false);
}

```

기능 설명

UI 표시/숨김 관리

- **ShowSelectedPropMenu():** 선택된 프로퍼티의 타입을 확인하여 적절한 편집 메뉴를 표시. 색상 변경, 회전, 이동, 이미지 업로드 기능의 사용 가능 여부를 체크.
- **HideMenu():** 모든 메뉴와 툴팁을 숨기고 상태를 초기화합니다.

편집 도구 토글

- 색상 편집, 이동, 자유 회전 도구를 토글 방식으로 활성화/비활성화. 하나의 도구만 활성화.

시각적 피드백

- 버튼 호버 시 스프라이트 변경과 툴팁 표시를 통해 사용자 경험을 향상.
- 선택된 도구는 다른 버튼들 비활성화하고 시각적으로 강조

- 연결된 모든 다른 비드를 비활성화하고 시각적으로 강조.

이벤트 처리

- 각 버튼 클릭 시 해당 이벤트를 발생시켜 외부 매니저에서 동작.
- 색상 선택 UI와의 통합을 통해 색상 변경 기능을 제공.

현지화

- [Lean Localization](#) [↗](#)을 통한 텍스트 현지화

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- [UGUI](#) [↗](#), [TextMesh Pro](#) [↗](#), [Lean Localization](#) [↗](#), Event System을 사용.
- [IMyRoomEditorEditableObject](#), [MyRoomEditorColorSelectUI](#)에 의존.

사용 예시

[MyRoomEditorPropEditingManager](#)에서 오브젝트가 선택 될 때, 오브젝트 편집 UI표시

```
private void OnObjectSelected(IMyRoomEditorEditableObject selectedObject)
{
    _editingObject = selectedObject;
    objectEditUI.ShowSelectedPropMenu(_editingObject);
}
```

관련 클래스

- [MyRoomEditorPropEditingManager](#)
- [MyRoomEditorPlacementManager](#)
- [MyRoomEditorColorSelectUI](#)
- [IMyRoomEditorEditableObject](#)
- [UGUI](#) [↗](#)
- [TextMesh Pro](#) [↗](#)
- [Lean Localization](#) [↗](#)

MyRoomEditorColorSelectUI

색상 선택 UI를 처리하는 클래스

개요

`MyRoomEditorColorSelectUI` 클래스는 `MyRoomEditor`에서 색상 선택 UI를 관리하는 컴포넌트입니다. 색상 리스트를 표시하고 사용자의 선택을 처리하며, 선택된 색상 인덱스를 이벤트로 전달합니다.

역할

- 색상 아이템 리스트 관리 및 표시
- 색상 선택 상태 관리
- 선택된 색상 이벤트 발생
- UI 초기화 및 상태 동기화

선언

```
public class MyRoomEditorColorSelectUI : MonoBehaviour
```

멤버

이벤트

```
/// <summary>  
/// 색상 선택 이벤트: 색상이 선택되었을 때 발생  
/// </summary>  
public event Action<int> OnSelectedColor;
```

속성

```
/// <summary>  
/// 색상 편집 항목 리스트: 개별 색상을 표시하는 UI 항목들  
/// </summary>  
[SerializeField]  
private List<MyRoomEditorColorEditItem> colorEditItems;
```

메서드

```
/// <summary>
/// 색상 리스트를 받아 UI에 표시하는 메서드.
/// 제공된 색상 리스트를 UI 항목에 할당하고 현재 선택된 색상을 하이라이트.
/// </summary>
/// <param name="colorList">표시할 색상 리스트</param>
/// <param name="currentColorIndex">현재 선택된 색상의 인덱스</param>
public void OnDrawColorList(List<MyRoomPropColor> colorList, int currentColorIndex)

/// <summary>
/// 색상 선택 이벤트 핸들러.
/// 특정 색상이 선택되었을 때 다른 모든 항목을 해제하고 선택 이벤트를 발생.
/// </summary>
/// <param name="index">선택된 색상의 인덱스</param>
private void OnSelectMaterialColor(int index)
```

코드 스니펫

색상 리스트 UI 표시

```
public void OnDrawColorList(List<MyRoomPropColor> colorList, int currentColorIndex)
{
    // 모든 항목을 초기화 상태로 설정
    InitializeListItem();

    // 각 색상을 UI 항목에 할당
    for (int colorListIndex = 0; colorListIndex < colorList.Count; colorListIndex++)
    {
        var uiItem = colorEditItems[colorListIndex];
        uiItem.gameObject.SetActive(true);
        uiItem.SetItemColor(colorList[colorListIndex].color);

        // 현재 선택된 색상인 경우 선택 상태로 표시
        if (colorListIndex == currentColorIndex) uiItem.Selected();
    }
}

private void InitializeListItem()
{
    colorEditItems.ForEach(item => item.Release());
    colorEditItems.ForEach(item => item.gameObject.SetActive(false));
}
```

색상 선택 이벤트


```
private void OnSelectMaterialColor(int index)
{
    // 활성화된 모든 항목을 해제
    colorEditItems.Where(item => item.gameObject.activeSelf).ToList().ForEach(i

    // 선택된 항목을 선택 상태로 변경
    colorEditItems[index].Selected();

    // 외부 이벤트 발생
    OnSelectedColor?.Invoke(index);
}
```

기능 설명

색상 리스트 표시

- `OnDrawColorList()`: 제공된 색상 리스트를 UI 아이템에 바인딩
- 현재 선택된 색상 인덱스에 따라 선택 상태 표시

선택 상태 관리

- 단일 선택 모드: 하나의 색상만 선택 가능
- 선택 변경 시 이전 선택 해제 및 새로운 선택 활성화
- 현재 적용된 색상을 시각적으로 구분하여 표시

이벤트 처리

- 색상 선택 시 인덱스를 이벤트로 전달
- 부모 컴포넌트가 선택 처리 수행

초기화

- 색상 항목들을 초기 설정하고 이벤트 연결
- 선택 해제 시 모든 항목을 초기 상태로 복원
- 리스트 변경 시 기존 항목들을 재활용하여 성능 최적화

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- `MyRoomEditorColorEditItem`에 의존.
- `MyRoomPropColor` 클래스 사용.

관련 클래스

- `MyRoomEditorColorEditItem`
- `MyRoomEditorObjectEditUI`
- `IColorEditableProp`

MyRoomEditorColorEditItem

선택 가능한 색상 항목을 나타내는 UI 컴포넌트

개요

`MyRoomEditorColorEditItem` 클래스는 MyRoom 에디터에서 색상 선택 UI의 개별 아이템을 관리하는 컴포넌트입니다. 색상 패널 표시와 선택 상태 관리를 담당하며, 색상 선택 이벤트를 발생시킵니다.

역할

- 색상 선택 UI 아이템 표시 및 상호작용 처리
- 선택 상태 시각적 피드백 제공
- 색상 값 설정 및 표시
- 색상 선택 이벤트 발생

선언

```
public class MyRoomEditorColorEditItem : MonoBehaviour
```

멤버

이벤트

```
/// <summary>  
/// 색상 선택 이벤트 (아이템 인덱스 전달)  
/// </summary>  
public event Action<int> OnSelectMaterialColorEvent;
```

속성

```
/// <summary>  
/// 색상 선택 버튼  
/// </summary>  
[SerializeField]  
private Button selectColorButton;
```

```

/// <summary>
/// 선택 상태 표시 오브젝트
/// </summary>
[SerializeField]
private GameObject selectedObject;

/// <summary>
/// 색상 패널 이미지
/// </summary>
[SerializeField]
private Image colorPanel;

```

메서드

```

/// <summary>
/// 색상 항목을 초기화하는 메서드.
/// 항목의 인덱스를 설정하여 선택 시 올바른 인덱스를 전달할 수 있도록 함.
/// </summary>
/// <param name="index">이 색상 항목의 인덱스</param>
public void Initialize(int index)

/// <summary>
/// 색상 항목을 선택된 상태로 변경하는 메서드.
/// 선택 상태 표시 오브젝트를 활성화하여 사용자가 선택된 항목을 시각적으로 확인할 수 있도록 함.
/// </summary>
public void Selected()

/// <summary>
/// 색상 항목의 색상을 설정하는 메서드.
/// 색상 패널의 색상을 지정된 색상으로 변경하여 사용자가 선택 가능한 색상을 미리 볼 수 있도록 함.
/// </summary>
/// <param name="color">표시할 색상</param>
public void SetItemColor(Color color)

/// <summary>
/// 버튼 클릭 이벤트 핸들러.
/// 색상 항목이 클릭되었을 때 선택 이벤트를 발생시키고 항목 인덱스를 전달.
/// </summary>
private void OnClick()

/// <summary>
/// 색상 항목을 해제하는 메서드.
/// 선택 상태 표시 오브젝트를 비활성화하여 선택 해제 상태로 변경.
/// </summary>
public void Release()

```

기능 설명

색상 표시

- `SetItemColor()`: 지정된 색상으로 패널 색상 설정
- UI를 통해 사용 가능한 색상 옵션 표시

선택 상태 관리

- `Selected()`: 선택 시 시각적 표시 활성화
- `Release()`: 선택 해제 시 시각적 표시 비활성화

이벤트 처리

- 버튼 클릭 시 인덱스와 함께 이벤트 발생
- 부모 UI 컴포넌트가 선택 처리 수행

초기화

- `Initialize()`: 아이템 인덱스 설정으로 고유 식별

의존성/상속 관계

- `MonoBehaviour`를 상속받음.
- [UGUI](#)  컴포넌트 사용 (Button, Image).

관련 클래스

- [MyRoomEditorColorSelectUI](#)
- [IColorEditableProp](#)
- [UGUI](#) 

PlacementType

오브젝트 배치 타입을 정의하는 열거형

개요

`PlacementType`은 MyRoomEditor에서 오브젝트 배치 타입을 정의하는 열거형입니다. 오브젝트가 배치될 수 있는 영역의 종류를 지정하며, 레이어 설정과 배치 로직에 사용됩니다.

역할

- 프로퍼티 배치 영역 타입 정의
- 배치 가능 영역의 분류 기준 제공
- 레이어 및 배치 검증 로직에 활용

선언

```
public enum PlacementType
```

열거형 값

```
public enum PlacementType
{
    Floor,      // 바닥
    Wall,       // 벽
    Ceiling,    // 천장
    Skybox      // 배경 전망
}
```

값 설명

Floor (바닥)

- 설명:** 바닥면에 배치되는 오브젝트 타입
- 특징:** 자유로운 이동과 회전이 가능
- 예시:** 가구, 장식품, 러그 등

Wall (벽)

- **설명:** 벽면에 부착되는 오브젝트 타입
- **특징:** 벽면 고정, 회전 제한, 방향 기반 기즈모
- **예시:** 그림, 벽걸이 장식, 스크린 등

Ceiling (천장)

- **설명:** 천장면에 배치되는 오브젝트 타입
- **특징:** 자유로운 이동과 회전이 가능
- **예시:** 천장 조명, 천장 장식 등

Skybox (배경 전망)

- **설명:** 배경 전망을 위한 특별한 배치 타입
- **특징:** 환경 배경 요소
- **예시:** 창문, 배경 풍경 등

관련 클래스

- [IPlaceableArea](#)
- [PlacementAreaProp](#)
- [PlacementAreaInProp](#)
- [MyRoomProp](#)